

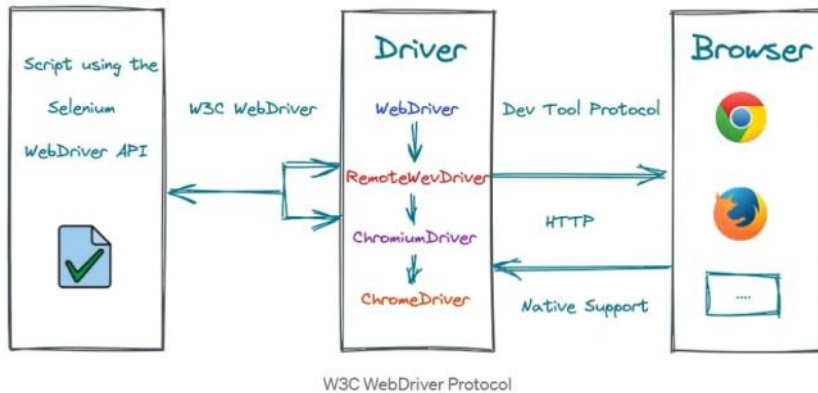
Introduction

Monday, May 21, 2018 12:32 PM

Selenium WebDriver is an open-source, cross-platform library designed to help automate browser testing. It defines set of methods but does not implements it.

How it Works

- You write your test script in your preferred language using methods like `findElement` provided by the language binding.
- The language binding translates these methods into commands that align with the WebDriver interface.
- The WebDriver implementation sends these commands to the appropriate browser driver.
- The browser driver interacts with the targeted browser to perform the actions (finding elements, clicking buttons, etc.).



W3C Webdriver: In Selenium 3.0, tests communicated with the browser at the end node through the JSON wire protocol at the local end, necessitating API encoding and decoding.

In Selenium 4, tests directly communicate through the W3C Protocol without encoding or decoding API requests. There is a direct information exchange between the client and server, without the need for the JSON Wire Protocol. As Selenium WebDriver and web browsers use the same protocol, automated Selenium testing will execute tests more consistently between different browsers. Flakiness in web automation can be considerably reduced if a standard protocol is used. JSON wire protocol was replaced with W3C protocol because browsers were following W3C standards, their executables were also compliant to W3C standards

Browser Drivers: Selenium uses drivers, specific to each browser in order to establish a secure connection with the browser without revealing the internal logic of browser's functionality.

- The **W3C WebDriver Protocol** is a RESTful API that uses JSON over HTTP. It is a standardized set of rules that defines how an external program (like your Selenium script) should communicate with a web browser. Every action in your code is converted into a standard web request.
- It defines *what* a "Click" command looks like in JSON, but Chrome's **Blink** engine and Firefox's **Gecko** engine execute that click using entirely different low-level system calls.
- W3C is the "Language" (the protocol), but the Executable is the "Translator/Interface" (the driver).

Key Components: The protocol organizes browser control into specific "endpoints":

- **Sessions:** Starting/stopping the browser.
- **Navigation:** Commands for get, back, forward, and refresh.
- **Elements:** Finding, clicking, and typing into HTML elements.
- **Cookies:** Standardized ways to manage browser data.

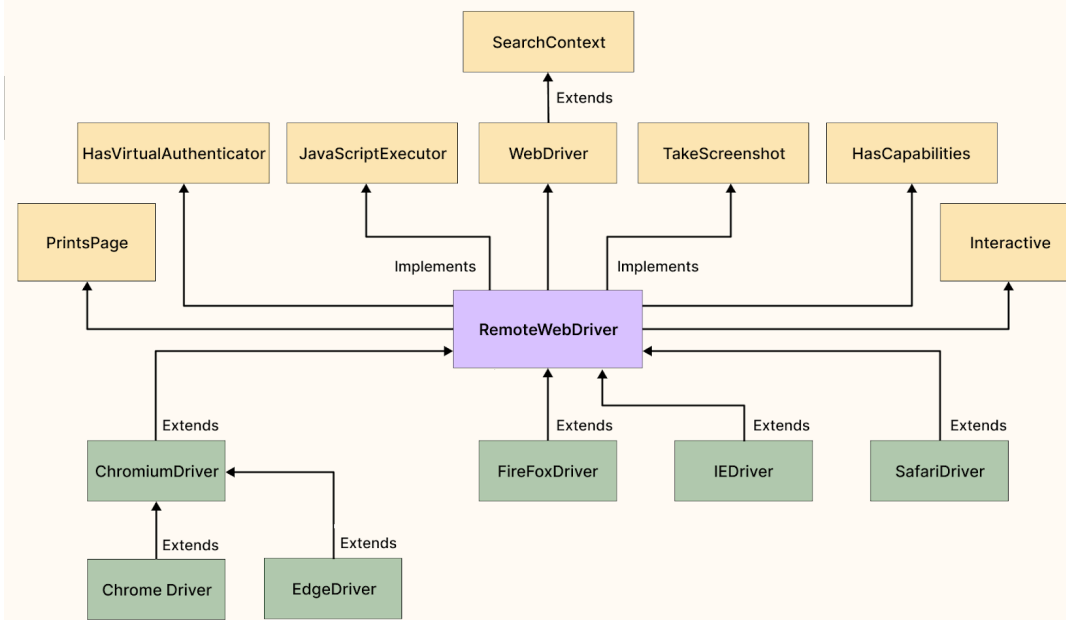
Browsers are designed to be "sandboxed" for safety. They don't allow external scripts to reach in and grab control of your mouse or keyboard.

The Middleman: The Browser Executable is a trusted tool created by the browser vendors themselves. It has the unique permissions required to bypass the sandbox and manipulate the browser safely.

Selenium Manager: It automatically detects your browser version and downloads the matching driver in the background. When we use `System.setProperty("webdriver.chrome.driver", "path")`, Java sets a JVM-wide system property that Selenium checks to locate the `ChromeDriver` executable. Selenium Manager, introduced in Selenium 4.1.1, steps in only if you *don't* set this property—it automatically downloads, installs, and manages the right driver version for your browser. When you call `new ChromeDriver()`, Selenium checks this property. If found, it uses the provided path to launch the driver process.

WebDriver Hierarchy

Selenium WebDriver Hierarchy



RemoteWebDriver Class

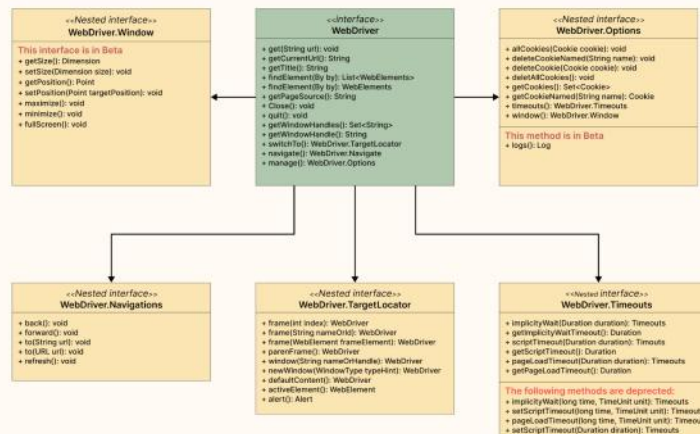
It is a fully implemented WebDriver Interface class extended by every BrowserDriver class within the Selenium framework. It enables you to execute your test scripts on remote machines or browsers. Most Frequently used RemoteWebDriver Class Constructors: RemoteWebDriver(Uri, ICapabilities)

RemoteWebdriver class implements the following interfaces:

WebDriver Interface

WebDriver Interface is the core of the Selenium WebDriver as it has all the required methods and respective nested interfaces defined within it, which helps in simulating user actions inside the browser.

Abstract methods and Nested Interfaces in WebDriver Interface

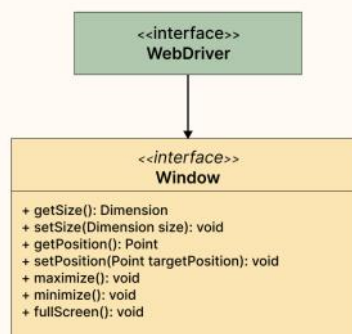


Nested Interfaces within WebDriver Interface

Window Interface

This interface has all the methods that help manage the current window.
Usage: driver.manage().window().

Nested interface within WebDriver interface WebDriver.Window



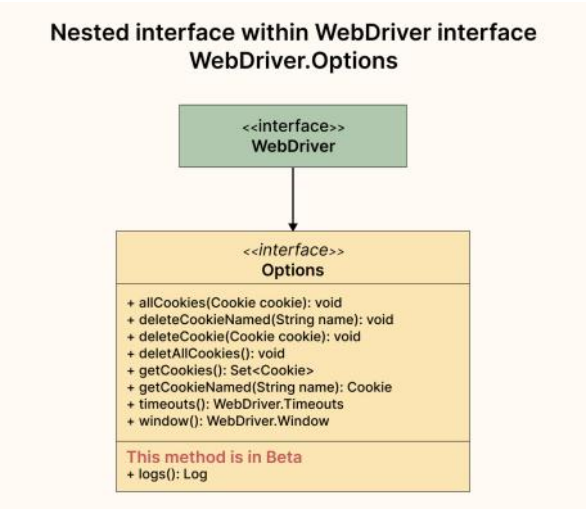
Options Interface

This interface has all the methods to help manage the stuff in a browser menu. With the help of this interface, we perform the following actions:

Add, get, and delete a cookie
Set timeouts in the browser
Manage window
Fetch different types of logs.

Usage: driver.manage().window().

driver.manage(): Accesses the Options interface of WebDriver instance. The Options interface configures various settings for the browser session. Think of it as the place where you tweak the browser's behavior.

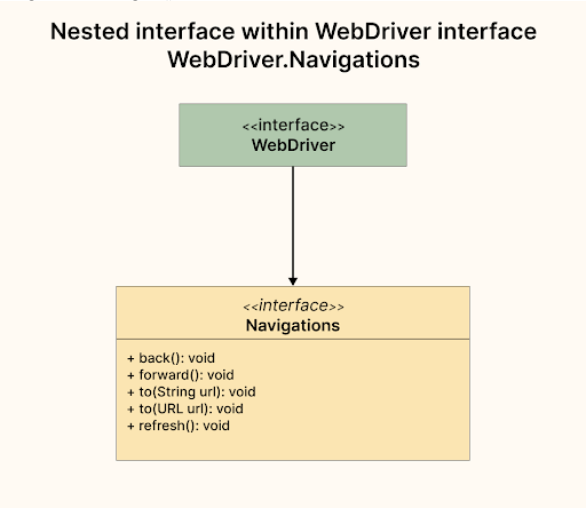


Navigation
Interface

This interface has all the methods to access the browser's history and navigate to a URL. With the help of this interface, we perform the following actions:

Navigate Back, Forward in the browser
Navigate to a URL in the browser
Refresh the WebPage

Usage: driver.navigate().



TargetLocator
Interface

This interface has all the methods to send future commands to different frames and windows. With the help of this interface, we perform the following actions:

Working with different Frames.
Working with different windows or Tabs in the browser.
Working with different Alerts in the browser.

Usage: driver.switchTo().

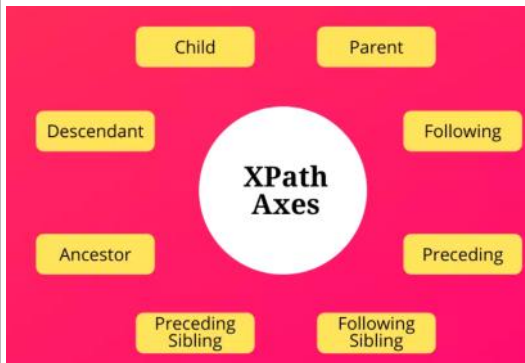
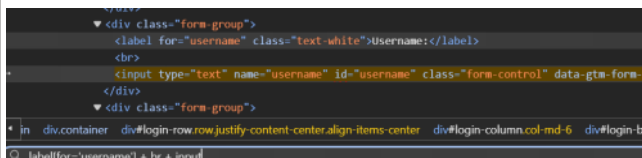
Abstract methods in WebElement Interface

<<interface>> Options
+ click(): void + submit(): void + sendKeys(CharSequence.. keysToSend): void + Clear(): void + getTagName(): String + getDomProperty(String name): String + getDomAttribute(String name): String + getAttribute(String name): String + getAriarole(): String + getAccessileName(): String + isSelected(): boolean + isEnabled: boolean + isDisplayed(): boolean + getText(): String + findElement(By by): List<WebElement> + findElements(By by): WebElement + getShadowRoot(): SearchContext + getLocation(): Point + getSize(): Dimension + getRect(): Rectangle + getCssValue(String propertyName): String

Search Context	SearchContext is the topmost interface in Selenium WebDriver that provides the base for element searching. It defines the method used to find web elements on a page but does not itself provide implementation. • It contains only two methods: findElement() and findElements(). • WebDriver, WebElement, and ShadowRoot implement SearchContext. • This enables element searching at both the page and element level.
----------------	--

Locators

21 May 2018 13:03

Locators Supported by Selenium	1. ID 2. ClassName 3. Name 4. LinkText 5. Xpath 6. CssSelector 7. TagName 8. Partial LinkText									
XPath Examples	- //input[@value='Log In']: replace input with * for wildcard - //td[@class='headertext'][@align='left'] - Child XPath: <code>//*[@id='ctl00_mainContent_ddl_destinationStation1_CTNR']//a[@value='AMD']</code> - //input[@type='submit' and @name='submitButton'] - //input[@type='submit' or @type='button']									
Traversing using XPath	- Sibling to Sibling: <code>//*[@attribute='value']/following-sibling::li[2]</code> - Child to Parent: <code>//*[@attribute='value']/parent::ul</code>									
Xpath: Normalize-space()	Removes Leading and Trailing Whitespace, Collapses Consecutive Whitespace <code>//button[normalize-space(text())='Submit Form']</code>									
Xpath Union Operator	List<WebElement> products = driver.findElements(By.xpath("//*[@id='product']/tbody/tr")); for (WebElement product : products) { List<WebElement> cells = product.findElements(<u>By.xpath("./th ./td")</u>); Other way: <code>//*[@id='product']/tbody/tr/*[self::td or self::tr]</code>									
Xpath Axes	XPath axes are used to specify the node selection relationship between nodes in an XML document. They define the direction in which the nodes are selected relative to the current node.  -WebElement lastInputField = driver.findElement(By.xpath("//input[last()-1]"));									
Locating using Text	- <code>//*[text()='Selenium']</code> - <code>//a[contains(text(),'Samual')]</code>									
Xpath Indexing	- <code>(//*[@class='g'])[2]</code>									
CSS Selector Examples	- input[value='Log In']: skip input for wildcard - tr[class='active'] td:nth-child(2): Accessing specific table cell using CSS Selector									
Counting Logic in Xpath and CSS	CSS :nth-child(n): Is Position-Based. It counts every element inside the parent. If the 3rd element isn't what you described, the locator fails. XPath [n]: Is Filter-Based. It finds all elements that match your criteria first, then picks the n^{th} one from that specific list. <pre><form> <h2>Forgot password</h2> <input type="text" placeholder="Name"> <input type="text" placeholder="Email"> </form></pre> <table><tr><th>Tool</th><th>Locator</th><th>Logic</th></tr><tr><td>CSS</td><td>input:nth-child(3)</td><td>"Find the 3rd child of the form. Verify it is an input."</td></tr><tr><td>XPath</td><td>//input[2]</td><td>"Find all inputs. Pick the 2nd one from that list."</td></tr></table> To make CSS act like XPath (and avoid counting the <h2>), use :nth-of-type i.e. input:nth-of-type(2) — This ignores the <h2> and only counts the input "siblings," making it much more stable if headers are added or removed later.	Tool	Locator	Logic	CSS	input:nth-child(3)	"Find the 3rd child of the form. Verify it is an input."	XPath	//input[2]	"Find all inputs. Pick the 2nd one from that list."
Tool	Locator	Logic								
CSS	input:nth-child(3)	"Find the 3rd child of the form. Verify it is an input."								
XPath	//input[2]	"Find all inputs. Pick the 2nd one from that list."								
Traversing using CSS	- Parent to child: parenttagname childtagname (just a space is required in CSS, whereas / is used in XPath) 									

Using Regular Expressions	- <code>//input[contains(@value,'Log In')]</code>: Contains, starts-with, ends-with - <code>//*[contains(text(),'Assyst')]</code> - CSS Starts with: <code>driver.findElement(By.cssSelector("[class^='head']"))</code> - CSS Ends with: <code>driver.findElement(By.cssSelector("[class\$='tent']"))</code> - CSS Contains: <code>driver.findElements(By.cssSelector("[class*='enu']"))</code>												
Working on Console of Browser	- XPath: <code>\$x("")</code> - CSS: <code>\$("")</code>												
Locating Items using Compound Class	- <code>driver.findElements(By.cssSelector(".button.btn.btn-primary.btn-xs"))</code> (button is the tagname) - <code>driver.findElement(By.cssSelector("div[class='cb-col cb-col-100 cb-scrd-itms']"))</code>												
Select all mobiles on flipkart of less than 10000	<code>//div[@class="Nx9bqj_4b5DiR" and number(translate(text(),"₹","")) < 10000]</code>												
getAttribute Depricated	DOM Property: Retrieves the value of a DOM property. Properties are dynamic and can be calculated or derived from other properties or attributes. e.g. <code>element.getDomProperty("value");</code> DOM Attribute: Retrieves the value of a DOM attribute. Attributes are static and directly assigned to an element. String <code>textContent = element.getDomProperty("readonly");</code>												
Advanced Xpath	<pre><?xml version="1.0" encoding="UTF-8"?> <bookstore> <book> <title lang="en">Design Patterns for High-quality Automated Tests</title> <price>9.99</price> </book> <book> <title lang="en">Testing in the Galaxy</title> <price>5.95</price> </book> </bookstore> <author>Anton Angelov</author></pre> <table border="1"> <tbody> <tr> <td><code>//@lang</code></td><td>Selects all nodes with lang attribute</td></tr> <tr> <td><code>//book/title //book/price</code></td><td>Selects all title and price nodes under book node</td></tr> <tr> <td><code>/bookstore/book(last())</code></td><td>Last book under bookstore</td></tr> <tr> <td><code>/bookstore/book(last()-1)</code></td><td></td></tr> <tr> <td><code>/bookstore/book(position())<3</code></td><td>Selects 1st and 2nd Book</td></tr> <tr> <td><code>/bookstore/book(price>35)</code></td><td></td></tr> </tbody> </table>	<code>//@lang</code>	Selects all nodes with lang attribute	<code>//book/title //book/price</code>	Selects all title and price nodes under book node	<code>/bookstore/book(last())</code>	Last book under bookstore	<code>/bookstore/book(last()-1)</code>		<code>/bookstore/book(position())<3</code>	Selects 1st and 2nd Book	<code>/bookstore/book(price>35)</code>	
<code>//@lang</code>	Selects all nodes with lang attribute												
<code>//book/title //book/price</code>	Selects all title and price nodes under book node												
<code>/bookstore/book(last())</code>	Last book under bookstore												
<code>/bookstore/book(last()-1)</code>													
<code>/bookstore/book(position())<3</code>	Selects 1st and 2nd Book												
<code>/bookstore/book(price>35)</code>													

Waits

10 April 2026 18:24

Implicit Wait	This is Generally discouraged in favor of Explicit waits. - Used to set a default waiting time for the entire session. The WebDriver will wait for this duration before throwing a NoSuchElementException. Also applicable for <code>pageLoadTimeout</code>. Works only with <code>findElement</code> and <code>findElements</code>. <code>driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));</code> <u><code>driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));</code></u>: This timeout determines how long WebDriver will wait for a page to completely load before throwing a <code>TimeoutException</code> <u><code>driver.manage().timeouts().scriptTimeout(Duration.ofSeconds(30));</code></u>: If a JavaScript script running in the browser takes longer than 30 seconds to complete, stop the script execution and throw an exception				
Explicit Wait	Allows waiting for a specific condition to be met before proceeding further. Can be customized for each element or condition. Utilizes <code>WebDriverWait</code> and <code>ExpectedConditions</code>. Default polling interval is 500ms. <pre>// Initialize the wait (Selenium 4 Duration syntax is correct) WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10)); // 1. Lambda: Safer to use a locator if 'delayedText' isn't initialized yet // If 'delayedText' was found earlier but is just hidden, this works: wait.until(d -> delayedText.isDisplayed()); // 2. Visibility: Returns the WebElement once visible WebElement visibleElement = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("elementId"))); // 3. Clickable: Returns the WebElement once clickable (visible + enabled) WebElement clickableElement = wait.until(ExpectedConditions.elementToBeClickable(By.id("clickableElementId"))); // 4. Presence: Returns the WebElement once in the DOM (even if hidden) WebElement presentElement = wait.until(ExpectedConditions.presenceOfElementLocated(By.id("elementId"))); // 5. Invisibility: Returns a Boolean (true if it disappears) boolean isInvisible = wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("elementId"))); // 6. Text Check: Returns a Boolean (true if text matches) boolean isTextPresent = wait.until(ExpectedConditions.textToBePresentInElementLocated(By.id("elementId"), "expectedText")); // 7. Attribute Check: Returns a Boolean (true if attribute matches) boolean isAttributeValueCorrect = wait.until(ExpectedConditions.attributeToBe(By.id("elementId"), "attributeName", "expectedValue"));</pre> Warning: Do not mix implicit and explicit waits. Doing so can cause unpredictable wait times. For example, setting an implicit wait of 10 seconds and an explicit wait of 15 seconds could cause a timeout to occur after 20 seconds. When you set both, they don't replace each other. They stack. <table><tr><td><code>driver.implicitly_wait(5)</code> # implicit: 5 seconds <code>WebDriverWait(driver, 15)</code> # explicit: 15 seconds</td><td>WebDriverWait runs a polling loop, default every 500ms. Each poll calls <code>find_element</code> internally. That single <code>find_element</code> call now blocks for up to 5 seconds due to implicit wait before returning <code>NoSuchElementException</code> to the polling loop. So the polling loop is no longer running every 500ms. It runs every 5 seconds. The implicit wait sabotages the polling frequency of the explicit wait, making it coarse and unreliable regardless of how high you set the explicit timeout. You lose the precision that is the entire point of using explicit waits.</td></tr><tr><td><code>driver.implicitly_wait(10)</code> # Checking that an element is NOT present <code>wait = WebDriverWait(driver, 3)</code> <code>wait.until(EC.invisibility_of_element_located((By.ID, "error-banner")))</code></td><td>Here you want a 3 second check. But the implicit wait holds <code>find_element</code> for 10 seconds before confirming the element is absent. Your "fast" negative check now takes 10 seconds every time, and negative assertions across a test suite compound into massive slowdowns.</td></tr></table> Explicit waits are strictly superior because they let you express the exact condition you're waiting for, on a per-interaction basis, with a predictable timeout. Implicit waits are a blunt instrument that obscures what your test is actually doing. Use Explicit Wait for almost everything. It is cleaner, utilizes <code>Duration</code> objects in Selenium 4, and is strictly superior to Implicit waits for modern web applications.	<code>driver.implicitly_wait(5)</code> # implicit: 5 seconds <code>WebDriverWait(driver, 15)</code> # explicit: 15 seconds	WebDriverWait runs a polling loop, default every 500ms. Each poll calls <code>find_element</code> internally. That single <code>find_element</code> call now blocks for up to 5 seconds due to implicit wait before returning <code>NoSuchElementException</code> to the polling loop. So the polling loop is no longer running every 500ms. It runs every 5 seconds. The implicit wait sabotages the polling frequency of the explicit wait, making it coarse and unreliable regardless of how high you set the explicit timeout. You lose the precision that is the entire point of using explicit waits.	<code>driver.implicitly_wait(10)</code> # Checking that an element is NOT present <code>wait = WebDriverWait(driver, 3)</code> <code>wait.until(EC.invisibility_of_element_located((By.ID, "error-banner")))</code>	Here you want a 3 second check. But the implicit wait holds <code>find_element</code> for 10 seconds before confirming the element is absent. Your "fast" negative check now takes 10 seconds every time, and negative assertions across a test suite compound into massive slowdowns.
<code>driver.implicitly_wait(5)</code> # implicit: 5 seconds <code>WebDriverWait(driver, 15)</code> # explicit: 15 seconds	WebDriverWait runs a polling loop, default every 500ms. Each poll calls <code>find_element</code> internally. That single <code>find_element</code> call now blocks for up to 5 seconds due to implicit wait before returning <code>NoSuchElementException</code> to the polling loop. So the polling loop is no longer running every 500ms. It runs every 5 seconds. The implicit wait sabotages the polling frequency of the explicit wait, making it coarse and unreliable regardless of how high you set the explicit timeout. You lose the precision that is the entire point of using explicit waits.				
<code>driver.implicitly_wait(10)</code> # Checking that an element is NOT present <code>wait = WebDriverWait(driver, 3)</code> <code>wait.until(EC.invisibility_of_element_located((By.ID, "error-banner")))</code>	Here you want a 3 second check. But the implicit wait holds <code>find_element</code> for 10 seconds before confirming the element is absent. Your "fast" negative check now takes 10 seconds every time, and negative assertions across a test suite compound into massive slowdowns.				
Fluent Wait(exPLICITWith Options)	FluentWait in Selenium is an important class when we are dealing with AJAX elements. When a user is exploring the website on a slower network, these elements may take more time to load than what they would have taken in lab testing. To play safe, FluentWait helps us mimic such scenarios and produce the best quality web application. Explicit Wait is the standard version, while Fluent Wait is the pro/customizable version. <pre>// Selenium 4 Fluent Wait Wait<WebDriver> wait = new FluentWait<>(driver) .withTimeout(Duration.ofSeconds(30)) // Max wait time .pollingEvery(Duration.ofMillis(500)) // Frequency to check .ignoring(NoSuchElementException.class) // Exceptions to ignore .withMessage("Timeout: Element not found after 30s"); WebElement foo2 = wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("#finish h4"))); System.out.println(foo2.getText());</pre> Selenium 4 Enhancement: <code>WebDriverWait</code> class (Explicit Wait) actually inherits from <code>FluentWait</code> and provides more functionality. This means they share the same DNA, but they are used for different levels of control. <code>WebDriverWait</code> ignores <code>NotFoundException</code> (parent of <code>NoSuchElementException</code>) by default out of the box while <code>FluentWait</code> ignores NOTHING by default, we must explicitly declare every exception to ignore. Use <code>WebDriverWait</code> when you are waiting on standard DOM conditions with <code>ExpectedConditions</code>. Use <code>FluentWait</code> when you need to wait on something that is not a <code>WebDriver</code> object, or when you want explicit control over which exceptions are ignored with nothing assumed by default.				
Fluent Wait Example	<code>WebDriverWait</code> is essentially a convenience wrapper around <code>FluentWait</code> with default settings: - Polling every 500 ms. - Ignoring <code>NoSuchElementException</code>. <code>FluentWait</code> gives you full control: - Custom polling intervals. - Custom exception handling (you can ignore multiple exceptions). - Ability to define your own condition functions beyond <code>ExpectedConditions</code>.				

	Imagine you're testing a web app where: • An element appears after a random delay. • Sometimes the element is present but not interactable because of a temporary overlay. • The overlay throws a <code>ElementClickInterceptedException</code> if you try to click too early. <code>WebDriverWait</code> will fail if the overlay intercepts the click, because <code>WebDriverWait</code> only ignores <code>NoSuchElementException</code> by default. With FluentWait <pre>Wait<WebDriver> wait = new FluentWait<>(driver) .withTimeout(Duration.ofSeconds(15)) .pollingEvery(Duration.ofMillis(300)) .ignoring(NoSuchElementException.class) .ignoring(ElementClickInterceptedException.class); // extra exception handling</pre> <pre>WebElement button = wait.until(driver -> { WebElement el = driver.findElement(By.id("submit")); return (el.isDisplayed() && el.isEnabled()) ? el : null; });</pre> • Here, <code>FluentWait</code> ignores both <code>NoSuchElementException</code> and <code>ElementClickInterceptedException</code>. • It keeps polling until the overlay disappears and the button is truly clickable. • This flexibility is not possible with plain <code>WebDriverWait</code> unless you subclass or extend it.
Why <code>FluentWait</code> can do more	<code>WebDriverWait</code> is essentially a simplified version of <code>FluentWait</code> with default settings. It polls the page every five hundred milliseconds and ignores the <code>NoSuchElementException</code> by default. <code>FluentWait</code> gives you full control. It allows you to set custom polling intervals, handle multiple custom exceptions, and define your own condition functions instead of relying only on <code>ExpectedConditions</code>. Because of this, there are situations where only <code>FluentWait</code> can solve the problem. Example Situation: Imagine you are testing a web application where an element appears after a random delay. Sometimes the element is present but not interactable because of a temporary overlay. The overlay throws an <code>ElementClickInterceptedException</code> if you try to click too early. Using <code>WebDriverWait</code> will fail if the overlay intercepts the click, because <code>WebDriverWait</code> only ignores <code>NoSuchElementException</code> by default. Using <code>FluentWait</code> allows you to chain configuration methods. You can set a timeout of fifteen seconds, poll every three hundred milliseconds, and explicitly tell it to ignore both <code>NoSuchElementException</code> and <code>ElementClickInterceptedException</code>. Inside the wait loop, you can pass a custom function that checks if the element is displayed and enabled, returning null otherwise. <code>FluentWait</code> will keep polling until the overlay disappears and the button is truly clickable. This flexibility is not possible with plain <code>WebDriverWait</code> unless you subclass or extend it. Key Takeaway: <code>WebDriverWait</code> is great for common conditions and has a simpler syntax. <code>FluentWait</code> is necessary when you need custom polling, multiple exception handling, or bespoke conditions. In scenarios where unexpected exceptions other than <code>NoSuchElementException</code> must be handled gracefully, <code>FluentWait</code> is the required solution.

Interactions

21 May 2018 14:04

Select dropdown	<pre>Select s = new Select(driver.findElement(By.name("my_html_select_box"))); s.selectByIndex(1); - Methods to Select Dropdown: 1. s.selectByValue(value) 2. s.selectByVisibleText(text) 3. s.selectByIndex(index) - Methods to Deselect Dropdown: 1. s.deselectAll() 2. s.deselectByValue(value) 3. s.deselectByVisibleText(text) 4. s.deselectByIndex(index) - Additional Dropdown Methods: - s.isMultiple(): Returns true if the dropdown allows multiple selection. - s.isSelected(): Returns true if the element is selected. - s.getOptions(): Gives a list of WebElement options. -getFirstSelectedOption() -getAllSelectedOptions() Without Using Select Class // Assuming the dropdown is a <div> element with a specific class WebElement dropdown = driver.findElement(By.cssSelector("div.dropdown-menu")); dropdown.click(); // Assuming the option is a <a> element with a specific text WebElement option = driver.findElement(By.xpath("//a[text()='Option 2']")); option.click();</pre>																												
Alert	<pre>System.out.println(driver.switchTo().alert().getText()); driver.switchTo().alert().sendKeys("fesfe"); driver.switchTo().alert().accept(); //Accept = ok done yes driver.switchTo().alert().dismiss(); WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10)); // Adjust timeout as needed try { Alert alert = wait.until(ExpectedConditions.alertIsPresent()); System.out.println("Alert text: " + alert.getText()); alert.accept(); // Or alert.dismiss() } catch (org.openqa.selenium.TimeoutException e) { System.out.println("Alert not found within the timeout."); }</pre>																												
Actions	Actions a = new Actions(driver); <table><tr><td rowspan="5">Build vs Perform</td><td>Build</td><td colspan="2">Responsible for compiling your individual commands into a single, composite action. • Role: It returns an Action object (an interface). packages your commands into a single Action. • The Logic: Think of this as "preparing the script." It takes your sequence—like moveToElement, click, and keyUp—and bundles them into one executable block. • When to use it: Use it explicitly if you want to store a complex sequence to reuse later or to keep your code organized.</td></tr><tr><td>Perform</td><td colspan="2">The perform() method is what actually triggers the interaction in the browser. • Role: It executes the sequence of actions. It returns void. sends those commands to the browser. • The Secret: If you check the Selenium source code, perform() actually calls build() internally if you haven't already. This is why a simple .perform() often works alone. • The Logic: This is the "play button." • While perform() can do both, using build().perform() is considered best practice for complex chains to ensure the entire sequence is correctly constructed before execution begins.</td></tr><tr><td>Feature</td><td>build()</td><td>perform()</td></tr><tr><td>Purpose</td><td>Compiles actions into a single bundle.</td><td>Executes the bundle in the browser.</td></tr><tr><td>Return Type</td><td>Action object</td><td>void</td></tr><tr><td>Internal Logic</td><td>Organizes the queue of events.</td><td>Calls build() (if needed) then triggers WebDriver.</td></tr><tr><td>Analogy</td><td>Writing down a recipe.</td><td>Actually cooking the meal.</td></tr><tr><td>Complex Interaction</td><td colspan="3">This is best for complex, multi-step interactions where you want to define the logic before running it. Actions builder = new Actions(driver); // 1. Build the sequence Action seriesOfActions = builder .moveToElement(loginButton) .click() .keyDown(Keys.SHIFT) .sendKeys("hello") .build(); // 2. Execute the sequence seriesOfActions.perform();</td></tr></table>			Build vs Perform	Build	Responsible for compiling your individual commands into a single, composite action. • Role: It returns an Action object (an interface). packages your commands into a single Action. • The Logic: Think of this as " preparing the script. " It takes your sequence—like moveToElement, click, and keyUp—and bundles them into one executable block. • When to use it: Use it explicitly if you want to store a complex sequence to reuse later or to keep your code organized.		Perform	The perform() method is what actually triggers the interaction in the browser. • Role: It executes the sequence of actions. It returns void. sends those commands to the browser. • The Secret: If you check the Selenium source code, perform() actually calls build() internally if you haven't already. This is why a simple .perform() often works alone. • The Logic: This is the " play button. " • While perform() can do both, using build().perform() is considered best practice for complex chains to ensure the entire sequence is correctly constructed before execution begins.		Feature	build()	perform()	Purpose	Compiles actions into a single bundle.	Executes the bundle in the browser.	Return Type	Action object	void	Internal Logic	Organizes the queue of events.	Calls build() (if needed) then triggers WebDriver.	Analogy	Writing down a recipe.	Actually cooking the meal.	Complex Interaction	This is best for complex, multi-step interactions where you want to define the logic before running it. Actions builder = new Actions(driver); // 1. Build the sequence Action seriesOfActions = builder .moveToElement(loginButton) .click() .keyDown(Keys.SHIFT) .sendKeys("hello") .build(); // 2. Execute the sequence seriesOfActions.perform();		
Build vs Perform	Build	Responsible for compiling your individual commands into a single, composite action. • Role: It returns an Action object (an interface). packages your commands into a single Action. • The Logic: Think of this as " preparing the script. " It takes your sequence—like moveToElement, click, and keyUp—and bundles them into one executable block. • When to use it: Use it explicitly if you want to store a complex sequence to reuse later or to keep your code organized.																											
	Perform	The perform() method is what actually triggers the interaction in the browser. • Role: It executes the sequence of actions. It returns void. sends those commands to the browser. • The Secret: If you check the Selenium source code, perform() actually calls build() internally if you haven't already. This is why a simple .perform() often works alone. • The Logic: This is the " play button. " • While perform() can do both, using build().perform() is considered best practice for complex chains to ensure the entire sequence is correctly constructed before execution begins.																											
	Feature	build()	perform()																										
	Purpose	Compiles actions into a single bundle.	Executes the bundle in the browser.																										
	Return Type	Action object	void																										
Internal Logic	Organizes the queue of events.	Calls build() (if needed) then triggers WebDriver.																											
Analogy	Writing down a recipe.	Actually cooking the meal.																											
Complex Interaction	This is best for complex, multi-step interactions where you want to define the logic before running it. Actions builder = new Actions(driver); // 1. Build the sequence Action seriesOfActions = builder .moveToElement(loginButton) .click() .keyDown(Keys.SHIFT) .sendKeys("hello") .build(); // 2. Execute the sequence seriesOfActions.perform();																												

		Simple Interaction For simple, one-off actions, you can skip build() because perform() handles the compilation for you. // perform() calls build() behind the scenes actions.contextClick(element).perform();
Mouse Hover and Click	a.moveToElement(driver.findElement(By.id("nav-link-accountList"))).click().build().perform();	
Mouse Hover, Click with Shift Key, and Typing	a.moveToElement(driver.findElement(By.id("nav-link-accountList"))).click().keyDown(Keys.SHIFT).sendKeys("hello").build().perform();	
Keyboard Shortcuts	a.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();	
DragandDrop	Action.clickAndHold("driver.....").moveToElement("driver..").release().build().perform();	
Pause	new Actions(driver) .moveToElement(clickable) .pause(Duration.ofSeconds(1)) .clickAndHold() .pause(Duration.ofSeconds(1)) .sendKeys("abc") .perform();	
Release All Inputs	((RemoteWebDriver) driver).resetInputState();	
sendKeysToActiveElement	@Test public void sendKeysToActiveElement() { driver.get("https://www.selenium.dev/selenium/web/single_text_input.html"); new Actions(driver) .sendKeys("abc") .perform(); WebElement textField = driver.findElement(By.id("textInput")); Assertions.assertEquals("abc", textField.getAttribute("value")); }	
sendKeysToDesignatedElement	@Test public void sendKeysToDesignatedElement() { driver.get("https://www.selenium.dev/selenium/web/single_text_input.html"); driver.findElement(By.tagName("body")).click(); WebElement textField = driver.findElement(By.id("textInput")); new Actions(driver) .sendKeys(textField, "Selenium!") .perform(); Assertions.assertEquals("Selenium!", textField.getAttribute("value")); }	
scrollToElement	WebElement iframe = driver.findElement(By.tagName("iframe")); new Actions(driver) .scrollToElement(iframe) .perform(); Scroll down by 500 pixels : actions.scrollByAmount(0, 500).perform();	
Actions vs Action	Actions is the class you use to construct a series of user interactions. Action is an interface that represents a single, completed sequence of actions built by the Actions class. Actions actions = new Actions(driver); Action combinedAction = actions .moveToElement(element1) .click() .moveToElement(element2) .contextClick() .build(); combinedAction.perform();	
Window Switch	Set<String> windowHandles = driver.getWindowHandles();// This is a linkedhashset which retains window order List<String> windowHandlesList = new ArrayList<>(windowHandles); driver.switchTo().window(windowHandlesList.get(1));	
Frames	WebElement iframe = driver.findElement(By.tagName("iframe")); driver.switchTo().frame(iframe);//Webelement driver.switchTo().frame("iframe-name");//frame name or id driver.switchTo().frame(0);//Index switchTo().parentFrame(): In case of nested frames, switches the focus to the parent frame SwitchTo().defaultContent(): Switch to main page	
Win Auth	driver.get("https://admin:admin@the-internet.herokuapp.com/");	
Js Executor	JavaScriptExecutor is an interface that is used to execute JavaScript with Selenium. RemoteWebDriver implements JavaScriptExecutor. - Use getElementById() when you know the exact ID of the element. - Use querySelector() for simple CSS selectors. - Use querySelectorAll() when you need to select multiple elements. - Use getElementsByTagName() when you know the tag name of the elements. - Use getElementsByClassName() when you know the class name of the elements.	
	executeScript()	This method executes the test script in the context of the currently selected window or frame
	executeAsyncScrip ()	This method executes the asynchronous piece of JavaScript on the current window or frame. An asynchronous script will be exeacted while the rest of the page continues parsing, which enhances responsiveness and application performance.
	Click	JavaScriptExecutor js = (JavaScriptExecutor) driver; js.executeScript("document.getElementById('element id ').click();"); WebElement firstYes = driver.findElement(By.xpath("//*[@id=\"yes\"]")); js.executeScript("arguments[0].click();", firstYes);

		js.executeScript("document.querySelector('#one').click();");										
	Scroll Down	jsExecutor.executeScript("arguments[0].scrollIntoView(true);", element); // Scroll down by 1000 pixels js.executeScript("window.scrollBy(0, 1000)"); // Scroll up by 1000 pixels js.executeScript("window.scrollBy(0, -1000)"); // Scroll to the bottom of the page js.executeScript("window.scrollTo(0, document.body.scrollHeight)"); // Scroll to the top of the page js.executeScript("window.scrollTo(0, 0)");										
	Send Text	WebElement fullName = driver.findElement(By.id("fullName")); js.executeScript("document.getElementById('fullName').value='saurav'");										
	Get Text	String text = (String) js.executeScript("return arguments[0].value;", fullName); String text = (String) jsExecutor.executeScript("return arguments[0].innerText;", element);										
	Type Text	js.executeScript("document.getElementById('Email').value='SoftwareTestingMaterial.com'");										
File Upload	driver.get("https://the-internet.herokuapp.com/upload"); File uploadFile = new File("src/test/resources/ss.png"); WebElement fileInput = driver.findElement(By.cssSelector("input[type=file]")); fileInput.sendKeys(uploadFile.getAbsolutePath()); driver.findElement(By.id("file-submit")).click(); WebElement fileName = driver.findElement(By.id("uploaded-files")); Assert.assertEquals(fileName.getText(), "ss.png");											
File Download	ChromeOptions options=new ChromeOptions(); String downloadPath = Paths.get(System.getProperty("user.dir"), "downloads").toString();//creates downloads folder in the project Map<String,Object>prefs=new HashMap<>(); prefs.put("download.default_directory",downloadPath); prefs.put("download.prompt_for_download",false);//Disabledownloadprompt prefs.put("download.directory_upgrade",true);//Allowdirectorychanges prefs.put("safebrowsing.enabled",true);//Enablesafebrowsing options.setExperimentalOption("prefs",prefs); options.setCapability(CapabilityType.ACCEPT_INSECURE_CERTS,true); WebDriverdriver=new ChromeDriver(options);											
Window Size	driver.manage().window().setSize(new Dimension(1024, 768)); Dimension size = driver.manage().window().getSize(); int width = size.getWidth(); int height = size.getHeight(); Rectangle rectangle = elementById("property").getRect(); System.out.println(rectangle.getDimension()); System.out.println(rectangle.getWidth() + " " + rectangle.getHeight()); Position of Window/Element Point position = driver.manage().window().getPosition(); int x = position.getX(); int y = position.getY(); System.out.println("Window Position: X = " + x + ", Y = " + y); Point point = elementById("position").getLocation(); System.out.println("X : " + point.getX()); System.out.println("Y : " + point.getY());											
Cookies	<table><tr><td>Add Cookie</td><td>driver.manage().addCookie(new Cookie("key", "value"));</td></tr><tr><td>Get Named Cookie</td><td>Cookie cookie = driver.manage().getCookieNamed("foo");</td></tr><tr><td>Get All Cookies</td><td>Set<Cookie> cookies = driver.manage().getCookies(); for (Cookie cookie : cookies) { if (cookie.getName().equals("test1")) { Assertions.assertEquals(cookie.getValue(), "cookie1"); } }</td></tr><tr><td>Delete Cookie</td><td>driver.manage().deleteCookieNamed("test1");</td></tr><tr><td>Delete All Cookies</td><td>driver.manage().deleteAllCookies();</td></tr></table>	Add Cookie	driver.manage().addCookie(new Cookie("key", "value"));	Get Named Cookie	Cookie cookie = driver.manage().getCookieNamed("foo");	Get All Cookies	Set<Cookie> cookies = driver.manage().getCookies(); for (Cookie cookie : cookies) { if (cookie.getName().equals("test1")) { Assertions.assertEquals(cookie.getValue(), "cookie1"); } }	Delete Cookie	driver.manage().deleteCookieNamed("test1");	Delete All Cookies	driver.manage().deleteAllCookies();	
Add Cookie	driver.manage().addCookie(new Cookie("key", "value"));											
Get Named Cookie	Cookie cookie = driver.manage().getCookieNamed("foo");											
Get All Cookies	Set<Cookie> cookies = driver.manage().getCookies(); for (Cookie cookie : cookies) { if (cookie.getName().equals("test1")) { Assertions.assertEquals(cookie.getValue(), "cookie1"); } }											
Delete Cookie	driver.manage().deleteCookieNamed("test1");											
Delete All Cookies	driver.manage().deleteAllCookies();											

Exceptions

03 August 2021 11:34

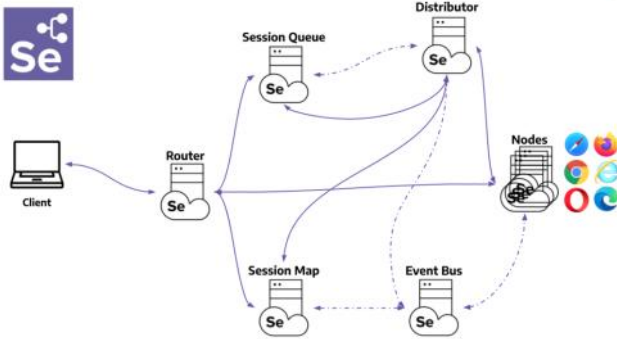
Parent Class of All Exceptions: WebDriverException->RuntimeException->Exception->Throwable.

Catch specific exceptions, log exception details, use asserts for testing purposes, use try-catch around risky code sections.

Exception Name	Description	Resolution
NoSuchElementException	Thrown when an operation attempts to access an element that is not present on the page.	• Verify Locator • Wait for Element • Check Frame
ElementClickInterceptedException	The ElementClickInterceptedException occurs when Selenium tries to click on an element, but another element intercepts the click. This often happens when there are overlapping elements, such as pop-ups or overlays, that cover the target element	• Scroll to the Element: WebElement element = driver.findElement(By.id("elementId")); ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", element); element.click(); • Use JavaScript Executor: JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("arguments[0].click();", element); • Check Resolution of the browser window • Wait for Element to be Clickable: WebElement element = wait.until(ExpectedConditions.elementToBeClickable(By.id("button")));
ElementNotVisibleException	Thrown when an operation is attempted on an element that is present in the DOM but not visible on the page.	Scroll to the Element Use Explicit Waits Use JavaScript Executor
ElementNotSelectableException	Thrown when trying to select an element that is not selectable, typically with options in a dropdown.	Use JavaScript Executor Use Explicit Waits Select Correct tag i.e option instead of div
StaleElementReferenceException	When we try to find an element using the findElement() method in WebDriver, Selenium keeps the referenceld of that element in memory if the element is found. Later when you try to interact with the element, instead of finding it again, it gets the saved referenceld and tries to locate the element with that referenceld. Although, there can be situations where you perform some action on the page, due to which the reference of a few elements gets updated or refreshed. In such cases, you would get StaleElementReferenceException. • The referenced element is not attached to the DOM anymore. • The referenced element has been deleted permanently.	Solution 1- Refresh the web page <pre>try { driver.findElement(By.cssSelector("webelement locator")).click(); } catch (StaleElementReferenceException e) { driver.navigate().refresh(); driver.findElement(By.cssSelector("webelement locator")).click(); }</pre> Solution 2- Using PageFactory In POM design pattern or while using the principles of Page Factory, we locate an element using the @FindBy annotation. It helps in updating the reference of the web element each time before any action is performed on it. This becomes very useful in eliminating the occurrence of StaleElementReferenceException.
TimeoutException	Indicates that an operation did not complete within the specified timeout period specified.	
NoSuchWindowException	Thrown when switching to a window that is not available or has been closed.	
NoSuchFrameException	Raised when the WebDriver is switching to a frame that is not available or does not exist.	
NoAlertPresentException	Occurs when an attempt is made to interact with an alert, but no alert is present on the page.	
InvalidSelectorException	Indicates that the provided selector is invalid or does not match a web element.	
NoSuchSessionException	Thrown when an attempt is made to interact with a session that is no longer available.	

Grid

03 April 2024 19:40

Architecture	 The diagram illustrates the Selenium Grid architecture. A Client sends a request to a Router. The Router interacts with a Session Queue, a Session Map, and an Event Bus. The Router also connects to a Distributor, which in turn connects to multiple Nodes. The Session Queue and Session Map are connected to the Event Bus, which also connects to the Distributor and the Nodes.	
Hub	- The hub is the central point of control in Selenium Grid. It's typically located on your local machine where you write and trigger your Selenium tests. - The hub receives test requests and then decides which node (or machine) will execute the test based on the requirements specified in the test (e.g., browser type, operating system). - The hub can manage multiple nodes, directing test execution to the appropriate machine. - A Hub is composed by the following components: Router: Receives and Forwards request to correct component Distributor: Routes test session requests to available nodes based on their capabilities. Session Map: Stores the relationship between the session id and the Node where the session is running New Session Queue: Holds all the new session requests in a FIFO order Event Bus: Communication path between the Nodes, Distributor, New Session Queue, and Session Map - Hub receives user request of port 4444 - Hub receives messages from node on 4442 - Hub sends messages to browser on 4443	
Router	- The Router is the entry point of the Grid, receiving all external requests, and forwards them to the correct component. - If the Router receives a new session request, it will be forwarded to the New Session Queue. - If the request belongs to an existing session, the Router will query the Session Map to get the Node ID where the session is running, and then the request will be forwarded directly to the Node. - The Router balances the load in the Grid by sending the requests to the component that is able to handle them better, without overloading any component that is not needed in the process.	
Distributor	The Distributor has two main responsibilities: Register and keep track of all Nodes and their capabilities: A Node registers to the Distributor by sending a Node registration event through the Event Bus. The Distributor reads it, and then tries to reach the Node via HTTP to confirm its existence. If the request is successful, the Distributor registers the Node and keeps track of all Nodes capabilities through the GridModel. Query the New Session Queue and process any pending new session requests: When a new session request is sent to the Router, it gets forwarded to the New Session Queue, where it will wait in the queue. The Distributor will poll the New Session Queue for pending new session requests, and then finds a suitable Node where the session can be created. After the session has been created, the Distributor stores in the Session Map the relation between the session id and Node where the session is being executed	
Session Map	The Session Map is a data store that keeps the relationship between the session id and the Node where the session is running. It supports the Router in the process of forwarding a request to the Node. The Router will ask the Session Map for the Node associated to a session id	
New Session Queue	The New Session Queue holds all the new session requests in a FIFO order. It has configurable parameters for setting the request timeout and request retry interval (how often the timeout will be checked). The Router adds the new session request to the New Session Queue and waits for the response. The New Session Queue regularly checks if any request in the queue has timed out, if so the request is rejected and removed immediately. The Distributor regularly checks if a slot is available. If so, the Distributor polls the New Session Queue for the first matching request. The Distributor then attempts to create a new session. Once the requested capabilities match the capabilities of any of the free Node slots, the Distributor attempts to get the available slot. If all the slots are busy, the Distributor will send the request back to the queue. If request times out while retrying or adding to the front of the queue, it will be rejected. After a session is created successfully, the Distributor sends the session information to the New Session Queue, which then gets sent back to the Router, and finally to the client.	
Nodes	A Grid can contain multiple Nodes. Each Node manages the slots for the available browsers of the machine where it is running. The Node registers itself to the Distributor through the Event Bus, and its configuration is sent as part of the registration message. By default, the Node auto-registers all browser drivers available on the path of the machine where it runs. It also creates one slot per available CPU for Chromium based browsers and Firefox. For Safari, only one slot is created. Through a specific configuration, it can run sessions in Docker containers or relay commands. A Node only executes the received commands, it does not evaluate, make judgments, or control anything other than the flow of commands and responses. The machines where the Node is running does not need to have the same operating system as the other components. For example, A Windows Node might have the capability of offering IE Mode on Edge as a browser option, whereas this would not be possible on Linux or Mac, and a Grid can have multiple Nodes configured with Windows, Mac, or Linux	
Event Bus	The Event Bus serves as a communication path between the Nodes, Distributor, New Session Queue, and Session Map. The Grid does most of its internal communication through messages, avoiding expensive HTTP calls. When starting the Grid in its fully distributed mode, the Event Bus is the first component that should be started.	
Docker compose	<pre>docker-compose-hub.yml x-common-node: &common-node shm_size: '2gb' depends_on: - selenium-hub environment: - SE_EVENT_BUS_HOST=selenium-hub - SE_EVENT_BUS_PUBLISH_PORT=4442 - SE_EVENT_BUS_SUBSCRIBE_PORT=4443 # SE_VNC_NO_PASSWORD=1 services: selenium-hub:</pre>	<pre>docker-compose-node.yml services: chrome: image: selenium/node-chrome:latest shm_size: '2gb' environment: - SE_EVENT_BUS_HOST=192.168.1.22 # IP address of the hub from the first compose file. #docker network inspect grid_selenium-network - SE_EVENT_BUS_PUBLISH_PORT=4442 - SE_EVENT_BUS_SUBSCRIBE_PORT=4443 - SE_NODE_OVERRIDE_MAX_SESSIONS=true - SE_NODE_MAX_SESSIONS=2</pre>

	<pre> image: selenium/hub:latest container_name: selenium-hub ports: - "4444:4444" - "4442:4442" - "4443:4443" networks: - selenium-network chrome: image: selenium/node-chrome:latest <<: *common-node environment: - SE_NODE_OVERRIDE_MAX_SESSIONS=true - SE_NODE_MAX_SESSIONS=2 networks: - selenium-network edge: image: selenium/node-edge:latest <<: *common-node networks: - selenium-network firefox: image: selenium/node-firefox:latest <<: *common-node networks: - selenium-network networks: selenium-network: driver: bridge </pre>		<pre> #- SE_VNC_NO_PASSWORD=1 networks: - selenium-network networks: selenium-network: driver: bridge </pre>
Starting Grid	Docker	<pre> docker-compose -f docker-compose.yml up -d docker-compose -f docker-compose-node.yml down Access hub UI at localhost:4444 </pre>	
	Standalone	<pre> java -jar selenium-server-<version>.jar standalone </pre>	
	Hub and Node	Both on same machine	<pre> Hub: java -jar selenium-server-<version>.jar hub Node1: java -jar selenium-server-<version>.jar node Node2: java -jar selenium-server-<version>.jar node --port 5555 </pre>
		On Different Machines	Registering Nodes when Hub listens on default port: <pre> java -jar selenium-server-<version>.jar node --hub <a href="http://<hub-ip>:4444">http://<hub-ip>:4444 </pre> Registering Nodes when Hub listens on Non-default port: <pre> java -jar selenium-server-<version>.jar hub --publish-events tcp://<hub-ip>:8886 -- subscribe-events tcp://<hub-ip>:8887 --port 8888 java -jar selenium-server-<version>.jar node --publish-events tcp://<hub-ip>:8886 -- subscribe-events tcp://<hub-ip>:8887 </pre>
Control number of browser instances from command line	<pre> docker-compose up --scale chrome=3 --scale firefox=2 -d </pre>		
Code	<pre> ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setCapability("browserVersion", "100"); chromeOptions.setCapability("platformName", "Windows"); // Showing a test name instead of the session id in the Grid UI chromeOptions.setCapability("se:name", "My simple test"); // Other type of metadata can be seen in the Grid UI by clicking on the // session info or via GraphQL chromeOptions.setCapability("se:sampleMetadata", "Sample metadata value"); WebDriver driver = new RemoteWebDriver(new URL("http://gridUrl:4444"), chromeOptions); driver.get("http://www.google.com"); driver.quit(); </pre>		

Thread Local

16 November 2021 16:56

ThreadLocal in Java is a way to achieve thread-safety. It enables us to create variables that are local to each thread. If two threads are executing the same code and that code has a reference to a ThreadLocal variable then the two threads can't see the local variable of each other. This same pattern is commonly used for other thread-specific framework components such as ExtentTest, loggers, test context objects, and API clients

```
// ThreadLocal to ensure each thread has its own WebDriver instance
private static ThreadLocal<WebDriver> tlDriver= new ThreadLocal<>();
public static void initDriver(String browser) {
    if (tlDriver.get() == null) {
        WebDriver driver = switch (browser.toLowerCase()) {
            case "chrome" -> new ChromeDriver();
            case "firefox" -> new FirefoxDriver();
            case "edge" -> new EdgeDriver();
            default -> throw new IllegalArgumentException("Unsupported browser: " + browser);
        };
        tlDriver.set(driver);
    }
}

public static WebDriver getDriver() {
    WebDriver driver = tlDriver.get();
    if (driver == null) {
        throw new IllegalStateException("Driver is not initialized. Call initDriver() first.");
    }
    return driver;
}

public static void quitBrowser() {
    WebDriver driver = tlDriver.get();
    if (driver != null) {
        driver.quit();
        tlDriver.remove();
    }
}
```

Internal Working	A common misconception is that ThreadLocal itself stores values for different threads. Internally, this is not how Java implements it. Every Thread object in Java contains a private data structure called ThreadLocalMap, and the ThreadLocal object acts merely as a key to access data stored within that map. When threadLocal.set(value) is called, Java retrieves the currently executing thread using Thread.currentThread() and stores the value inside that thread's private ThreadLocalMap. Similarly, when threadLocal.get() is called, Java looks up the value associated with that ThreadLocal key in the current thread's private map and returns it. As a result, Thread-1 accesses only its own data, Thread-2 accesses only its own data, and no thread can directly access another thread's stored values. Therefore, the actual data resides inside the Thread object rather than inside the ThreadLocal object itself.
Set	<pre>private static ThreadLocal<String> user = new ThreadLocal<>(); user.set("Saurav");</pre> Internally <pre>Thread current = Thread.currentThread(); current.threadLocalMap.put(user, "Saurav");</pre>
Why remove() Is Required	In automation frameworks, test execution frameworks such as TestNG and Cucumber use thread pools to reuse threads instead of creating new threads for every test execution. If a ThreadLocal value is not removed after a test completes, the associated object remains stored inside the thread's ThreadLocalMap. When the same thread is reused for another test, it may still contain stale data from the previous execution, leading to browser session leaks, memory leaks, and unpredictable test failures. Calling threadLocal.remove() explicitly removes the entry from the current thread's ThreadLocalMap, ensuring that the thread starts with a clean state when it is reused by the thread pool. In Selenium frameworks, failing to invoke driver.remove() after driver.quit() can leave orphaned WebDriver instances attached to long-lived worker threads, causing excessive memory consumption and unstable parallel execution. For this reason, remove() is considered mandatory whenever ThreadLocal is used with thread pools.
Static threadlocal	So that different test run objects share same thread local container ie There is one shared ThreadLocal object which internally provides isolated storage for every thread.

Design Patterns

30 September 2024 08:25

Introduction

Design Patterns are reusable Templates to common software design problems. They also promote code reusability, maintainability, and flexibility.

Creational Patterns: (How objects are created)

- Deals with object instantiation.
- Abstracts the instantiation process.

Singleton	• Purpose: Ensures a class has only one instance and provides a global point of access • Use Case: Logging, configuration management, thread pools. • Implementation: • Private constructor. • Static instance variable. • Static method to get the instance. <pre>public class Singleton { private static Singleton instance; private Singleton() {} public static Singleton getInstance() { if (instance == null) { instance = new Singleton(); } return instance; } }</pre>
Factory Method	• Purpose: Defines an interface for creating an object, but lets subclasses decide which class to instantiate. • Use Case: When you don't know the exact type of object needed at compile time. • Implementation: • Abstract creator class with a factory method. • Concrete creator subclasses that override the factory method. • Example(Java): Creating different types of shapes.
Abstract Factory	• Purpose: Provides an interface for creating families of related or dependent objects without specifying their concrete classes. • Use Case: Creating UI toolkits with different look and feel (e.g., Windows vs. macOS). • Implementation: • Abstract factory interface. • Concrete factory classes. • Abstract product interfaces. • Concrete product classes. • Example(Java): Creating GUI elements for different operating systems.
Builder	• Purpose: Separates the construction of a complex object from its representation so that the same construction process can create different representations. • Use Case: Creating complex objects with many optional parameters. • Implementation: • Builder interface. • Concrete builder classes. • Director class (optional). • Example(Java): Building a computer with various components.
Prototype	• Purpose: Specifies the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype. • Use Case: When creating objects is expensive and you want to copy existing objects. • Implementation: • Prototype interface with a clone() method. • Concrete prototype classes. • Example(Java): Copying complex data objects.

Structural Patterns: (How objects are composed)

- Deals with object composition and relationships.
- Simplifies complex structures.

Adapter	• Purpose: Allows interfaces of existing classes to work together. • Use Case: When you have an existing class with an incompatible interface. • Implementation: • Adapter class that implements the target interface and wraps the adaptee. • Example(Java): Adapting a legacy API to a new interface.
Bridge	• Purpose: Decouples an abstraction from its implementation so that the two can vary independently. • Use Case: When you have multiple variations of both an abstraction and an implementation. • Implementation: • Abstraction interface and implementation interface. • Concrete abstraction and concrete implementation classes. • Example(Java): Creating different UI themes and drawing APIs.
Composite	• Purpose: Composes objects into tree structures to represent part-whole hierarchies. • Use Case: When you need to treat individual objects and compositions of objects uniformly. • Implementation: • Component interface. • Leaf and composite classes. • Example(Java): File system directories and files.
Decorator	• Purpose: Attaches additional responsibilities to an object dynamically. • Use Case: When you want to add behavior to an object without modifying its class. • Implementation: • Component interface. • Concrete component class. • Decorator abstract class. • Concrete decorator classes. • Example(Java): Adding features to a text editor.
Facade	• Purpose: Provides a unified interface to a set of interfaces in a subsystem. • Use Case: When you want to simplify the use of a complex subsystem. • Implementation: • Facade class that encapsulates the subsystem. • Example(Java): Simplifying a complex library.

	<table> <tr> <td>Flyweight</td><td> • Purpose: Uses sharing to support large numbers of fine-grained objects efficiently. • Use Case: When you have many objects with shared state. • Implementation: • Flyweight interface. • Concrete flyweight classes. • Flyweight factory. • Example(Java): Rendering characters in a text editor. </td></tr> <tr> <td>Proxy</td><td> • Purpose: Provides a surrogate or placeholder for another object to control access to it. • Use Case: When you want to add a layer of control or protection around an object. • Implementation: • Subject interface. • Real subject class. • Proxy class. • Example(Java): Lazy loading, remote proxies. </td></tr> </table>	Flyweight	• Purpose: Uses sharing to support large numbers of fine-grained objects efficiently. • Use Case: When you have many objects with shared state. • Implementation: • Flyweight interface. • Concrete flyweight classes. • Flyweight factory. • Example(Java): Rendering characters in a text editor.	Proxy	• Purpose: Provides a surrogate or placeholder for another object to control access to it. • Use Case: When you want to add a layer of control or protection around an object. • Implementation: • Subject interface. • Real subject class. • Proxy class. • Example(Java): Lazy loading, remote proxies.														
Flyweight	• Purpose: Uses sharing to support large numbers of fine-grained objects efficiently. • Use Case: When you have many objects with shared state. • Implementation: • Flyweight interface. • Concrete flyweight classes. • Flyweight factory. • Example(Java): Rendering characters in a text editor.																		
Proxy	• Purpose: Provides a surrogate or placeholder for another object to control access to it. • Use Case: When you want to add a layer of control or protection around an object. • Implementation: • Subject interface. • Real subject class. • Proxy class. • Example(Java): Lazy loading, remote proxies.																		
Behavioral Patterns: (How objects interact)	• Deals with object communication and algorithms. • Defines how objects collaborate. <table> <tr> <td>Chain of Responsibility</td><td> • Purpose: Avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. • Use Case: When multiple objects can handle a request. • Implementation: • Handler interface. • Concrete handler classes. • Example(Java): Handling event propagation. </td></tr> <tr> <td>Command</td><td> • Purpose: Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations. • Use Case: Implementing undo/redo, transaction processing. • Implementation: • Command interface. • Concrete command classes. • Invoker and receiver classes. • Example(Java): Implementing menu actions. </td></tr> <tr> <td>Interpreter</td><td> • Purpose: Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language. • Use Case: Parsing and evaluating expressions. • Implementation: • Abstract expression interface. • Terminal and non-terminal expression classes. • Example(Java): Implementing a simple calculator </td></tr> <tr> <td>Iterator</td><td> • Purpose: Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. • Use Case: Traversing collections. • Implementation: • Iterator interface. • Concrete iterator class. • Aggregate interface. • Concrete aggregate class. • Example(Java): Traversing a list. </td></tr> <tr> <td>Mediator</td><td> • Purpose: Defines an object that encapsulates how a set of objects interact. • Use Case: Reducing coupling between objects. • Implementation: • Mediator interface. • Concrete mediator class. • Colleague classes. • Example(Java): Chat room, air traffic control. • Explanation: The Mediator pattern centralizes communication between objects. Instead of objects communicating directly with each other, they communicate through the mediator. This reduces coupling and simplifies the system. </td></tr> <tr> <td>Memento</td><td> • Purpose: Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later. • Use Case: Implementing undo/redo, saving game states. • Implementation: • Originator class (the object whose state is saved). • Memento class (the object that stores the state). • Caretaker class (the object that manages the mementos). • Example(Java): Saving and restoring the state of a text editor. • Explanation: The Memento pattern allows you to save and restore the state of an object without exposing its internal details. </td></tr> <tr> <td>Observer</td><td> • Purpose: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. • Use Case: Implementing event handling, model-view-controller (MVC). • Implementation: • Subject interface. • Concrete subject class. • Observer interface. • Concrete observer classes. • Example(Java): Implementing a stock ticker. • Explanation: The Observer pattern allows objects to subscribe to events and receive notifications when those events occur. </td></tr> <tr> <td>State</td><td> • Purpose: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class. • Use Case: Implementing state machines, handling different modes of operation. • Implementation: • Context class (the object whose state changes). • State interface. • Concrete state classes. • Example(Java): Implementing a TCP connection. • Explanation: The State pattern encapsulates state-specific behavior in separate classes, making it easier to add or modify states. </td></tr> <tr> <td>Strategy</td><td> • Purpose: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it. • Use Case: Implementing different sorting algorithms, payment methods. • Implementation: • Strategy interface. • Concrete strategy classes. • Context class. </td></tr> </table>	Chain of Responsibility	• Purpose: Avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. • Use Case: When multiple objects can handle a request. • Implementation: • Handler interface. • Concrete handler classes. • Example(Java): Handling event propagation.	Command	• Purpose: Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations. • Use Case: Implementing undo/redo, transaction processing. • Implementation: • Command interface. • Concrete command classes. • Invoker and receiver classes. • Example(Java): Implementing menu actions.	Interpreter	• Purpose: Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language. • Use Case: Parsing and evaluating expressions. • Implementation: • Abstract expression interface. • Terminal and non-terminal expression classes. • Example(Java): Implementing a simple calculator	Iterator	• Purpose: Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. • Use Case: Traversing collections. • Implementation: • Iterator interface. • Concrete iterator class. • Aggregate interface. • Concrete aggregate class. • Example(Java): Traversing a list.	Mediator	• Purpose: Defines an object that encapsulates how a set of objects interact. • Use Case: Reducing coupling between objects. • Implementation: • Mediator interface. • Concrete mediator class. • Colleague classes. • Example(Java): Chat room, air traffic control. • Explanation: The Mediator pattern centralizes communication between objects. Instead of objects communicating directly with each other, they communicate through the mediator. This reduces coupling and simplifies the system.	Memento	• Purpose: Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later. • Use Case: Implementing undo/redo, saving game states. • Implementation: • Originator class (the object whose state is saved). • Memento class (the object that stores the state). • Caretaker class (the object that manages the mementos). • Example(Java): Saving and restoring the state of a text editor. • Explanation: The Memento pattern allows you to save and restore the state of an object without exposing its internal details.	Observer	• Purpose: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. • Use Case: Implementing event handling, model-view-controller (MVC). • Implementation: • Subject interface. • Concrete subject class. • Observer interface. • Concrete observer classes. • Example(Java): Implementing a stock ticker. • Explanation: The Observer pattern allows objects to subscribe to events and receive notifications when those events occur.	State	• Purpose: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class. • Use Case: Implementing state machines, handling different modes of operation. • Implementation: • Context class (the object whose state changes). • State interface. • Concrete state classes. • Example(Java): Implementing a TCP connection. • Explanation: The State pattern encapsulates state-specific behavior in separate classes, making it easier to add or modify states.	Strategy	• Purpose: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it. • Use Case: Implementing different sorting algorithms, payment methods. • Implementation: • Strategy interface. • Concrete strategy classes. • Context class.
Chain of Responsibility	• Purpose: Avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. • Use Case: When multiple objects can handle a request. • Implementation: • Handler interface. • Concrete handler classes. • Example(Java): Handling event propagation.																		
Command	• Purpose: Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations. • Use Case: Implementing undo/redo, transaction processing. • Implementation: • Command interface. • Concrete command classes. • Invoker and receiver classes. • Example(Java): Implementing menu actions.																		
Interpreter	• Purpose: Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language. • Use Case: Parsing and evaluating expressions. • Implementation: • Abstract expression interface. • Terminal and non-terminal expression classes. • Example(Java): Implementing a simple calculator																		
Iterator	• Purpose: Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. • Use Case: Traversing collections. • Implementation: • Iterator interface. • Concrete iterator class. • Aggregate interface. • Concrete aggregate class. • Example(Java): Traversing a list.																		
Mediator	• Purpose: Defines an object that encapsulates how a set of objects interact. • Use Case: Reducing coupling between objects. • Implementation: • Mediator interface. • Concrete mediator class. • Colleague classes. • Example(Java): Chat room, air traffic control. • Explanation: The Mediator pattern centralizes communication between objects. Instead of objects communicating directly with each other, they communicate through the mediator. This reduces coupling and simplifies the system.																		
Memento	• Purpose: Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later. • Use Case: Implementing undo/redo, saving game states. • Implementation: • Originator class (the object whose state is saved). • Memento class (the object that stores the state). • Caretaker class (the object that manages the mementos). • Example(Java): Saving and restoring the state of a text editor. • Explanation: The Memento pattern allows you to save and restore the state of an object without exposing its internal details.																		
Observer	• Purpose: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. • Use Case: Implementing event handling, model-view-controller (MVC). • Implementation: • Subject interface. • Concrete subject class. • Observer interface. • Concrete observer classes. • Example(Java): Implementing a stock ticker. • Explanation: The Observer pattern allows objects to subscribe to events and receive notifications when those events occur.																		
State	• Purpose: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class. • Use Case: Implementing state machines, handling different modes of operation. • Implementation: • Context class (the object whose state changes). • State interface. • Concrete state classes. • Example(Java): Implementing a TCP connection. • Explanation: The State pattern encapsulates state-specific behavior in separate classes, making it easier to add or modify states.																		
Strategy	• Purpose: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it. • Use Case: Implementing different sorting algorithms, payment methods. • Implementation: • Strategy interface. • Concrete strategy classes. • Context class.																		

		• Example(Java): Implementing different compression algorithms. • Explanation: The Strategy pattern allows you to choose an algorithm at runtime, making your code more flexible.
	Template Method	• Purpose: Defines the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure. • Use Case: Implementing frameworks, defining common steps in a process. • Implementation: • Abstract class with a template method. • Concrete subclasses that override specific steps. • Example(Java): Implementing different report generation processes. • Explanation: The Template Method pattern provides a reusable structure for algorithms while allowing subclasses to customize specific steps.
	Visitor	• Purpose: Represents an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates. • Use Case: Implementing compilers, performing operations on complex data structures. • Implementation: • Visitor interface. • Concrete visitor classes. • Element interface. • Concrete element classes. • Example(Java): Implementing a compiler's syntax tree traversal. • Explanation: The Visitor pattern allows you to add new operations to an object structure without modifying the structure itself.
Singleton	• Create Private Static Instance of the Class • Create Private Constructor • Public Static getInstance for instance/object <pre> public class Browser { 4 usages new * private static Browser browser; 3 usages private Browser(){} 1 usage new * public static Browser getBrowser(){ 1 usage new * if(browser==null){ browser=new Browser(); } return browser; } public void display(){ no usages new * System.out.println("Browser Info"); } } public class Main { public static void main(String[] args) { Browser.getBrowser().display(); } } </pre> • Singleton pattern does not work well in parallel execution: Use ThreadLocal with Parallel Execution • Ensures that certain utilities (loggers) have only one instance across the entire framework.	
Selenium Singleton	<pre> public class WebDriverSingleton { private static final ThreadLocal<WebDriver> driver = new ThreadLocal<>(); private WebDriverSingleton() {} public static WebDriver getDriver(String browser) { if (driver.get() == null) { switch (browser.toLowerCase()) { case "chrome": ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.addArguments("--start-maximized"); driver.set(new ChromeDriver(chromeOptions)); break; case "firefox": FirefoxOptions firefoxOptions = new FirefoxOptions(); firefoxOptions.addArguments("--start-maximized"); driver.set(new FirefoxDriver(firefoxOptions)); break; case "edge": EdgeOptions edgeOptions = new EdgeOptions(); edgeOptions.addArguments("--start-maximized"); driver.set(new EdgeDriver(edgeOptions)); break; default: throw new IllegalArgumentException("Unsupported browser: " + browser); } } return driver.get(); } } public static void quitDriver() { if (driver.get() != null) { driver.get().quit(); driver.remove(); } } </pre>	

	<pre> } } @BeforeMethod @Parameters("browser") public void setup(String browser) { driver = WebDriverSingleton.getDriver(browser); } @Test public void testExample1() { driver.get("https://www.example.com"); Assert.assertEquals(driver.getTitle(), "Example Domain"); } </pre>
Factory Pattern(Used by webDriver)	A Factory Pattern (Follows LSP) says that just define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate. In other words, subclasses are responsible to create the instance of the class. The Factory Method pattern is a creational design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created. It encapsulates the object creation logic, decoupling it from the client code that uses the objects. The Factory Method Pattern is also known as Virtual Constructor. Advantages • The Factory Pattern allows WebDriver to abstract away the specific implementation details of different browser drivers. • This makes it easier to switch between browsers without modifying client code. • New browser drivers can be added to the framework without affecting existing tests. • Different browsers can be configured independently. PageFactory.initElements(driver, this) is used to initialize the page objects' elements. Selenium uses reflection to instantiate and initialize the elements of the page, which is another form of the Factory pattern. Components of Factory Method Design Pattern • Product: Represents the interface of the objects created by the factory method. • ConcreteProduct: Implements the Product interface and represents the concrete objects created by the factory method. • Creator: Declares the factory method, which returns an instance of the Product interface. • ConcreteCreator: Implements the factory method to create instances of ConcreteProduct. <pre> public interface BrowserDriver { WebDriver createDriver(); } public class ChromeDriverManager implements BrowserDriver { @Override public WebDriver createDriver() { return new ChromeDriver(); } } public class EdgeDriverManager implements BrowserDriver { @Override public WebDriver createDriver() { return new EdgeDriver(); } } public class DriverFactory { public static BrowserDriver getDriver(String browserType) { return switch (browserType.toLowerCase()) { case "chrome" -> new ChromeDriverManager(); case "firefox" -> new FirefoxDriverManager(); case "edge" -> new EdgeDriverManager(); default -> throw new IllegalArgumentException("Unknown browser type: " + browserType); }; } } public static void main(String[] args) { String browserType = "edge"; WebDriver driver = DriverFactory.getDriver(browserType).createDriver(); driver.manage().window().maximize(); driver.get("https://www.google.com/"); System.out.println(driver.getTitle()); driver.quit(); } </pre> The Factory Method Design Pattern is a powerful tool for creating objects in a flexible, decoupled, and extensible manner. By encapsulating object creation logic in subclasses, the Factory Method pattern promotes code reusability, maintainability, and scalability. While it may introduce some complexity, especially in large codebases, its benefits outweigh the drawbacks, making it a valuable addition to any developer's toolkit. When PageFactory.initElements is called: • It reads all the fields annotated with @FindBy (or similar annotations). • For each field, it dynamically generates a proxy object (using a mechanism like Java's Reflection or Proxy classes). • The proxy object doesn't immediately locate the WebElement; instead, it defers the lookup until the element is actually interacted with. This is known as lazy initialization. • The proxy handles the interaction with the WebDriver, locating the element on the page at runtime.
Fluent Interface	It is an object-oriented API design that allows us to chain method calls together in a readable and intuitive manner. To implement it, we need to declare methods that return objects from the same class. As a result, we'll be able to chain together multiple method calls. <pre> class Query { private String select; private String from; </pre>

```

public Query select(String select) {
    this.select = select;
    return this;
}

public Query from(String from) {
    this.from = from;
    return this;
}

public void execute() {
    System.out.println("Executing query: SELECT " + select + " FROM " + from);
}
}

```

Query query = new Query().select("id", "name").from("users").execute();

Java8's *Stream API* is using the fluent interface pattern and allows users to manipulate streams of data in a very declarative way.

```

Stream<String> processedNumbers = numbers.distinct()
    .filter(nr -> nr % 2 == 0)
    .skip(1)
    .limit(4)
    .map(nr -> "#" + nr)
    .peek(nr -> System.out.println(nr));

```

Builder Pattern

The Builder Design Pattern is a creational pattern that allows for the step-by-step creation of complex objects. It is mostly used when object can't be created in single step like in the de-serialization of a complex object.

While fluent and builder patterns aim to improve code readability, there are distinct differences. The Fluent Interface is focused on method chaining, while the Builder Pattern is concerned with constructing complex objects

Advantages

- Useful when the constructor has many fields
- Useful when the constructor has few optional and few mandatory fields
- Can be used in UI automation for fill registration forms

With Fluent Interface	Without Fluent Interface
@Data <pre> public class Product { private String name; private Double price; private String description; private String category; private Double discount; private boolean freeshipping; private int stock; // Private constructor to enforce usage of Builder private Product(ProductBuilder builder) { this.name = builder.name; this.price = builder.price; this.description = builder.description; this.category = builder.category; this.discount = builder.discount; this.freeshipping = builder.freeshipping; this.stock = builder.stock; } // Static inner ProductBuilder class public static class ProductBuilder { private String name; private Double price; //optional fields private String description; private String category; private Double discount; private boolean freeshipping; private int stock; //public constructor for mandatory fields public ProductBuilder(String name, Double price) { this.name = name; this.price = price; } //setter for optional fields public ProductBuilder setDescription(String description) { this.description = description; return this; } public ProductBuilder setCategory(String category) { this.category = category; return this; } public ProductBuilder setDiscount(Double discount) { this.discount = discount; return this; } } </pre>	@Data <pre> public class Product { private final String name; // Required private final Double price; // Required private String description; // Optional private String category; // Optional private Double discount; // Optional private boolean freeshipping; // Optional private int stock; // Optional // Private constructor to enforce usage of Builder private Product(Builder builder) { this.name = builder.name; this.price = builder.price; this.description = builder.description; this.category = builder.category; this.discount = builder.discount; this.freeshipping = builder.freeshipping; this.stock = builder.stock; } // Static inner Builder class public static class Builder { private final String name; private final Double price; //optional fields private String description; private String category; private Double discount; private boolean freeshipping; private int stock; //public constructor for mandatory fields public Builder(String name, Double price) { this.name = name; this.price = price; } //setter for optional fields public void setDescription(String description) { this.description = description; } public void setCategory(String category) { this.category = category; } public void setDiscount(Double discount) { this.discount = discount; } } </pre>

	<pre> public ProductBuilder setFreeshipping(boolean freeshipping) { this.freeshipping = freeshipping; return this; } public ProductBuilder setStock(int stock) { this.stock = stock; return this; } public Product build(){ return new Product(this); } } public class Runner { public static void main(String[] args) { Product myTv = new Product.ProductBuilder("MI TV", 12500.0).build(); System.out.println(myTv); Product appleTv = new Product.ProductBuilder("Apple TV", 125000.0) .setDescription("Mehenga TV") .setCategory("Electronics") .build(); appleTv.setDiscount(5.2); System.out.println(appleTv); } } </pre>	<pre> public void setFreeshipping(boolean freeshipping) { this.freeshipping = freeshipping; } public void setStock(int stock) { this.stock = stock; } public Product build() { return new Product(this); } } public class Runner { public static void main(String[] args) { Product.Builder builder = new Product.Builder("MI TV", 12500.0); builder.setDescription("Affordable TV"); builder.setCategory("Electronics"); builder.setDiscount(10.0); builder.setFreeshipping(true); builder.setStock(50); Product myTv = builder.build(); System.out.println(myTv); } } </pre>
--	--	--

Strategy Pattern	Strategy Design Pattern is a behavioral design pattern that allows the behavior of a class instance to be changed dynamically. This pattern is useful when there are multiple ways or strategies to perform a task, and you want to switch between these strategies at runtime. Imagine a scenario where we are developing test scripts to validate API responses. For this task, we can employ multiple strategies, such as validating for a valid response or checking for an invalid response. Designing the Strategy Design pattern has 4 steps <table> <tr> <td>Step 1: Define the Strategy Interface</td><td> <pre> @FunctionalInterface public interface ValidationStrategy { void validate(Response response); } </pre> </td></tr> <tr> <td>Step 2: Implement Concrete Strategy Classes</td><td> <pre> import java.util.Set; public class ValidResponseStrategy implements ValidationStrategy { @Override public void validate(Response response) { int expectedStatusCode = 200; if (response.getStatusCode() != expectedStatusCode) { throw new AssertionError("Expected status code " + expectedStatusCode + ", but got " + response.getStatusCode()); } } } public class InvalidResponseStrategy implements ValidationStrategy { private static final Set<Integer> INVALID_STATUS_CODES = Set.of(400, 404, 500); @Override public void validate(Response response) { if (INVALID_STATUS_CODES.contains(response.getStatusCode())) { throw new AssertionError("Did not expect status code " + response.getStatusCode() + ", which is considered invalid"); } } } </pre> </td></tr> <tr> <td>Step 3: Create the Context Class</td><td> <pre> public class ResponseValidatorContext { private ValidationStrategy strategy; public ResponseValidatorContext(ValidationStrategy strategy) { this.strategy = strategy; } public void setStrategy(ValidationStrategy strategy) { this.strategy = strategy; } public void validateResponse(Response response) { strategy.validate(response); } } </pre> </td></tr> <tr> <td>Step 4: Utilize the Strategy Pattern</td><td> <pre> public class Main { public static void main(String[] args) { Response response = new Response(200); // Using ValidResponseStrategy with a lambda expression ValidationStrategy validStrategy = r -> { int expectedStatusCode = 200; if (r.getStatusCode() != expectedStatusCode) { throw new AssertionError("Expected status code " + expectedStatusCode + ", but got " + r.getStatusCode()); } }; } } </pre> </td></tr> </table>	Step 1: Define the Strategy Interface	<pre> @FunctionalInterface public interface ValidationStrategy { void validate(Response response); } </pre>	Step 2: Implement Concrete Strategy Classes	<pre> import java.util.Set; public class ValidResponseStrategy implements ValidationStrategy { @Override public void validate(Response response) { int expectedStatusCode = 200; if (response.getStatusCode() != expectedStatusCode) { throw new AssertionError("Expected status code " + expectedStatusCode + ", but got " + response.getStatusCode()); } } } public class InvalidResponseStrategy implements ValidationStrategy { private static final Set<Integer> INVALID_STATUS_CODES = Set.of(400, 404, 500); @Override public void validate(Response response) { if (INVALID_STATUS_CODES.contains(response.getStatusCode())) { throw new AssertionError("Did not expect status code " + response.getStatusCode() + ", which is considered invalid"); } } } </pre>	Step 3: Create the Context Class	<pre> public class ResponseValidatorContext { private ValidationStrategy strategy; public ResponseValidatorContext(ValidationStrategy strategy) { this.strategy = strategy; } public void setStrategy(ValidationStrategy strategy) { this.strategy = strategy; } public void validateResponse(Response response) { strategy.validate(response); } } </pre>	Step 4: Utilize the Strategy Pattern	<pre> public class Main { public static void main(String[] args) { Response response = new Response(200); // Using ValidResponseStrategy with a lambda expression ValidationStrategy validStrategy = r -> { int expectedStatusCode = 200; if (r.getStatusCode() != expectedStatusCode) { throw new AssertionError("Expected status code " + expectedStatusCode + ", but got " + r.getStatusCode()); } }; } } </pre>
Step 1: Define the Strategy Interface	<pre> @FunctionalInterface public interface ValidationStrategy { void validate(Response response); } </pre>								
Step 2: Implement Concrete Strategy Classes	<pre> import java.util.Set; public class ValidResponseStrategy implements ValidationStrategy { @Override public void validate(Response response) { int expectedStatusCode = 200; if (response.getStatusCode() != expectedStatusCode) { throw new AssertionError("Expected status code " + expectedStatusCode + ", but got " + response.getStatusCode()); } } } public class InvalidResponseStrategy implements ValidationStrategy { private static final Set<Integer> INVALID_STATUS_CODES = Set.of(400, 404, 500); @Override public void validate(Response response) { if (INVALID_STATUS_CODES.contains(response.getStatusCode())) { throw new AssertionError("Did not expect status code " + response.getStatusCode() + ", which is considered invalid"); } } } </pre>								
Step 3: Create the Context Class	<pre> public class ResponseValidatorContext { private ValidationStrategy strategy; public ResponseValidatorContext(ValidationStrategy strategy) { this.strategy = strategy; } public void setStrategy(ValidationStrategy strategy) { this.strategy = strategy; } public void validateResponse(Response response) { strategy.validate(response); } } </pre>								
Step 4: Utilize the Strategy Pattern	<pre> public class Main { public static void main(String[] args) { Response response = new Response(200); // Using ValidResponseStrategy with a lambda expression ValidationStrategy validStrategy = r -> { int expectedStatusCode = 200; if (r.getStatusCode() != expectedStatusCode) { throw new AssertionError("Expected status code " + expectedStatusCode + ", but got " + r.getStatusCode()); } }; } } </pre>								

```

    });
}
};

// Using InvalidResponseStrategy with a lambda expression
Set<Integer> invalidStatusCodes = Set.of(400, 404, 500);
ValidationStrategy invalidStrategy = r -> {
    if (invalidStatusCodes.contains(r.getStatusCode())) {
        throw new AssertionError(
            "Did not expect status code " + r.getStatusCode() + ", which is considered invalid"
        );
    }
};

ResponseValidatorContext context = new ResponseValidatorContext(validStrategy);
context.validateResponse(response);

context.setStrategy(invalidStrategy);
context.validateResponse(response);
}
}

```

The Strategy design pattern is a powerful tool in the arsenal of software engineers aiming to write clean, flexible, and maintainable code. Particularly useful in scenarios involving varying algorithms, it separates the concerns of algorithm selection from the context in which they are used. By embracing this pattern, developers can ensure that their code is both scalable and resilient to change, embodying the principles of good software design.

Decorator Design Pattern(OCP)

Structural design pattern used to dynamically add new functionality to an object without altering its structure. It achieves this by wrapping the original object with a set of decorator objects that enhance or modify its behavior.

- Base Component: This is the interface or abstract class representing the core entity (e.g., Coffee).
- Concrete Component: This is the base implementation of the component (e.g., PlainCoffee).
- Decorator: This is an abstract class or interface implementing the base component, which adds the ability to decorate.
- Concrete Decorators: These are the actual decorators that add functionality (e.g., Milk, Sugar).

```

// Base Component
interface Coffee {
    String getDescription();
    double getCost();
}

// Concrete Component
class PlainCoffee implements Coffee {
    @Override
    public String getDescription() {
        return "Plain Coffee";
    }

    @Override
    public double getCost() {
        return 5.0; // Base price of plain coffee
    }
}

// Decorator
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;

    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }

    @Override
    public String getDescription() {
        return coffee.getDescription();
    }

    @Override
    public double getCost() {
        return coffee.getCost();
    }
}

// Concrete Decorators
class Milk extends CoffeeDecorator {
    public Milk(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", Milk";
    }

    @Override
    public double getCost() {
        return super.getCost() + 1.5; // Additional cost for milk
    }
}

class Sugar extends CoffeeDecorator {
    public Sugar(Coffee coffee) {
        super(coffee);
    }
}

```

```

@Override
public String getDescription() {
    return super.getDescription() + ", Sugar";
}

@Override
public double getCost() {
    return super.getCost() + 0.5; // Additional cost for sugar
}
}

class WhippedCream extends CoffeeDecorator {
    public WhippedCream(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", Whipped Cream";
    }

    @Override
    public double getCost() {
        return super.getCost() + 2.0; // Additional cost for whipped cream
    }
}

// Main Class
public class CoffeeShop {
    public static void main(String[] args) {
        // Start with a plain coffee
        Coffee myCoffee = new PlainCoffee();
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());

        // Add Milk
        myCoffee = new Milk(myCoffee);
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());

        // Add Sugar
        myCoffee = new Sugar(myCoffee);
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());

        // Add Whipped Cream
        myCoffee = new WhippedCream(myCoffee);
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());
    }
}

```

Selenium Decorator

```

// Base Component
interface Element {
    void click();
    void sendKeys(String text);
    String getText();
}

// Concrete Component
class BaseElement implements Element {
    private WebElement webElement;

    public BaseElement(WebElement webElement) {
        this.webElement = webElement;
    }

    @Override
    public void click() {
        webElement.click();
    }

    @Override
    public void sendKeys(String text) {
        webElement.sendKeys(text);
    }

    @Override
    public String getText() {
        return webElement.getText();
    }
}

// Decorator
abstract class ElementDecorator implements Element {
    protected Element element;

    public ElementDecorator(Element element) {
        this.element = element;
    }

    @Override
    public void click() {
        element.click();
    }

    @Override

```



```

    public void sendKeys(String text) {
        element.sendKeys(text);
    }

    @Override
    public String getText() {
        return element.getText();
    }
}

// Concrete Decorators
class LoggingElement extends ElementDecorator {
    public LoggingElement(Element element) {
        super(element);
    }

    @Override
    public void click() {
        System.out.println("Clicking element");
        super.click();
    }

    @Override
    public void sendKeys(String text) {
        System.out.println("Typing: " + text);
        super.sendKeys(text);
    }

    @Override
    public String getText() {
        String text = super.getText();
        System.out.println("Reading text: " + text);
        return text;
    }
}

class ExceptionHandlingElement extends ElementDecorator {
    public ExceptionHandlingElement(Element element) {
        super(element);
    }

    @Override
    public void click() {
        try {
            super.click();
        } catch (Exception e) {
            System.err.println("Error clicking element: " + e.getMessage());
        }
    }

    @Override
    public void sendKeys(String text) {
        try {
            super.sendKeys(text);
        } catch (Exception e) {
            System.err.println("Error typing: " + e.getMessage());
        }
    }
}

```

Usage in Framework:

```

WebElement webElement = driver.findElement(By.id("example"));

// Wrap with decorators
Element element = new LoggingElement(new ExceptionHandlingElement(new BaseElement(webElement)));

// Perform actions
element.click();
element.sendKeys("Test");
System.out.println(element.getText());

```

Use Case 2: Screenshot on Failure

```

class ScreenshotElement extends ElementDecorator {
    private WebDriver driver;

    public ScreenshotElement(Element element, WebDriver driver) {
        super(element);
        this.driver = driver;
    }

    @Override
    public void click() {
        try {
            super.click();
        } catch (Exception e) {
            takeScreenshot();
            throw e;
        }
    }
}

```

```

    }
}

@Override
public void sendKeys(String text) {
    try {
        super.sendKeys(text);
    } catch (Exception e) {
        takeScreenshot();
        throw e;
    }
}

private void takeScreenshot() {
    File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
    try {
        FileUtils.copyFile(screenshot, new File("screenshot_" + System.currentTimeMillis() + ".png"));
        System.out.println("Screenshot captured!");
    } catch (IOException e) {
        System.err.println("Failed to save screenshot: " + e.getMessage());
    }
}
}

```

Usage:

```

Element element = new ScreenshotElement(new BaseElement(webElement), driver);
element.click();

```

Apache POI

15 December 2024 00:00

Read Data	<pre>public static Map<String, String> getRowData(String filePath, int rowNum) throws IOException { Map<String, String> rowData = new HashMap<>(); try (FileInputStream fileInputStream = new FileInputStream(filePath); Workbook workbook = new XSSFWorkbook(fileInputStream)) { Sheet sheet = workbook.getSheetAt(0); Row headerRow = sheet.getRow(0); if (headerRow == null) { throw new IllegalArgumentException("Header row is missing in the sheet."); } Row dataRow = sheet.getRow(rowNum); if (dataRow == null) { throw new IllegalArgumentException("Row number " + rowNum + " is missing in the sheet."); } for (Cell headerCell : headerRow) { int columnIndex = headerCell.getColumnIndex(); String header = headerCell.getStringCellValue(); Cell dataCell = dataRow.getCell(columnIndex); String value = getCellValueAsString(dataCell); rowData.put(header, value); } } return rowData; } private static String getCellValueAsString(Cell cell) { return switch (cell.getCellType()) { case STRING -> cell.getStringCellValue(); case NUMERIC -> String.valueOf(cell.getNumericCellValue()); case BOOLEAN -> String.valueOf(cell.getBooleanCellValue()); default -> ""; }; }</pre>
Update Data	<pre>public static void updateCellData(String filePath, int rowNum, String columnHeader, String newValue) throws IOException { try (FileInputStream fileInputStream = new FileInputStream(filePath); Workbook workbook = new XSSFWorkbook(fileInputStream)) { Sheet sheet = workbook.getSheetAt(0); Row headerRow = sheet.getRow(0); if (headerRow == null) { throw new IllegalArgumentException("Header row is missing in the sheet."); } int columnIndex = -1; for (Cell headerCell : headerRow) { if (headerCell.getStringCellValue().equalsIgnoreCase(columnHeader)) { columnIndex = headerCell.getColumnIndex(); break; } } if (columnIndex == -1) { throw new IllegalArgumentException("Column header \"" + columnHeader + "\" not found in the sheet."); } Row dataRow = sheet.getRow(rowNum); if (dataRow == null) { dataRow = sheet.createRow(rowNum); } Cell cellToUpdate = dataRow.getCell(columnIndex); if (cellToUpdate == null) { cellToUpdate = dataRow.createCell(columnIndex); } cellToUpdate.setCellValue(newValue); try (FileOutputStream fileOutputStream = new FileOutputStream(filePath)) { workbook.write(fileOutputStream); } } }</pre>

Cucumber

02 March 2024 07:45

Syntax	@FeatureTag Feature: Feature Description @SmokeTest @RegressionTest Scenario: Successful Login Given: Preconditions are mentioned in the Given keyword. When: The purpose of the When Steps is to describe the user action. Then: The purpose of Then Steps is to observe the expected output. The observations should be related to the business value/benefit of your Feature description. And: Used to add additional conditions or actions to Given, When, or Then steps. But: Used to add conditions or actions that are exceptions to the previous steps.	
Testng Runner	AbstractTestNGCucumberTests needs to be inherited by the runner	
Tagging	A feature or scenario can have as many tags as you like. Separate them with spaces Tags can be placed above the following Gherkin elements: • Feature • Scenario • Scenario Outline • Examples Scenario Outline: Steps will run conditionally if tagged Given user is logged in When user clicks <link> Then user will be logged out @mobile Examples: link logout link on mobile @desktop Examples: link logout link on desktop It is not possible to place tags above Background or steps (Given, When, Then, And and But). Tags are inherited by child elements. Tags that are placed above a Feature will be inherited by Scenario, Scenario Outline, or Examples. Tags that are placed above a Scenario Outline will be inherited by Examples.	
And Tag in Test Runner	cucumber --tags "@smoke and @regression"	
Or Tag in Test Runner	cucumber --tags "@smoke or @regression"	
NotTag in Test Runner	cucumber --tags "not @wip"	
	(@smoke or @ui) and (not @slow)	
Hooks	Scenario hooks	Scenario hooks run for every scenario. • Before hooks run before the first step of each scenario • After hooks run after the last step of each scenario, even when the step result is failed, undefined, pending, or skipped
	Step hooks	Step hooks are invoked before and after a step. The hooks have 'invoke around' semantics, meaning that if a BeforeStep hook is executed the AfterStep hooks will also be executed regardless of the result of the step. If a step did not pass, the following step and its hooks will be skipped • BeforeStep • AfterStep
	Conditional hooks	Hooks can be conditionally selected for execution based on the tags of the scenario. To run a particular hook only for certain scenarios, you can associate a Before or After hook with a tag expression. @After("@browser and not @headless") public void doSomethingAfter(Scenario scenario){ }
	Global hooks	Global hooks will run once before any scenario is run or after all scenario have been run. BeforeAll run before any scenario is run. AfterAll run after all scenarios have been executed.
Background	It runs before each and every scenario were for a feature in which it is defined	
Hooks Vs Background	Hook runs before Background Before Hooks->Background->Scenario->After Hook Background cannot be tagged, hooks can be: @Before("@Smoke") Orders Hooks: @Before(order=0),@Before(order=1)	
{string} Regex in feature File	{string} matches quoted strings: "text" Given the user is on the "login" page @Given("the user is on the {string} page") public void theUserIsOnThePage(String page) { System.out.println("User is on the " + page + " page"); }	
(.*) Regex in feature File	(.*) matches any character sequence, including spaces, without needing quotes Given the user is on the "login" page @Given("^the user is on the (.*) page\$") public void theUserIsOnThePage(String page) {	

	<pre> System.out.println("User is on the " + page + " page"); } }</pre>															
{int}	Given the user has "42" points															
{float}	Given the price is "3.14"															
Scenario Outline	When The "AddComment" request is sent with the "Post" HTTP method on IssueID as "<ISSUEID>" Examples: <table><tr><th>RowNumber</th><th>ISSUEID</th><th>DISPLAY_NAME</th><th>UPDATE_AUTHOR_EMAIL_ADDRESS</th><th>COMMENT</th></tr><tr><td>1</td><td>ISSUEID</td><td>DISPLAY_NAME</td><td>UPDATE_AUTHOR_EMAIL_ADDRESS</td><td>COMMENT</td></tr><tr><td>2</td><td>ISSUEID</td><td>DISPLAY_NAME</td><td>UPDATE_AUTHOR_EMAIL_ADDRESS</td><td>COMMENT</td></tr></table> @When("The {string} request is sent with the {string} HTTP method on IssueID as {string}") public void request_is_sent_with_http_method_on_issueid(String resource, String httpMethod, String issueID) { issueID = excelSheetReader.readCell(issueID); if (httpMethod.equalsIgnoreCase("POST")) { addCommentResponse = SpecBuilders.SendPostAndReturnResponse(updatedJsonPayload, issueID, resource); } }	RowNumber	ISSUEID	DISPLAY_NAME	UPDATE_AUTHOR_EMAIL_ADDRESS	COMMENT	1	ISSUEID	DISPLAY_NAME	UPDATE_AUTHOR_EMAIL_ADDRESS	COMMENT	2	ISSUEID	DISPLAY_NAME	UPDATE_AUTHOR_EMAIL_ADDRESS	COMMENT
RowNumber	ISSUEID	DISPLAY_NAME	UPDATE_AUTHOR_EMAIL_ADDRESS	COMMENT												
1	ISSUEID	DISPLAY_NAME	UPDATE_AUTHOR_EMAIL_ADDRESS	COMMENT												
2	ISSUEID	DISPLAY_NAME	UPDATE_AUTHOR_EMAIL_ADDRESS	COMMENT												
Dependency Injection	DI allows you to "inject" the things a class needs (its dependencies) from the outside, instead of letting the class create or manage them itself. For Example:Car class might depend on a Engine class to run. Without DI, the Car class would directly create or manage the Engine instance within its code, which makes the two classes tightly coupled. This approach can create problems, particularly when you need to test, extend, or modify the classes in the future.															
PicoContainer	Dependency Injection (DI) in Cucumber using PicoContainer allows you to share state and objects between step definitions, hooks, and other classes. • No Manual Management: Dependency management is handled by PicoContainer, so you don't need to manually manage object lifecycles or dependencies between steps. • Consistency Across Steps: Objects maintain their state across different steps within the same scenario, making tests easier to write and maintain. • Simple Integration: Integrating PicoContainer with Cucumber requires minimal setup and aligns with Cucumber's aim to simplify writing BDD style tests. Using PicoContainer to share state between steps in a scenario is easy and non-intrusive. All you need is a constructor that requires an object that PicoContainer can create and inject. PicoContainer is invisible. Add a dependency to cucumber-picocontainer and make sure that the constructors for the step classes require an instance of the same class. Simply hand it some classes and it will instantiate each one, correctly wired together via their constructors. That's it. Cucumber scans your classes with step definitions in them, passes them to PicoContainer, then asks it to create new instances for every scenario. • Add PicoContainer to the Project • Create a Test Context class which will hold all the objects state • Divide the Steps class into multiple steps classes with logical separation • Write Constructor to share Test Context <table><tr><td>Add Dependency</td><td><pre><dependencies> <dependency> <groupId>io.cucumber</groupId> <artifactId>cucumber-picocontainer</artifactId> <version>7.0.0</version> <scope>test</scope> </dependency> </dependencies></pre></td></tr><tr><td>Create Shared Class</td><td> Define a class that will hold the shared state or objects. <pre>public class SharedState { private String importantData; public void setImportantData(String data) { this.importantData = data; } public String getImportantData() { return importantData; } }</pre></td></tr><tr><td>Feature File</td><td> Feature: Shared State Example Scenario: Sharing state between steps Given I set the important data to "expectedData" in step definition 1 Then I should have access to the important data in step definition 2 </td></tr><tr><td>Step Definition with Dependency Injection</td><td><pre>import io.cucumber.java.en.Given; public class StepDefinitions1 { private final SharedState sharedState; public StepDefinitions1(SharedState sharedState) { this.sharedState = sharedState; } @Given("I set the important data to {string} in step definition 1") public void setImportantData(String data) { sharedState.setImportantData(data); } } ----- import io.cucumber.java.en.Then; import static org.junit.Assert.assertEquals; public class StepDefinitions2 { private final SharedState sharedState;</pre></td></tr></table>	Add Dependency	<pre><dependencies> <dependency> <groupId>io.cucumber</groupId> <artifactId>cucumber-picocontainer</artifactId> <version>7.0.0</version> <scope>test</scope> </dependency> </dependencies></pre>	Create Shared Class	Define a class that will hold the shared state or objects. <pre>public class SharedState { private String importantData; public void setImportantData(String data) { this.importantData = data; } public String getImportantData() { return importantData; } }</pre>	Feature File	Feature: Shared State Example Scenario: Sharing state between steps Given I set the important data to "expectedData" in step definition 1 Then I should have access to the important data in step definition 2	Step Definition with Dependency Injection	<pre>import io.cucumber.java.en.Given; public class StepDefinitions1 { private final SharedState sharedState; public StepDefinitions1(SharedState sharedState) { this.sharedState = sharedState; } @Given("I set the important data to {string} in step definition 1") public void setImportantData(String data) { sharedState.setImportantData(data); } } ----- import io.cucumber.java.en.Then; import static org.junit.Assert.assertEquals; public class StepDefinitions2 { private final SharedState sharedState;</pre>							
Add Dependency	<pre><dependencies> <dependency> <groupId>io.cucumber</groupId> <artifactId>cucumber-picocontainer</artifactId> <version>7.0.0</version> <scope>test</scope> </dependency> </dependencies></pre>															
Create Shared Class	Define a class that will hold the shared state or objects. <pre>public class SharedState { private String importantData; public void setImportantData(String data) { this.importantData = data; } public String getImportantData() { return importantData; } }</pre>															
Feature File	Feature: Shared State Example Scenario: Sharing state between steps Given I set the important data to "expectedData" in step definition 1 Then I should have access to the important data in step definition 2															
Step Definition with Dependency Injection	<pre>import io.cucumber.java.en.Given; public class StepDefinitions1 { private final SharedState sharedState; public StepDefinitions1(SharedState sharedState) { this.sharedState = sharedState; } @Given("I set the important data to {string} in step definition 1") public void setImportantData(String data) { sharedState.setImportantData(data); } } ----- import io.cucumber.java.en.Then; import static org.junit.Assert.assertEquals; public class StepDefinitions2 { private final SharedState sharedState;</pre>															

		<pre>public StepDefinitions2(SharedState sharedState) { this.sharedState = sharedState; } @Then("I should have access to the important data in step definition 2") public void verifyImportantData() { String data = sharedState.getImportantData(); System.out.println("The important data is: " + data); assertEquals("Expected data not found", "expectedData", data); } }</pre>												
	Output	The important data is: expectedData												
DI-Selenium Example	Create a Test Context class	<pre>public class TestContext { private WebDriverManager webDriverManager; private PageObjectManager pageObjectManager; public TestContext(){ webDriverManager = new WebDriverManager(); pageObjectManager = new PageObjectManager(webDriverManager.getDriver()); } public WebDriverManager getWebDriverManager() { return webDriverManager; } public PageObjectManager getPageObjectManager() { return pageObjectManager; } }</pre>												
	Divide the Steps file	<table><tr><th>OLD Step File</th><th>Constructor for Step Definition classes to share TestContext</th></tr><tr><td><pre>public class HomePageSteps { WebDriver driver; HomePage homePage; PageObjectManager pageObjectManager; WebDriverManager webDriverManager; @Given("^user is on Home Page\$") public void user_is_on_Home_Page(){ webDriverManager = new WebDriverManager(); driver = webDriverManager.getDriver(); pageObjectManager = new PageObjectManager(driver); homePage = pageObjectManager.getHomePage(); homePage.navigateTo_HomePage(); } @When("^he search for \"([^\"]*)\"\$") public void he_search_for(String product) { homePage.perform_Search(product); } }</pre></td><td> We need WebDriverManager & PageObjectManager in every step file, otherwise, you need to create objects for both classes using new operator again and again, which is Kill Bill. Now with just adding Constructor to HomePageSteps file and pass TestContext as a Parameter to constructor would take all the pain. Within the TestContext object we have everything available which is required for the test. So now let's see how the new HomePageSteps class would look. <pre>public class HomePageSteps { TestContext testContext; HomePage homePage; public HomePageSteps(TestContext context) { testContext = context; homePage = testContext.getPageObjectManager().getHomePage(); } @Given("^user is on Home Page\$") public void user_is_on_Home_Page(){ homePage.navigateTo_HomePage(); } @When("^he search for \"([^\"]*)\"\$") public void he_search_for(String product) { homePage.perform_Search(product); } }</pre></td></tr></table>	OLD Step File	Constructor for Step Definition classes to share TestContext	<pre>public class HomePageSteps { WebDriver driver; HomePage homePage; PageObjectManager pageObjectManager; WebDriverManager webDriverManager; @Given("^user is on Home Page\$") public void user_is_on_Home_Page(){ webDriverManager = new WebDriverManager(); driver = webDriverManager.getDriver(); pageObjectManager = new PageObjectManager(driver); homePage = pageObjectManager.getHomePage(); homePage.navigateTo_HomePage(); } @When("^he search for \"([^\"]*)\"\$") public void he_search_for(String product) { homePage.perform_Search(product); } }</pre>	We need WebDriverManager & PageObjectManager in every step file, otherwise, you need to create objects for both classes using new operator again and again, which is Kill Bill. Now with just adding Constructor to HomePageSteps file and pass TestContext as a Parameter to constructor would take all the pain. Within the TestContext object we have everything available which is required for the test. So now let's see how the new HomePageSteps class would look. <pre>public class HomePageSteps { TestContext testContext; HomePage homePage; public HomePageSteps(TestContext context) { testContext = context; homePage = testContext.getPageObjectManager().getHomePage(); } @Given("^user is on Home Page\$") public void user_is_on_Home_Page(){ homePage.navigateTo_HomePage(); } @When("^he search for \"([^\"]*)\"\$") public void he_search_for(String product) { homePage.perform_Search(product); } }</pre>								
OLD Step File	Constructor for Step Definition classes to share TestContext													
<pre>public class HomePageSteps { WebDriver driver; HomePage homePage; PageObjectManager pageObjectManager; WebDriverManager webDriverManager; @Given("^user is on Home Page\$") public void user_is_on_Home_Page(){ webDriverManager = new WebDriverManager(); driver = webDriverManager.getDriver(); pageObjectManager = new PageObjectManager(driver); homePage = pageObjectManager.getHomePage(); homePage.navigateTo_HomePage(); } @When("^he search for \"([^\"]*)\"\$") public void he_search_for(String product) { homePage.perform_Search(product); } }</pre>	We need WebDriverManager & PageObjectManager in every step file, otherwise, you need to create objects for both classes using new operator again and again, which is Kill Bill. Now with just adding Constructor to HomePageSteps file and pass TestContext as a Parameter to constructor would take all the pain. Within the TestContext object we have everything available which is required for the test. So now let's see how the new HomePageSteps class would look. <pre>public class HomePageSteps { TestContext testContext; HomePage homePage; public HomePageSteps(TestContext context) { testContext = context; homePage = testContext.getPageObjectManager().getHomePage(); } @Given("^user is on Home Page\$") public void user_is_on_Home_Page(){ homePage.navigateTo_HomePage(); } @When("^he search for \"([^\"]*)\"\$") public void he_search_for(String product) { homePage.perform_Search(product); } }</pre>													
DataTable	Scenario: Correct non-zero number of books found by author by map Given I have the following books in the store by map <table><tr><td> title</td><td> author</td><td> </td></tr><tr><td> The Devil in the White City</td><td> Erik Larson</td><td> </td></tr><tr><td> The Lion, the Witch and the Wardrobe</td><td> C.S. Lewis</td><td> </td></tr><tr><td> In the Garden of Beasts</td><td> Erik Larson</td><td> </td></tr></table> When I search for books by author Erik Larson Then I find 2 books Cucumber repeats this process for each subsequent row <pre>[{"title": "The Devil in the White City", "author": "Erik Larson"}, {"title": "The Lion, the Witch and the Wardrobe", "author": "C.S. Lewis"}, {"title": "In the Garden of Beasts", "author": "Erik Larson"}]</pre> @Given("^I have the following books in the store by map\$") <pre>public void haveBooksInTheStoreByMap(DataTable table) { List<Map<String, String>> rows = table.asMaps(String.class, String.class); for (Map<String, String> columns : rows) { store.addBook(new Book(columns.get("title"), columns.get("author"))); } }</pre>		title	author		The Devil in the White City	Erik Larson		The Lion, the Witch and the Wardrobe	C.S. Lewis		In the Garden of Beasts	Erik Larson	
title	author													
The Devil in the White City	Erik Larson													
The Lion, the Witch and the Wardrobe	C.S. Lewis													
In the Garden of Beasts	Erik Larson													
Running Tests in Parallel	<pre>public class TestNGTestRunner extends AbstractTestNGCucumberTests { @Override</pre>													

	<pre> @DataProvider(parallel = true) public Object[][] scenarios() { return super.scenarios(); } } </pre> Add Surefire plugin. The default thread count of the dataprovider in parallel mode is 10. To change this the dataproviderthreadcount property needs to be added to the configuration section of the Surefire plugin in the POM. <pre> <build> <plugins> <plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-plugin</artifactId> <version>3.11.0</version> <configuration> <source>21</source> <target>21</target> <encoding>UTF-8</encoding> </configuration> </plugin> <plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-surefire-plugin</artifactId> <version>3.2.2</version> <configuration> <suiteXmlFiles> <suiteXmlFile>testng.xml</suiteXmlFile> </suiteXmlFiles> <properties> <property> <name>dataproviderthreadcount</name> <value>2</value> </property> </properties> </configuration> </plugin> </plugins> </build> </pre> This will run tests within same feature file in parallel. To run tests across feature file in parallel use testng.xml <pre> <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd> <suite name="Suite" parallel="classes" thread-count="2"> <test name="Test"> <classes> <class name="TestRunner.TestRunner"/> <class name="TestRunner.TestRunner2"/> </classes> </test> <!-- Test --> </suite> <!-- Suite --> </pre> <pre> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd> <suite name="Cucumber Suite" parallel="methods" thread-count="4"> <test name="Cucumber Scenario Parallel Tests"> <classes> <class name="parallel.RunCucumberTest"/> </classes> </test> </suite> </pre>
Run failed test cases in cucumber	<pre> @RunWith(Cucumber.class) @CucumberOptions(features = "src/test/resources/features", glue = "stepDefinitions", //only package name required plugin = {"pretty", "html:target/cucumber-reports", "rerun:target/rerun.txt"}) public class RunCucumberTest { } New Runner @RunWith(Cucumber.class) @CucumberOptions(features = "@target/rerun.txt", glue = "stepDefinitions") public class RerunFailedTests { } </pre>
Cucumber Options	Cucumber Options are used to configure properties for Cucumber tests

Options Type	Purpose	Default Value
dryRun	true: Checks if all the Steps have the Step Definition	false
features	set: The paths of the feature files	{ }
glue	set: The paths of the step definition files	{ }
tags	instruct: What tags in the features files should be executed	{ }
monochrome	true: Display the console Output in much readable way	false
format	set: What all report formatters to use	false
strict	true: Will fail execution if there are undefined or pending steps	false

Cucumber Code Flow

Level 0	What happens when I click Run on TestNGTestRunner? public class TestNGTestRunner extends AbstractTestNGCucumberTests TestNG discovers this class because of its XML or IDE runner and then executes methods inherited from AbstractTestNGCucumberTests which internally creates a TestNGCucumberRunner. This object is responsible for driving Cucumber.																
Level 1: Test Runner Starts Execution	Execution begins when the runner class is launched i.e extends AbstractTestNGCucumberTests The responsibility of the runner is: - Discover feature files - Discover glue code - Initialize the Cucumber engine - Register plugins - Start execution At this point no browser exists and no step definitions are executed.																
Level 2: Parsing feature files	The Gherkin parser converts feature file into an AST (Abstract Syntax Tree), something like: <table border="1"> <tr> <th>Feature File</th><th>AST</th></tr> <tr> <td>Feature: Login</td><td>Feature</td></tr> <tr> <td>Scenario: Successful login</td><td> </td></tr> <tr> <td>Given user is on login page</td><td>Scenario</td></tr> <tr> <td>When user enters credentials</td><td> </td></tr> <tr> <td>Then dashboard should be displayed</td><td>Given</td></tr> <tr> <td></td><td>When</td></tr> <tr> <td></td><td>Then</td></tr> </table> Nothing is executed yet.	Feature File	AST	Feature: Login	Feature	Scenario: Successful login		Given user is on login page	Scenario	When user enters credentials		Then dashboard should be displayed	Given		When		Then
Feature File	AST																
Feature: Login	Feature																
Scenario: Successful login																	
Given user is on login page	Scenario																
When user enters credentials																	
Then dashboard should be displayed	Given																
	When																
	Then																
Level 3: Step Definition Discovery	Cucumber scans glue packages. It discovers methods such as: @Given("user is on login page") Internally it builds a registry: Pattern → Method Reference Example: "user is on login page" → LoginSteps.userIsOnLoginPage() This mapping is built before execution starts. Note: Cucumber evaluates the text rather than enforcing the keyword, it treats the annotations (@Given, @When, @Then, @And) as interchangeable aliases during the matching phase. In newer versions of Cucumber, you can even use the generic @Step("user is on login page") annotation to match <i>any</i> Gherkin keyword!																
Level 4: Scenario Starts	Cucumber creates an internal Scenario object. It maintains: - scenario name - tags - status - execution state Plugins are notified: Scenario Started																
Level 5	Cucumber creates Step Definition objects via reflection. Conceptually: <pre>new LoginSteps() new HomeSteps() new CartSteps()</pre> This object creation happens automatically. Developers don't instantiate these classes themselves.																
Level 6: Hook Execution	Before any step runs, @Before methods are executed. Typical uses: - browser initialization - test data setup - environment preparation - logging Execution order: Scenario Start → @Before hooks → Step execution → @After hooks → Scenario End																
Level 7: Step Matching	For: Given user is on login page Cucumber searches the registry. It finds @Given("user is on login page") and invokes the corresponding method. Conceptually: Text in feature file → Pattern matching → Java method																
Level 8: Reflection Invocation	Cucumber reads step text from feature files at runtime, so compile-time method invocation is impossible. During startup, it builds a mapping between Gherkin step patterns and Java methods. Example: Feature: Given user is on login page Step Definition:																


```
@Given("user is on login page")
public void userIsOnLoginPage()
```

During execution:

1. Find matching method for the step text.
2. Create the Step Definition object.
3. Invoke the method dynamically using Java Reflection.

Conceptually:

```
Feature Step
→ Step Registry
→ Method Object
→ Reflection
→ method.invoke(object)
```

Instead of:

```
object.userIsOnLoginPage();
```

Cucumber internally performs:

```
method.invoke(object);
```

Thus, Cucumber uses Java Reflection to create Step Definition objects and invoke step methods dynamically.

By default, Cucumber's object factory creates Step Definition classes using no-argument constructors.

Example:

```
public class LoginSteps {
}
```

works.

But:

```
public class LoginSteps {
    public LoginSteps(TestContextSetup testContextSetup) {
    }
}
```

fails because Cucumber does not know how to supply the TestContextSetup dependency.

When using a DI framework like PicoContainer, object creation is delegated to the container. PicoContainer resolves dependencies and supports constructor injection, allowing parameterized constructors.

Without PicoContainer:

```
Feature
→ Cucumber
→ new LoginSteps()
→ Reflection invokes methods
```

With PicoContainer:

```
Feature
→ Cucumber
→ PicoContainer
→ new LoginSteps(driver)
→ Reflection invokes methods
```

Reflection is used in both cases. PicoContainer changes who creates the object, not how methods are invoked.

Cucumber maps Gherkin step text to Java methods and uses Java Reflection to create Step Definition objects and dynamically invoke methods via method.invoke(). Since feature files are external and step text is available only at runtime, normal compile-time method invocation is not possible. By default, Cucumber uses no-arg constructors; with PicoContainer, object creation is delegated to the DI container, enabling constructor injection.

Level 9: Step Execution

Each step executes sequentially:

Given → When → Then

If one step fails then Remaining steps are skipped and Scenario status becomes: FAILED

Level 10: Plugin Event System

During execution, Cucumber publishes events:

TestRunStarted

ScenarioStarted

StepStarted

StepFinished

ScenarioFinished

TestRunFinished

Plugins subscribe to these events.

	Examples: • HTML reports • JSON reports • Extent reports Plugins do not execute tests. They only observe events.
Level 11: @After hook	After scenario completion: @After methods execute. Typical responsibilities: • closing browser • taking screenshots • cleanup • logging
Level 12: Scenario Result Published	Status becomes: PASSED FAILED SKIPPED Plugins consume these events and generate reports.
Workflow	JVM→Test Runner→Cucumber Engine→Feature Parser→Step Registry→Scenario Object→@Before Hooks→ Create Step Definition Objects→Match Steps→Invoke Methods using Reflection→Execute Given→Execute When →Execute Then→@After Hooks→Publish Events→Generate Reports

TestNG

17 February 2024 13:22

Test Annotation	1. @Test: Marks a method as a test method 2. @BeforeSuite: Marks a method to run before any test suite 3. @AfterSuite: Marks a method to run after all tests in a suite have run 4. @BeforeTest: Marks a method to run before any test method belonging to the classes inside the <code><test></code> tag in XML 5. @AfterTest: Marks a method to run after all test methods belonging to the classes inside the <code><test></code> tag in XML 6. @BeforeClass: Marks a method to run before the first test method in the current class is invoked 7. @AfterClass: Marks a method to run after all the test methods in the current class have been run 8. @BeforeMethod: Marks a method to run before each test method 9. @AfterMethod: Marks a method to run after each test method <pre><suite> <test> <classes> <class> </class> </classes> </test> </suite></pre>
Inheritance and Annotation	In TestNG, the execution order follows a strict top-down (Inheritance) approach for setup methods and a bottom-up approach for teardown methods. 1. @Before Methods (Setup)When we run a test in a child class, TestNG looks up the inheritance chain. It executes the @Before methods starting from the Base Class and moves down to the Child Class. Logic: Ensures the foundation (environment, database connections) is ready before the specific test requirements are configured. Order: Base @Before --> Child @Before --> Test2. 2. @After Methods (Teardown)Teardown methods work in reverse. TestNG executes the @After method in the Child Class first, then moves up to the Base Class. Logic: Cleans up the specific local data before closing the general global connections. Order: Test --> Child @After --> Base @After If a child class overrides a @Before method (uses the same method name as the base class), TestNG will only run the child's version. To run both, ensure they have unique method names or use <code>super.methodName()</code> —though TestNG usually handles names across the hierarchy automatically.
@DataProvider	Marks a method as a data provider for test methods. It provides data to a test method. <pre>@DataProvider(name = "simpleData") public Object[][] createData() { return new Object[][] { { "John", 25 }, { "Jane", 30 }, { "Alex", 22 } }; }</pre> <pre>@Test(dataProvider = "simpleData") public void testWithDataProvider(String name, int age) { System.out.println("Name: " + name + ", Age: " + age); }</pre> DP in another file: <code>@Test(dataProvider = "flights",dataProviderClass= DataProviders.class)</code>
@Parameters	Marks a method parameter as a parameter for the test method. It allows passing parameters to test methods from the TestNG XML file. <pre><suite name="Suite"> <test name="Test"> <parameter name="username" value="user123" /> <parameter name="password" value="secret123" /> <classes> <class name="com.example.MyTestClass" /> </classes> </test></suite></pre> <pre>@Parameters({"username", "password"}) @Test public void testLogin(String username, String password) { System.out.println("Username: " + username); System.out.println("Password: " + password); // Your test logic here using the provided parameters }</pre>
Disable a test	<code>@Test(enabled=false)</code>
Timeout	<code>@Test(timeOut=10000)</code> Or <pre><?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Suite"> <test thread-count="5" name="Test" time-out="5000"> <classes> <class name="testRunner.TestNGRunner"/> </classes> </test><!-- Test --> </suite><!-- Suite --></pre>
Exclude Tests	<pre><test name="Test"> <classes></pre>

	<pre> <class name="com.example.MyTestClass"> <methods> <exclude name="testMethodToExclude"/> </methods> </class> </classes> </test> </pre> We can also include Regex: <methods> <exclude name="test.*"/> </methods>
Exclude Tests Based on Groups	<pre> @Test(groups = {"smoke"}) public void testMethod1() { // Test code for smoke group } <test name="Test"> <groups> <run> <include name="smoke"/> <exclude name="regression"/> </run> </groups> <classes> <class name="com.example.MyTestClass"/> </classes> </test> </pre>
Run Whole Package	<pre> <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd"> <suite name="Suite"> <test name="Test"> <packages> <package name="com.example.tests"/> </packages> </test> </suite> </pre>
Test Tagging	<pre> @Test(groups = {"smoke", "regression"}) public void testMethod1() { System.out.println("Test Method 1"); } <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Test Suite"> <test name="Test Methods"> <groups> <run> <include name="smoke"/> </run> </groups> <classes> <class name="org.saurav.TestSetup"/> <class name="org.saurav.TestMethods"/> <class name="org.saurav.TestExecution"/> </classes> </test> </suite> </pre> <u>to run beforeclass/method/test when using groups give @BeforeClass(alwaysRun = true)</u> <pre> @BeforeClass(alwaysRun = true) public void beforeClass() { System.out.println("Before Class - Executed before the first test method in the current class."); } @AfterClass(alwaysRun = true) public void afterClass() { System.out.println("After Class - Executed after all test methods in the current class."); } </pre>
dependsOnMethods	<pre> @Test(groups = {"smoke"}, dependsOnMethods = {"testMethod2", "testMethod1"}) else <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="TestNG XML Dependency Suite" > <test name="ToolsQA" > <groups> <dependencies> <group depends-on= "openbrowser" name= "login"/></group> </dependencies> </groups> <classes> <class name="GroupDependency" /> </classes> </test> </suite> </pre>
dependsOnGroups	<pre> @Test(groups = { "groupB"}, dependsOnGroups = {"groupA", "group2"}) </pre>
Order of Execution in Inheritance	TestNG runs @Before methods from the highest superclass to the subclass, and @After methods from the subclass back to the highest superclass
preserve-order	TestNG will run your tests in the order they are found in the XML file. If you want the classes and methods listed in this file to be run in an unpredictable order, set the preserve-order attribute to false

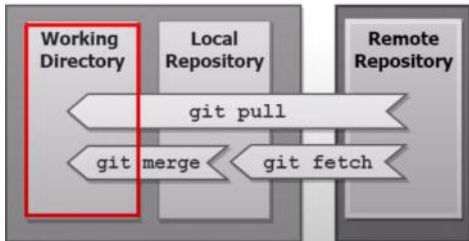
	<pre><!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" > <suite name="preserve-order Suite"> <test name="preserve-order Test" preserve-order="true"> <classes> <class name="Testing_Pack.Test_One"/> <class name="Testing_Pack.Test_Two"/> <class name="Testing_Pack.Test_Three"/> </classes> </test> </suite></pre> Tests will run as One,Two,Three, else it would have run as One,three,two	
@Ignore	<pre>@Ignore public class TestcaseSample { @Test public void testMethod1() { } @Test public void testMethod2() { } }</pre>	
Rerunning failed tests	Using testng-failed.xml • Run your TestNG suite: After the initial run, TestNG generates a testng-failed.xml file in the test-output folder. This file contains the list of failed test cases. • Execute testng-failed.xml: Right-click on this file and run it as a TestNG suite. This will re-execute only the failed test cases.	Using IRetryAnalyzer • Create a RetryAnalyzer class: Implement the IRetryAnalyzer interface and define a retry() method. This method determines whether a test case should be re-tried based on a certain condition (e.g., maximum retry count). • Apply the RetryAnalyzer to Test Methods: Use the retryAnalyzer attribute in the @Test annotation to specify the RetryAnalyzer class. • Run your TestNG suite: When a test case fails, the RetryAnalyzer will attempt to re-run it up to the specified maximum number of retries. <pre>----Setup RetryAnalyzer import org.testng.IRetryAnalyzer; import org.testng.ITestResult; public class RetryAnalyzer implements IRetryAnalyzer { private int count = 0; private static final int maxRetryCount = 3; public boolean retry(ITestResult result) { if (count < maxRetryCount) { count++; return true; } return false; } } ----Applying RetryAnalyzer to a Test Method @Test(retryAnalyzer = RetryAnalyzer.class) public void testMethod() { // Test logic here }</pre>
@Factory Annotation	<pre>import org.testng.annotations.Factory; public class TestFactory { @Factory public Object[] factoryMethod() { return new Object[] { new TestClass(1), new TestClass(2) }; } } // Test class import org.testng.annotations.Test; public class TestClass { private int parameter; public TestClass(int parameter) { this.parameter = parameter; } @Test public void testMethod() { System.out.println("Test method is running with parameter: " + parameter); // Add your test logic here using the 'parameter' } }</pre>	
@Test(invocationCount = 3)	InvocationCount is used when we want to run the same test multiple times. threadPoolSize controls how many threads TestNG will use to execute the invocations of a test method Ex: @Test(threadPoolSize = 3, invocationCount = 6) <pre>public void testMethod() { // Test logic }</pre>	

TestNG Priorities	@Test (priority = 1) If we have methods with priority 0,1,2,3 and some without any priority, First: 0 Priority Second: No priority Third: 1 In case of conflict, alphabetical order of testName		
Soft Assert	<pre>SoftAssert softassert = new SoftAssert(); softassert.assertEquals(originalTitle, expectedTitle); softassert.assertAll();</pre>		
Parallel	• Methods: This will run the parallel tests on all @Test methods in TestNG. • Tests: All the test cases present inside the <test> tag will run with this value. • Classes: All the test cases present inside the classes that exist in the XML will run in parallel. • Instances: This value will run all the test cases parallelly inside the same instance. <pre><?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Suite" parallel="methods" thread-count="2"> <test name="Test"> <parameter name="userName" value="saurav"/> <classes> <class name="TestRunner.TestNGTest"/> </classes> </test> <!-- Test --> <test name="Test2"> <parameter name="userName" value="singh"/> <classes> <class name="TestRunner.TestNGTest2"/> </classes> </test> </suite> <!-- Suite --></pre> default thread count for parallel execution is 5		
Exception handling	<pre>@Test(expectedExceptions = NumberFormatException.class) public void testNumberFormatException() { Integer.parseInt("abc"); // This will throw a NumberFormatException }</pre> This test will pass if exception occurs and fail when it doesn't		
ITestContext	ITestContext is used to store and share data across the tests in selenium by using TestNG framework. This interface provides information about the current test execution context. It allows you to retrieve data such as the suite name, test name, test parameters, and test status. • <code>getSuite()</code>: Get information about the test suite. • <code>getName()</code>: Get the name of the test. • <code>getStartDate()</code>: Get the start time of the test execution. • <code>getEndDate()</code>: Get the end time of the test execution. • <code>setAttribute(String name, Object value)</code>: Set an attribute in the context.//test level variable • <code>getSuite().setAttribute(String name, Object value)</code>: Set an attribute in the suite.//suite level variable • <code>getAttribute(String name)</code>: Retrieve an attribute from the context <pre>@BeforeTest public void SetData(ITestContext context){ String Customer_id = "C11012034"; context.setAttribute("CustID", Customer_id); System.out.println("Value is stored in ITestContext"); System.out.println("+++++++"); } @Test public void Test1a(ITestContext context){ String Customer_id1 = (String) context.getAttribute("CustID"); System.out.println("In Test1, Value stored in context is: "+Customer_id1); System.out.println("+++++++"); }</pre>		
ITestListener	ITestListener is an interface implemented in the class, and that class overrides the ITestListener-defined methods. It listens to the desired events and executes the methods accordingly. • <code>onStart()</code>: invoked after test class is instantiated and before execution of any testNG method. • <code>onTestSuccess()</code>: invoked on the success of a test • <code>onTestFailure()</code>: invoked on the failure of a test • <code>onTestSkipped()</code>: invoked when a test is skipped • <code>onTestFailedButWithinSuccessPercentage()</code>: invoked whenever a method fails but within the defined success percentage • <code>onFinish()</code>: invoked after all tests of a class are executedThe above-mentioned methods use the parameters ITestContext and ITestResult. The ITestContext is a class that contains information about the test run. The ITestResult is an interface that defines the result of the test. <table border="1"> <tr> <td>Testng.xml</td><td> <pre><?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Reporting Suite"> <listeners></pre> </td></tr> </table>	Testng.xml	<pre><?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Reporting Suite"> <listeners></pre>
Testng.xml	<pre><?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Reporting Suite"> <listeners></pre>		

	<pre> <listener class-name="com.example.listeners.MyTestListener" /> </listeners> <test name="Regression Tests"> <classes> <class name="com.example.tests.LoginTest" /> </classes> </test> </suite> </pre>
	<pre> @Listeners Annotation package com.example.tests; import org.testng.annotations.Listeners; import com.example.listeners.MyTestListener; // This ensures all child classes are tracked by the listener @Listeners(MyTestListener.class) public class BaseTest { // Common setup/teardown logic here } </pre>
AutoRead Prop Files	Default location src/test/resources <pre> ResourceBundle.getBundle("routes"); </pre>
Pass Data from xml to cucumber runner	<pre> public class TestRunner extends AbstractTestNGCucumberTests { @Override @DataProvider(parallel = false) public Object[][] scenarios() { return super.scenarios(); } @Parameters({ "browser" }) @BeforeMethod public static void setUpScenario(String browser) { WebDriverManager.initDriver(browser); } @AfterMethod public void tearDown(){ WebDriverManager.quitBrowser(); } } </pre>

Git

29 November 2024 08:21

Command	Description
git init	Initializes a new Git repository in the current directory.
git clone <url>	Clones an existing Git repository from 1 a remote URL.
git add <file>	Adds a file to the staging area.
git commit -m "<message>"	Commits the staged changes with a message.
git status	Shows the current state of the working directory.
git diff	Shows the difference between the current state and the previous commit.
git log	Shows the commit history.
git checkout <branch>	Switches to a different branch.
git branch <branch-name>	Creates a new branch.
git checkout -b <name>	creates and switches
git checkout commitHash	used to switch your working directory to the state of a specific commit. This allows you to view or inspect a project's state at a particular point in history. However, this action places you in a detached HEAD state, meaning you're not on any branch. You are not on any branch. Any changes made here will not be associated with a branch unless you explicitly create one. If you make changes and want to keep them, create a new branch: git checkout -b new-branch-name Modern Alternative: git switch --detach <commitHash>
git merge <branch>	Merges a branch into the current branch. git checkout master git pull origin master git checkout subBranch git merge master
git rebase -i HEAD~2	Combines multiple commits into a single commit to maintain a linear commit history add squash text before other commits to be merged
git push <remote> <branch> -f to force when you revert commit in local	Pushes changes to a remote repository.
git pull <remote> <branch>	Pulls changes from a remote repository.
git pull -r	fetch changes from a remote repository and integrate them into your current local branch using a rebase instead of the default merge .
git remote add <name> <url>	Adds a remote repository.
git remote show <name>	Shows information about a remote repository.
git fetch <remote>	Fetches changes from a remote repository to local repository. Need to merge in order to see changes in working directory. 
git stash	Temporarily saves changes to the working directory.
git stash pop	Restores the most recently stashed changes.
git config --global user.name "<name>"	Sets the global user name.
git config --global user.email "<email>"	Sets the global user email.
git config --global core.editor "<editor>"	Sets the global default editor.
Delete .idea from remote	1. Add to .gitignore

	2. git rm -r --cached . 3. Stage, commit and push the change
Delete untracked files	git clean -f
git reset	Staging to Working Directory
git reset --soft HEAD~1	revert commit and changes move to staging, hard will delete permanently. after ~ we can also give commit hash above which we want to delete things we can also give commit hash
git reset --hard HEAD~1	Revert commit and delete data
git revert <commit-hash> git revert <commit-hash1> <commit-hash2>	git revert is used to undo a specific commit by creating a new commit that reverses the changes made by the original commit. Unlike git reset, which can rewrite history, git revert is a safe operation because it doesn't alter the commit history, making it ideal for collaborative workflows. For Conflicts git add <file> git revert --continue git push
git bisect start git bisect bad [<commit>] git bisect good [<commit>]	used to perform a binary search through a repository's commit history to identify the commit that introduced a bug or issue. It is particularly useful in large repositories with numerous commits, as it narrows down the problematic commit efficiently. The process involves marking a good commit (where the code works) and a bad commit (where the bug exists). Git then automatically checks out commits in the middle of the range for you to test, helping narrow down the problematic commit. Git will now check out a commit halfway between the good and bad commits. Test this commit to determine if it's good or bad: If the commit is bad, mark it as: git bisect bad else git bisect good
git commit --amend -m "New Message"	Fix a Commit Message
git add <file> git commit --amend	Add Changes to the Last Commit
git reset --soft HEAD~1 git restore --staged <file> git commit --amend	Remove Changes from the Last Commit

Aspect	Merge	Rebase	Squash
History	Preserves branch history with merge commits	Creates a linear history. Reapplies commits from one branch on top of another, creating a linear history by rewriting commit history.	Combines multiple commits into one
Commit Granularity	Keeps all original commits intact Creates One more commit	Keeps individual commits intact No Additional Commits	Reduces all commits to a single one
Complexity	Simple, no rewriting	More complex, rewrites history	Requires manual consolidation
Risk of Conflicts	During the merge process only	During the rebase process	During squashing, if conflicts exist
Use in Shared Branches	Safe	Risky	Risky unless handled carefully
Clean History	Can be messy	Clean and linear	Very clean and concise

Maven

16 February 2024 08:36

Maven is a build automation tool used primarily for Java projects. It simplifies project management by handling:

- Dependency management (Automatic JAR downloads)
- Build process (Compiling, testing, packaging, and deploying)
- Project structure standardization
- Plugin-based extensibility

Maven Coordinates: Group id, artifact id, packaging and Version

Maven Lifecycle

clean	Removes previous build artifacts (target/ folder)
validate	Checks if the project is correct
compile	Compiles the source code (.java → .class)
test	Runs unit tests (src/test/java)
package	Creates a JAR/WAR (target/my-app.jar)
verify	Runs integration tests (src/test/java using Failsafe)
install	Installs the package into the local Maven repository
deploy	Deploys the package to a remote repository
site	Generates project documentation

mvn clean install: This will **clean**, **compile**, **test**, **package**, **verify**, and **install** your project.

Settings.xml is used to modify behavior of Maven

Bin Folder: Contains scripts for executing Maven.

Boot: Contains a jar file responsible for creating a class loader for Maven execution

Conf: This has settings.xml which when customised is placed under m2 directory

Lib: Contains Maven core classes for building. Includes Maven dependencies and Java components necessary for Maven's operation.

```
mvn archetype:generate -DarchetypeGroupId=org.apache.maven.archetypes -DarchetypeArtifactId=maven-archetype-quickstart -DgroupId=com.example -DartifactId=my-project -Dversion=1.0-SNAPSHOT -DinteractiveMode=false
```

Maven goals represent individual tasks that can be executed during the build process, while plugins are extensions that provide additional functionality and encapsulate collections of goals. Together, goals and plugins enable Maven to efficiently manage and automate the build lifecycle of a project

Scopes

1. Compile is default scope: Available in all classpaths and propagated to dependent projects
2. test: Only available for test compilation and execution phases
3. provided: Available during compilation and testing, but not at runtime. Not for packaging
4. runtime: Not available during compilation, but available at runtime. Not for Compilation

maven-surefire-plugin (For Unit Testing)	Runs JUnit and TestNG tests in src/test/java(*test.java) Used in the test phase Fails the build on test failures
maven-failsafe-plugin (For Integration Testing)	Runs integration tests (*IT.java) Executes in integration-test & verify phases Ensures tests don't break the build immediately

Extent Reports

25 March 2025 08:57

Core Classes	• ExtentReports: Main engine responsible for Creating tests, Recording status and Writing report. ExtentReports extent = new ExtentReports(); • ExtentSparkReporter: Responsible for HTML report creation, Appearance/configuration and Report title and theme. ExtentSparkReporter spark = new ExtentSparkReporter("reports/index.html");
Setup	• Create reporter: String path = System.getProperty("user.dir") + "/reports/index.html"; ExtentSparkReporter spark = new ExtentSparkReporter(path); • Configure report spark.config().setReportName("Web Automation Results"); spark.config().setDocumentTitle("Test Results"); • Create ExtentReports object ExtentReports extent = new ExtentReports(); extent.attachReporter(spark); extent.setSystemInfo("Tester", "Saurav"); extent.setSystemInfo("Environment", "QA"); extent.setSystemInfo("Browser", "Chrome");
Create Test	ExtentTest test = extent.createTest("Login Test"); Every test case should have its own ExtentTest.
Logging	• Pass: test.pass("Login successful"); • Fail: test.fail("Login failed"); • Info: test.info("Entering credentials"); • Warning: test.warning("Password field is slow"); • Skip: test.skip("Skipped due to dependency failure");
Flush	Without this, report won't be written. extent.flush();
Thread-Safe Parallel Execution	• Setup: ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>(); • Inside listener: extentTest.set(extent.createTest(result.getMethod().getMethodName())); • Retrieve: extentTest.get().pass("Test passed"); Without ThreadLocal, parallel execution corrupts reports.
ExtentManager	<pre>package utils; import com.aventstack.extentreports.ExtentReports; import com.aventstack.extentreports.reporter.ExtentSparkReporter; public class ExtentManager { private static ExtentReports extent; private ExtentManager(){} public static ExtentReports getInstance() { if(extent == null){ String reportPath = System.getProperty("user.dir")+ "/reports/ExtentReport.html"; ExtentSparkReporter spark = new ExtentSparkReporter(reportPath); spark.config().setDocumentTitle("Automation Report"); spark.config().setReportName("Regression Suite"); extent = new ExtentReports(); extent.attachReporter(spark); extent.setSystemInfo("Tester","Saurav"); extent.setSystemInfo("Framework","Cucumber"); extent.setSystemInfo("Browser","Chrome"); extent.setSystemInfo("Environment","QA"); } return extent; } }</pre>
ExtentTestManager	<pre>package utils; import com.aventstack.extentreports.ExtentTest; public class ExtentTestManager {</pre>

	<pre> private static final ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>(); public static ExtentTest getTest() { return extentTest.get(); } public static void setTest(ExtentTest test) { extentTest.set(test); } public static void unload() { extentTest.remove(); } } </pre>
ScreenshotUtils	<pre> package utils; import org.apache.commons.io.FileUtils; import org.openqa.selenium.*; import java.io.File; import java.io.IOException; public class ScreenshotUtils { public static String capture(WebDriver driver, String scenarioName) throws IOException { File source = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE); String destination = System.getProperty("user.dir")+ "/reports/screenshots/"+ scenarioName.replace(" ", "_")+ ".png"; FileUtils.copyFile(source,new File(destination)); return destination; } } </pre>
Hooks	<pre> package hooks; import com.aventstack.extentreports.Status; import io.cucumber.java.*; import org.openqa.selenium.WebDriver; import utils.*; public class Hooks { private static final var extent = ExtentManager.getInstance(); private WebDriver driver; @Before public void beforeScenario(Scenario scenario){ ExtentTestManager.setTest(extent.createTest(scenario.getName())); } @After public void afterScenario(Scenario scenario) throws Exception{ if(scenario.isFailed()){ String screenshot =ScreenshotUtils.capture(driver,scenario.getName()); ExtentTestManager.getTest().fail("Scenario Failed").addScreenCaptureFromPath(screenshot); }else{ ExtentTestManager.getTest().pass("Scenario Passed"); } extent.flush();// this should be done from @AfterSuite(testng in case of large tests ExtentTestManager.unload(); } } </pre>
Logging inside Step Definitions	<pre> ExtentTestManager.getTest().info("Launching Browser"); ExtentTestManager.getTest().info("Entering Username"); ExtentTestManager.getTest().info("Click Login"); </pre>

Component	Responsibility
ExtentManager	Create the singleton ExtentReports instance
ExtentTestManager	Manage ThreadLocal<ExtentTest>
Hooks @Before	Create ExtentTest for the scenario
Hooks @AfterStep	Log step status
Hooks @After	Capture screenshots, finalize the scenario, remove ThreadLocal
@AfterSuite (TestNG)	Call extent.flush() exactly once

```

public class ExtentReportManager {

    public static ExtentReports extentReports;

    public static ExtentReports createInstance(String fileName, String reportName, String documentTitle) {
        ExtentSparkReporter extentSparkReporter = new ExtentSparkReporter(fileName);
        extentSparkReporter.config().setReportName(reportName);
        extentSparkReporter.config().setDocumentTitle(documentTitle);
        extentSparkReporter.config().setTheme(Theme.STANDARD);
        extentSparkReporter.config().setEncoding("utf-8");

        extentReports = new ExtentReports();
        extentReports.attachReporter(extentSparkReporter);
        return extentReports;
    }

    public static String getReportNameWithTimeStamp() {
        DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy_MM_dd_HH_mm_ss");
        LocalDateTime localDateTime = LocalDateTime.now();
        String formattedTime = dateTimeFormatter.format(localDateTime);
        return "TestReport" + formattedTime + ".html";
    }

    public static void logPassDetails(String log) {
        ExtentReportConfigListener.extentTest.get().pass(MarkupHelper.createLabel(log, ExtentColor.GREEN));
    }

    public static void logFailureDetails(String log) {
        ExtentReportConfigListener.extentTest.get().fail(MarkupHelper.createLabel(log, ExtentColor.RED));
    }
}

public class ExtentReportConfigListener implements ITestListener {

    private static ExtentReports extentReports;
    public static ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>();

    public void onStart(ITestContext context) {
        String fileName = ExtentReportManager.getReportNameWithTimeStamp();
        String fullReportPath = System.getProperty("user.dir") + "\\reports\\" + fileName;
        extentReports = ExtentReportManager.createInstance(fullReportPath, "Test API Automation Report",
            "Test ExecutionReport");
    }

    public void onFinish(ITestContext context) {
        if (extentReports != null)
            extentReports.flush();
    }

    public void onTestStart(ITestResult result) {
        ExtentTest test = extentReports.createTest(
            "Test Name " + result.getTestClass().getName() + " - " + result.getMethod().getMethodName());
        extentTest.set(test);
    }

    public void onTestFailure(ITestResult result) {
        ExtentReportManager.logFailureDetails(result.getThrowable().getMessage());
        String stackTrace = Arrays.toString(result.getThrowable().getStackTrace());
        stackTrace = stackTrace.replaceAll(",", "<br>");
        String formmatedTrace = "<details>\n" + "    <summary>Click Here To See Exception Logs</summary>\n" + "    "
            + stackTrace + "\n" + "</details>\n";
        ExtentReportManager.logExceptionDetails(formmatedTrace);
    }
}

```

Log4j2.xml

25 March 2025 08:58

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="INFO">
  <Properties>
    <Property name="basePath">./logs</Property>
  </Properties>
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
    </Console>
    <RollingFile name="File" fileName="${basePath}/automation.log"
      filePattern="${basePath}/automation-%d{yyyy-MM-dd}.log">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
      <SizeBasedTriggeringPolicy size="50000"/>
    </RollingFile>
  </Appenders>
  <Loggers>
    <Root level="Info"><!-- Here you can use multiple log levels All < Trace < Debug < Info < Warn < Error < Fatal < Off -->
      <AppenderRef ref="File"/> <!-- <AppenderRef ref="Console"/> -->
    </Root>
  </Loggers>
</Configuration>
```

<code><Configuration status="WARN"></code>	• The <code><Configuration></code> tag is the root element that encapsulates all Log4j2 configurations. • The status attribute controls Log4j2's internal debugging logs, which are useful for diagnosing configuration issues. • Options for status: • OFF: No internal logs from Log4j2. • ERROR: Logs only errors in Log4j2 configuration. • WARN (default): Logs warnings and errors about configuration issues. • INFO: Provides more detailed information, such as component initialization. • DEBUG: Outputs detailed logs, including internal processing of Log4j2. • TRACE: Most verbose, showing fine-grained details of Log4j2 operations				
<code><Properties> <Property name="basePath">./logs</Property> </Properties></code>	Defines reusable key-value pairs for configuration. These properties can be referenced elsewhere in the file using <code>\${propertyName}</code>. Common Usage: Set file paths, environment variables, or dynamic configurations. Example: <code>\${env:USER_HOME}</code> to get the user's home directory.				
<code><Appenders> ... </Appenders></code>	<table><tr><td data-bbox="502 1317 678 1556">Console Appender</td><td data-bbox="678 1317 1583 1556"><pre><Console name="Console" target="SYSTEM_OUT"> <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/> </Console></pre> Purpose: Logs messages to the console (stdout or stderr). Options for target: SYSTEM_OUT: Writes logs to standard output (stdout). SYSTEM_ERR: Writes logs to standard error (stderr). </td></tr><tr><td data-bbox="502 1556 678 2112">File Appender</td><td data-bbox="678 1556 1583 2112"><pre><RollingFile name="File" fileName="\${basePath}/automation.log" filePattern="\${basePath}/automation-%d{yyyy-MM-dd}.log"> ... </RollingFile></pre> Purpose: Writes logs to files and supports file rollover. Attributes: fileName: The current log file. filePattern: The pattern for rollover files (e.g., <code>\${basePath}/automation-%d{yyyy-MM-dd}.log</code> for daily logs). Triggering Policies: • SizeBasedTriggeringPolicy: Rolls over when the file exceeds a specific size: <code><SizeBasedTriggeringPolicy size="50MB"/></code> • TimeBasedTriggeringPolicy: Rolls over at specific time intervals: <code><TimeBasedTriggeringPolicy interval="1" modulate="true"/></code> • CompositeTriggeringPolicy: <pre><CompositeTriggeringPolicy> <Policies></pre></td></tr></table>	Console Appender	<pre><Console name="Console" target="SYSTEM_OUT"> <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/> </Console></pre> Purpose: Logs messages to the console (stdout or stderr). Options for target: SYSTEM_OUT: Writes logs to standard output (stdout). SYSTEM_ERR: Writes logs to standard error (stderr).	File Appender	<pre><RollingFile name="File" fileName="\${basePath}/automation.log" filePattern="\${basePath}/automation-%d{yyyy-MM-dd}.log"> ... </RollingFile></pre> Purpose: Writes logs to files and supports file rollover. Attributes: fileName: The current log file. filePattern: The pattern for rollover files (e.g., <code>\${basePath}/automation-%d{yyyy-MM-dd}.log</code> for daily logs). Triggering Policies: • SizeBasedTriggeringPolicy: Rolls over when the file exceeds a specific size: <code><SizeBasedTriggeringPolicy size="50MB"/></code> • TimeBasedTriggeringPolicy: Rolls over at specific time intervals: <code><TimeBasedTriggeringPolicy interval="1" modulate="true"/></code> • CompositeTriggeringPolicy: <pre><CompositeTriggeringPolicy> <Policies></pre>
Console Appender	<pre><Console name="Console" target="SYSTEM_OUT"> <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/> </Console></pre> Purpose: Logs messages to the console (stdout or stderr). Options for target: SYSTEM_OUT: Writes logs to standard output (stdout). SYSTEM_ERR: Writes logs to standard error (stderr).				
File Appender	<pre><RollingFile name="File" fileName="\${basePath}/automation.log" filePattern="\${basePath}/automation-%d{yyyy-MM-dd}.log"> ... </RollingFile></pre> Purpose: Writes logs to files and supports file rollover. Attributes: fileName: The current log file. filePattern: The pattern for rollover files (e.g., <code>\${basePath}/automation-%d{yyyy-MM-dd}.log</code> for daily logs). Triggering Policies: • SizeBasedTriggeringPolicy: Rolls over when the file exceeds a specific size: <code><SizeBasedTriggeringPolicy size="50MB"/></code> • TimeBasedTriggeringPolicy: Rolls over at specific time intervals: <code><TimeBasedTriggeringPolicy interval="1" modulate="true"/></code> • CompositeTriggeringPolicy: <pre><CompositeTriggeringPolicy> <Policies></pre>				

		<pre> <SizeBasedTriggeringPolicy size="50MB"/> <TimeBasedTriggeringPolicy interval="1"/> </Policies> </CompositeTriggeringPolicy> </pre>
PatternLayout	<pre> <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/> </pre> Common Pattern Elements: %d{pattern}: Date and time (e.g., %d{yyyy-MM-dd HH:mm:ss}). %t: Thread name. %level: Log level (INFO, DEBUG, etc.). %logger{length}: Logger name truncated to the specified length. %msg: The actual log message. %n: Newline.	
Loggers Section	Root	<pre> <Root level="Info"> <AppenderRef ref="File"/> </Root> </pre> The root logger applies globally to all loggers unless overridden. Attributes: level: Minimum level of logs to capture (e.g., DEBUG, INFO, WARN). AppenderRef: Specifies which appenders the logger uses.
	Custom Loggers	<pre> <Logger name="com.example" level="DEBUG" additivity="false"> <AppenderRef ref="Console"/> </Logger> </pre> Attributes: name: Logger name (usually a package or class name). level: Minimum log level for this logger. additivity: Whether to inherit appenders from parent loggers.

WebDriverListener

25 March 2025 08:59

Introduction	It intercepts and logs various events that occur during WebDriver interactions, providing insights into the execution flow and potential issues.
DriverListener	<pre>public class DriverListener implements WebDriverListener { private final WebDriver driver; public DriverListener(WebDriver driver) { this.driver = driver; } @Override public void beforeGetTitle(WebDriver driver) { System.out.println("Getting the title"); } @Override public void beforeClick(WebElement element) { takeScreenshot("before_click"); } @Override public void afterClick(WebElement element) { takeScreenshot("after_click"); } @Override public void onError(Object target, Method method, Object[] args, InvocationTargetException e) { takeScreenshot("On Error"); } // public void takeScreenshot(String message) { // String base64Screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BASE64); // ExtentCucumberAdapter.addTestStepLog("Screenshot"); // ExtentCucumberAdapter.getCurrentStep() // .info(message, MediaEntityBuilder.createScreenCaptureFromBase64String(base64Screenshot).build()); // } public void takeScreenshot(String message) { try { File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE); String timestamp = new SimpleDateFormat("dd MMM yy HH mm ss").format(new Date()); String screenshotName = "screenshot_" + timestamp + ".png"; String destPath = System.getProperty("user.dir") + "/ExtentReports/screenshots/" + screenshotName; FileUtils.copyFile(screenshot, new File(destPath)); ExtentCucumberAdapter.addTestStepLog("Screenshot:"); ExtentCucumberAdapter.getCurrentStep().info(message, MediaEntityBuilder.createScreenCaptureFromPath(destPath).build()); } catch (IOException e) { e.printStackTrace(); } } }</pre>
Extent.properties	<pre>extent.reporter.spark.start=true extent.reporter.spark.out=/AutomationTestResult.html #FolderName basefolder.name=ExtentReports/SparkReport_ basefolder.datetimestamppattern=d MMM YY HH mm ss</pre>
EventFiringDecorator	EventFiringDecorator acts as a wrapper (a design pattern called the Decorator pattern) around your actual WebDriver instance (e.g., ChromeDriver, FirefoxDriver) When you call a method on decoratedDriver (e.g., decoratedDriver.get("...") or button.click()), the EventFiringDecorator intercepts that call. • Before Event: Before executing the actual method on the originalDriver, the EventFiringDecorator checks if the corresponding "before" method is implemented in any of the registered WebDriverListeners. If beforeClick is implemented in MyLoggingListener, it calls listener.beforeClick(element). ◦ Actual Method Execution: The EventFiringDecorator then calls the actual click() method on the originalDriver. • After Event: After the actual method completes successfully, the EventFiringDecorator checks for and calls the corresponding "after" method (e.g., listener.afterClick(element)). • Error Event: If the actual method throws an exception, the EventFiringDecorator catches it and calls the listener.onError() method. ----- The EventFiringDecorator acts as a wrapper (using the Decorator Pattern) around your actual WebDriver instance. However, the engine that makes this possible without you writing manual "if-then" statements for every click is Dynamic Proxies and Java Reflection. 1. The Component: Dynamic Proxy When you call new EventFiringDecorator(listener).decorate(driver), Selenium doesn't just return your driver; it creates a Proxy Object at runtime. • The Interceptor: This Proxy sits between your code and the real Browser Driver. • The Catch-All: Every time you call a method like click() or sendKeys(), it doesn't go straight to the driver. It hits the Proxy first. 2. The Engine: Java Reflection The Proxy uses Java Reflection to inspect what you are trying to do. It essentially "looks" at the method name and arguments you sent. • Step A: Discovery: If you call element.click(), Reflection tells the Proxy: <i>"The user is calling the 'click' method on a WebElement."</i> • Step B: The Look-up: The Proxy scans your DriverListener class to see if you have a method that matches the pattern beforeClick. • Step C: The Dispatch: It invokes your listener method automatically using Reflection, then passes the original command to the real driver. 3. The Execution Flow (The "Handshake") When you execute button.click() on a decorated driver, the following sequence occurs: 1. Interception: The Proxy catches the click() call. 2. Before Event: The Proxy uses Reflection to find and execute listener.beforeClick(element). 3. Actual Execution: The Proxy then calls the "real" click() on the original WebDriver (ChromeDriver, etc.). ◦ <i>Note: This is where the actual browser interaction happens.</i> 4. Error Handling (Reflection-based): If the real click() throws an exception (e.g., NoSuchElementException), the Proxy catches it and immediately routes it to listener.onError(). 5. After Event: If successful, the Proxy uses Reflection to find and execute listener.afterClick(element). Without Reflection, the Selenium developers would have had to write a massive, hard-coded switch statement for every single possible action in the WebDriver

	API. Because of Reflection, the system is decoupled: the Decorator doesn't need to "know" what's inside your listener; it just looks for method names that match the events it's intercepting. Key Takeaway: You provide the Logic (the Listener), Selenium provides the Interception (the Proxy), and Java provides the Eyes to see which method to call (Reflection).
Decorate Your Driver	<pre>// 1. Create your actual WebDriver instance WebDriver originalDriver = new ChromeDriver(); // 2. Create your listener WebDriverListener listener = new MyLoggingListener(); // 3. Decorate your original driver with the listener WebDriver decoratedDriver = new EventFiringDecorator(listener).decorate(originalDriver); // Now, use the decoratedDriver for your interactions decoratedDriver.get("https://www.example.com"); WebElement button = decoratedDriver.findElement(By.id("someButton")); button.click(); // This will trigger the beforeClick and afterClick events in MyLoggingListener decoratedDriver.quit();</pre>

Miscellaneous

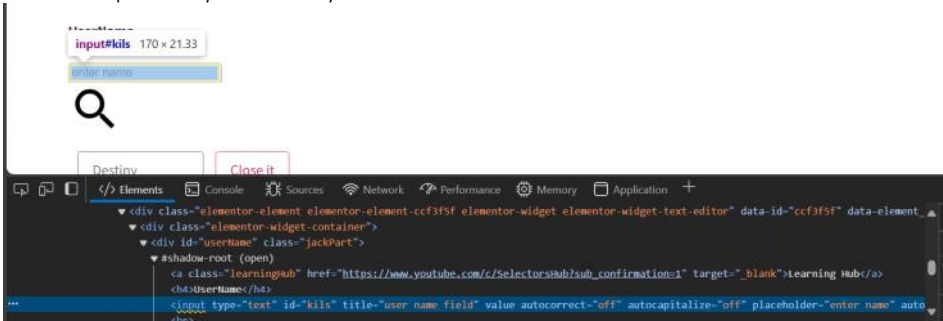
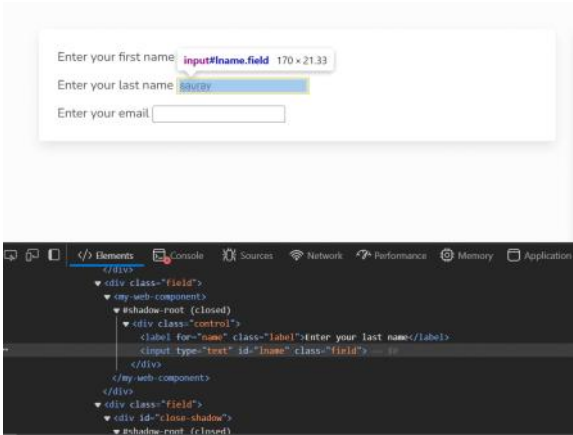
21 May 2018 14:28

WebDriver driver=new ChromeDriver();	WebDriver is an interface, while ChromeDriver is a class that implements it. By using the interface as the reference type, your code becomes "browser-neutral." WebDriver (Interface): WebDriver driver = new ChromeDriver(); You can swap new ChromeDriver() for new FirefoxDriver() or new EdgeDriver() without changing any other part of your test script. This makes the code loosely coupled(Dependency Inversion Principle and LSP).		
Selenium Manager	Selenium Manager is a built-in CLI tool introduced in Selenium 4.6+ that automatically manages browser drivers (chromedriver, geckodriver, etc.) without manual setup. It detects your browser version and downloads the matching driver at runtime. To bypass it and use a local/manual driver path in Java: System.setProperty("webdriver.chrome.driver", "/path/to/chromedriver"); WebDriver driver = new ChromeDriver();		
Screenshot of web element	<pre>WebElement logo = driver.findElement(By.xpath("//div[@id='divLogo']/img")); File file = logo.getScreenshotAs(OutputType.FILE); File destFile = new File("logo.png"); FileUtils.copyFile(file, destFile);</pre>		
Selenium 4 new tab	In Selenium 4, users can open a URL in a new tab along with the existing tab. For example, if the user wants to open 2 URLs in two different tabs simultaneously, it can be done with Selenium 4. <pre>driver.get("https://www.google.com/"); driver.switchTo().newWindow(WindowType.TAB); driver.navigate().to("https://www.crmpro.com/"); //switch to().window() is still required to operate on original window</pre>		
Open a new window	Using Selenium 4, users can open a new window on the same browser. For example, if the user wants to access two applications in the same browser, it can now be achieved. <pre>driver.get("https://www.google.com/"); driver.switchTo().newWindow(WindowType.WINDOW); driver.navigate().to("https://www.crmpro.com/"); //switch to().window() is still required to operate on original window</pre>		
Object Location	Selenium 4 enables users to obtain coordinates, dimensions, height, width, etc., as the location of web elements or objects. <pre>WebElement logo1 = driver.findElement(By.xpath("//div[@id='divLogo']/img")); System.out.println("Height: " + logo1.getRect().getDimension().getHeight()); System.out.println("Width: " + logo1.getRect().getDimension().getWidth()); System.out.println("X Location: " + logo1.getRect().getX()); System.out.println("Y Location: " + logo1.getRect().getY());</pre>		
Relative Locators	These are also known as Friendly Locators, and this functionality is being added to find out the element which is present nearby to other web element or, we can say that it can find the web elements based on GUI location. There are five locators added in Selenium 4: below(): Web element located below for the specified element. toLeftOf(): Target web element which is present to the left of specified element. toRightOf(): Target web element which is presented to the right of a specified element. above(): Web element located above for the specified element. near(): Target web element which is away(approx. 50 pixels) from the specified element. sendKeys to Email Field: driver.findElement(RelativeLocator.with(By.tagName("input")).above(password)).sendKeys("ab@rmal.com"); Chaining of Relative Locators By submitLocator = RelativeLocator.with(By.tagName("button")).below(By.id("email")).toRightOf(By.id("cancel"));		
Chrome Dev tools	Selenium 4 introduces changes in the Chrome DevTools API. The Chrome driver class now extends the Chromium Driver class, providing access to predefined methods for dev tool operations. Note: Chromium Driver extends the Remote Web driver class. Operations include: - Enable Network Offline - Enable Network Online - Get Console Logs - Load Insecure Website <table border="1"><tr><td>Browser Emulation</td><td><pre>@Test public void testMobileEmulation() throws InterruptedException { WebDriverManager.chromedriver().setup(); ChromeDriver driver=new ChromeDriver(); driver.manage().window().maximize(); DevTools devtools=driver.getDevTools(); devtools.createSession(); devtools.send(Emulation .setDeviceMetricsOverride(600, 1000, 50, true, Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty())); driver.get("https://raahulshettyacademy.com/angularAppdemo/"); driver.findElement(By.cssSelector(".navbar-toggler")).click(); Thread.sleep(3000); driver.findElement(By.linkText("Library")).click(); }</pre></td></tr></table>	Browser Emulation	<pre>@Test public void testMobileEmulation() throws InterruptedException { WebDriverManager.chromedriver().setup(); ChromeDriver driver=new ChromeDriver(); driver.manage().window().maximize(); DevTools devtools=driver.getDevTools(); devtools.createSession(); devtools.send(Emulation .setDeviceMetricsOverride(600, 1000, 50, true, Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty())); driver.get("https://raahulshettyacademy.com/angularAppdemo/"); driver.findElement(By.cssSelector(".navbar-toggler")).click(); Thread.sleep(3000); driver.findElement(By.linkText("Library")).click(); }</pre>
Browser Emulation	<pre>@Test public void testMobileEmulation() throws InterruptedException { WebDriverManager.chromedriver().setup(); ChromeDriver driver=new ChromeDriver(); driver.manage().window().maximize(); DevTools devtools=driver.getDevTools(); devtools.createSession(); devtools.send(Emulation .setDeviceMetricsOverride(600, 1000, 50, true, Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty())); driver.get("https://raahulshettyacademy.com/angularAppdemo/"); driver.findElement(By.cssSelector(".navbar-toggler")).click(); Thread.sleep(3000); driver.findElement(By.linkText("Library")).click(); }</pre>		

	Geo Location @Test public void geoTest() { WebDriverManager.chromedriver().setup(); ChromeDriver driver=new ChromeDriver(); driver.manage().window().maximize(); DevTools devtools=driver.getDevTools(); devtools.createSession(); Map<String,Object> geoLocale=new HashMap<>(); geoLocale.put("latitude",35.652832); geoLocale.put("longitude", 139.839478); geoLocale.put("accuracy", 100); driver.executeCdpCommand("Emulation.setGeolocationOverride",geoLocale); driver.get("https://www.google.com"); driver.navigate().refresh(); }																												
Action Class Methods	click(WebElement): Replaces moveToElement(onElement).click(). Used for clicking on a specific web element. clickAndHold(WebElement): Replaces moveToElement(onElement).clickAndHold(). Clicks on an element without releasing the click. contextClick(WebElement): Replaces moveToElement(onElement).contextClick(). Performs the right-click operation. doubleClick(WebElement): Replaces moveToElement(element).doubleClick(). Performs a double click on an element.																												
Options	In Selenium 3, capabilities were defined in a session by using Desired Capabilities classes. As of Selenium 4, you must use the browser options classes. For remote driver sessions, a browser options instance is required as it determines which browser will be used. <table><tr><td>browserName</td><td> Browser name is set by default when using an Options class instance. ChromeOptions chromeOptions = new ChromeOptions(); String name = chromeOptions.getBrowserName(); </td></tr><tr><td>browserVersion</td><td> This capability is optional, this is used to set the available browser version at remote end. In recent versions of Selenium, if the version is not found on the system, it will be automatically downloaded by Selenium Manager ChromeOptions chromeOptions = new ChromeOptions(); String version = "latest"; chromeOptions.setBrowserVersion(version); </td></tr><tr><td>pageLoadStrategy</td><td> The document.readyState property of a document describes the loading state of the current document. options.setPageLoadStrategy(PageLoadStrategy.NORMAL); <table><tr><td>Strategy</td><td>Ready State</td><td>Purpose</td></tr><tr><td>normal</td><td>complete</td><td>Used by default, waits for all resources to download</td></tr><tr><td>eager</td><td>interactive</td><td>DOM access is ready, but other resources like images may still be loading</td></tr><tr><td>none</td><td>Any</td><td>Does not block WebDriver at all</td></tr></table> When navigating to a new page via URL, by default, WebDriver will hold off on completing a navigation method (e.g., driver.navigate().get()) until the document ready state is complete. This does not necessarily mean that the page has finished loading, especially for sites like Single Page Applications that use JavaScript to dynamically load content after the Ready State returns complete. Note also that this behavior does not apply to navigation that is a result of clicking an element or submitting a form. If a page takes a long time to load as a result of downloading assets (e.g., images, css, js) that aren't important to the automation, you can change from the default parameter of normal to eager or none to speed up the session. This value applies to the entire session, so make sure that your waiting strategy is sufficient to minimize flakiness. </td></tr><tr><td>platformName</td><td> This identifies the operating system at the remote-end, fetching the platformName returns the OS name. In cloud-based providers, setting platformName sets the OS at the remote-end. ChromeOptions chromeOptions = new ChromeOptions(); String platform = "OS X 10.6"; chromeOptions.setPlatformName(platform); </td></tr><tr><td>acceptInsecureCerts</td><td> This capability checks whether an expired (or) invalid TLS Certificate is used while navigating during a session. If the capability is set to false, an insecure certificate error will be returned as navigation encounters any domain certificate problems. If set to true, invalid certificate will be trusted by the browser. All self-signed certificates will be trusted by this capability by default. Once set, acceptInsecureCerts capability will have an effect for the entire session. ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setAcceptInsecureCerts(true); </td></tr><tr><td>Timeouts</td><td> A WebDriver session is imposed with a certain session timeout interval, during which the user can control the behaviour of executing scripts or retrieving information from the browser. Each session timeout is configured with combination of different timeouts as described below: <table><tr><td>Script Timeout</td><td> Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setScriptTimeout(duration); </td></tr><tr><td>Page Load Timeout</td><td> Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); </td></tr></table></td></tr></table>	browserName	Browser name is set by default when using an Options class instance. ChromeOptions chromeOptions = new ChromeOptions(); String name = chromeOptions.getBrowserName();	browserVersion	This capability is optional, this is used to set the available browser version at remote end. In recent versions of Selenium, if the version is not found on the system, it will be automatically downloaded by Selenium Manager ChromeOptions chromeOptions = new ChromeOptions(); String version = "latest"; chromeOptions.setBrowserVersion(version);	pageLoadStrategy	The document.readyState property of a document describes the loading state of the current document. options.setPageLoadStrategy(PageLoadStrategy.NORMAL); <table><tr><td>Strategy</td><td>Ready State</td><td>Purpose</td></tr><tr><td>normal</td><td>complete</td><td>Used by default, waits for all resources to download</td></tr><tr><td>eager</td><td>interactive</td><td>DOM access is ready, but other resources like images may still be loading</td></tr><tr><td>none</td><td>Any</td><td>Does not block WebDriver at all</td></tr></table> When navigating to a new page via URL, by default, WebDriver will hold off on completing a navigation method (e.g., driver.navigate().get()) until the document ready state is complete. This does not necessarily mean that the page has finished loading, especially for sites like Single Page Applications that use JavaScript to dynamically load content after the Ready State returns complete. Note also that this behavior does not apply to navigation that is a result of clicking an element or submitting a form. If a page takes a long time to load as a result of downloading assets (e.g., images, css, js) that aren't important to the automation, you can change from the default parameter of normal to eager or none to speed up the session. This value applies to the entire session, so make sure that your waiting strategy is sufficient to minimize flakiness.	Strategy	Ready State	Purpose	normal	complete	Used by default, waits for all resources to download	eager	interactive	DOM access is ready, but other resources like images may still be loading	none	Any	Does not block WebDriver at all	platformName	This identifies the operating system at the remote-end, fetching the platformName returns the OS name. In cloud-based providers, setting platformName sets the OS at the remote-end. ChromeOptions chromeOptions = new ChromeOptions(); String platform = "OS X 10.6"; chromeOptions.setPlatformName(platform);	acceptInsecureCerts	This capability checks whether an expired (or) invalid TLS Certificate is used while navigating during a session. If the capability is set to false, an insecure certificate error will be returned as navigation encounters any domain certificate problems. If set to true, invalid certificate will be trusted by the browser. All self-signed certificates will be trusted by this capability by default. Once set, acceptInsecureCerts capability will have an effect for the entire session. ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setAcceptInsecureCerts(true);	Timeouts	A WebDriver session is imposed with a certain session timeout interval, during which the user can control the behaviour of executing scripts or retrieving information from the browser. Each session timeout is configured with combination of different timeouts as described below: <table><tr><td>Script Timeout</td><td> Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setScriptTimeout(duration); </td></tr><tr><td>Page Load Timeout</td><td> Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); </td></tr></table>	Script Timeout	Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setScriptTimeout(duration);	Page Load Timeout	Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS);
browserName	Browser name is set by default when using an Options class instance. ChromeOptions chromeOptions = new ChromeOptions(); String name = chromeOptions.getBrowserName();																												
browserVersion	This capability is optional, this is used to set the available browser version at remote end. In recent versions of Selenium, if the version is not found on the system, it will be automatically downloaded by Selenium Manager ChromeOptions chromeOptions = new ChromeOptions(); String version = "latest"; chromeOptions.setBrowserVersion(version);																												
pageLoadStrategy	The document.readyState property of a document describes the loading state of the current document. options.setPageLoadStrategy(PageLoadStrategy.NORMAL); <table><tr><td>Strategy</td><td>Ready State</td><td>Purpose</td></tr><tr><td>normal</td><td>complete</td><td>Used by default, waits for all resources to download</td></tr><tr><td>eager</td><td>interactive</td><td>DOM access is ready, but other resources like images may still be loading</td></tr><tr><td>none</td><td>Any</td><td>Does not block WebDriver at all</td></tr></table> When navigating to a new page via URL, by default, WebDriver will hold off on completing a navigation method (e.g., driver.navigate().get()) until the document ready state is complete. This does not necessarily mean that the page has finished loading, especially for sites like Single Page Applications that use JavaScript to dynamically load content after the Ready State returns complete. Note also that this behavior does not apply to navigation that is a result of clicking an element or submitting a form. If a page takes a long time to load as a result of downloading assets (e.g., images, css, js) that aren't important to the automation, you can change from the default parameter of normal to eager or none to speed up the session. This value applies to the entire session, so make sure that your waiting strategy is sufficient to minimize flakiness.	Strategy	Ready State	Purpose	normal	complete	Used by default, waits for all resources to download	eager	interactive	DOM access is ready, but other resources like images may still be loading	none	Any	Does not block WebDriver at all																
Strategy	Ready State	Purpose																											
normal	complete	Used by default, waits for all resources to download																											
eager	interactive	DOM access is ready, but other resources like images may still be loading																											
none	Any	Does not block WebDriver at all																											
platformName	This identifies the operating system at the remote-end, fetching the platformName returns the OS name. In cloud-based providers, setting platformName sets the OS at the remote-end. ChromeOptions chromeOptions = new ChromeOptions(); String platform = "OS X 10.6"; chromeOptions.setPlatformName(platform);																												
acceptInsecureCerts	This capability checks whether an expired (or) invalid TLS Certificate is used while navigating during a session. If the capability is set to false, an insecure certificate error will be returned as navigation encounters any domain certificate problems. If set to true, invalid certificate will be trusted by the browser. All self-signed certificates will be trusted by this capability by default. Once set, acceptInsecureCerts capability will have an effect for the entire session. ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setAcceptInsecureCerts(true);																												
Timeouts	A WebDriver session is imposed with a certain session timeout interval, during which the user can control the behaviour of executing scripts or retrieving information from the browser. Each session timeout is configured with combination of different timeouts as described below: <table><tr><td>Script Timeout</td><td> Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setScriptTimeout(duration); </td></tr><tr><td>Page Load Timeout</td><td> Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); </td></tr></table>	Script Timeout	Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setScriptTimeout(duration);	Page Load Timeout	Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS);																								
Script Timeout	Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setScriptTimeout(duration);																												
Page Load Timeout	Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS);																												

	<pre>chromeOptions.setPageLoadTimeout(duration);</pre> Implicit Wait Timeout This specifies the time to wait for the implicit element location strategy when locating elements. The default timeout 0 is imposed when a new session is created by WebDriver. <pre>ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setImplicitWaitTimeout(duration);</pre>
unhandledPromptBehavior	Specifies the state of current session's user prompt handler. Defaults to dismiss and notify state • dismiss • accept • dismiss and notify • accept and notify • ignore <pre>ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setUnhandledPromptBehaviour(UnexpectedAlertBehaviour.DISMISS_AND_NOTIFY);</pre>
proxy	<pre>Proxy proxy = new Proxy(); proxy.setHttpProxy("<HOST:PORT>"); ChromeOptions options = new ChromeOptions(); options.setCapability("proxy", proxy);</pre>
Common Args	<pre>options.addArguments("--start-maximized");</pre> Commonly used args include --start-maximized, --headless=new and --user-data-dir=... <pre>options.addArguments("--blink-settings=imagesEnabled=false");</pre>
Download Directory	<pre>String downloadFilePath = "C:\\Your\\Custom\\Path"; Map<String, Object> prefs = new HashMap<>(); // Set the default download directory prefs.put("download.default_directory", downloadFilePath); // (Optional) Disable the "Save As" popup to ensure downloads are automatic prefs.put("download.prompt_for_download", false); // (Optional) Specific to PDF: open in browser vs download prefs.put("plugins.always_open_pdf_externally", true); // 3. Initialize ChromeOptions and set the experimental "prefs" ChromeOptions options = new ChromeOptions(); options.setExperimentalOption("prefs", prefs);</pre>
ChromeOptions.addArgument	• start-maximized: Opens Chrome in maximize mode: <code>options.addArguments("--start-maximized");</code> • incognito: Opens Chrome in incognito mode: <code>options.addArguments("--incognito");</code> • headless: Opens Chrome in headless mode: <code>options.addArguments("--headless");</code> • disable-extensions: Disables existing extensions on Chrome browser • disable-popup-blocking: Disables pop-ups displayed on Chrome browser • make-default-browser: Makes Chrome default browser • version: Prints chrome browser version • disable-infobars: Prevents Chrome from displaying the notification 'Chrome is being controlled by automated software' • Accept TLS: <code>options.setAcceptInsecureCerts(true);</code> • Change User Agent: "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36"
Add Chrome Extension	<pre>ChromeOptions options = getDefaultChromeOptions(); Path path = Paths.get("src/test/resources/extensions/webextensions-selenium-example.crx"); File extensionFilePath = new File(path.toUri()); options.addExtensions(extensionFilePath);</pre>
Set Window Size	<pre>Dimension dimension = new Dimension(1024, 768); driver.manage().window().setSize(dimension);</pre>
Right-click using WebDriver	<pre>Actions act = new Actions(driver); act.contextClick().perform();</pre>
Drag and drop using WebDriver	<pre>Actions act = new Actions(driver); WebElement source = driver.findElement(By.xpath("-----")); // source element WebElement target = driver.findElement(By.xpath("-----")); // target element act.dragAndDrop(source, target).perform();</pre>
Verify checkbox/radio state	<pre>boolean isChecked = element.isSelected();</pre>
Simulate browser back and forward	<pre>driver.navigate().back(); driver.navigate().forward(); driver.navigate().refresh();</pre>
Set Dimension Of Browser	<pre>Dimension d = new Dimension(420,600); driver.manage().window().setSize(d);</pre>
Double Click	<pre>Actions action = new Actions(driver); action.doubleClick(ele); action.perform();</pre>
Sending Keys to Element	<pre>Keyboard Keys with WebDriver: driver.findElement(By...).sendKeys(Keys.TAB);</pre>
isDisplayed()	to check visibility (only for hidden items in page source). To check if an element is present or not, use <code>size()</code> not <code>isDisplayed()</code>. <code>isSelected()</code>: Checks if the element is selected (e.g., checkboxes). <code>isDisplayed()</code>: Verifies if the element is present in the DOM and displayed on the web page. <code>isEnabled()</code>: Checks if the element is currently enabled for user interaction. Mostly textbox might be disabled. This function checks for disabled attribute to be true.
In headless more better to manually scroll to element else	<pre>JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("window.scrollTo(0,500)");</pre>

test might fail in some cases.	
Scroll Inside Table	<code>js.executeScript("document.querySelector('.tableFixHead').scrollTop=5000");</code> (scrollLeft is also a method)
Block PopUps	<code>ChromeOptions options = new ChromeOptions(); options.setExperimentalOption("excludeSwitches", Arrays.asList("disable-popup-blocking"));</code>
Taking Screenshot	<code>File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE); // Copy the screenshot to a destination file FileUtils.copyFile(src, new File("C:\\Users\\singh\\Desktop\\screenshot.png"));</code>
Carry Xpath	<code>public void selectValuefromDropdown(String dropdownLocator, String optionsLocator, String option) { clickElement (dropdownLocator, "Dropdown Clicked"); List<WebElement> dates = driver.findElements(By.xpath(optionsLocator)); dates.stream().filter(i > i.findElement(By.xpath("./span")).getText().equalsIgnoreCase(option)).limit(1).forEach(i -> i.click()); }</code>
Control Clicks	<code>for (int i = 1; i < count; i++) { String tabclick = Keys.chord(Keys.CONTROL, Keys.ENTER); scope.findElements(By.tagName("a")).get(i).sendKeys(tabclick); }</code>
Limit Scope	<code>WebElement footer= driver.findElement.... System.out.println(Footer.findElements(By.tagName("a")).size()); Using String.format: String xpath = String.format("//button[@id='%s']", buttonId);</code>
Broken Link Validator	<code>public static void verifyLink(String url) { try { URL link = new URL(url); URLConnection httpURLConnection = (URLConnection) link.openConnection(); httpURLConnection.setConnectTimeout(3000); // Set connection timeout to 3 seconds httpURLConnection.connect(); if (httpURLConnection.getResponseCode() == 200) { System.out.println(url + " - " + httpURLConnection.getResponseMessage()); } else { System.out.println(url + " - " + httpURLConnection.getResponseMessage() + " - " + "is a broken link"); } } catch (Exception e) { System.out.println(url + " - " + "is a broken link"); } } }</code>
Database	<code>import java.sql.Connection; import java.sql.DriverManager; import java.sql.ResultSet; import java.sql.Statement; public class DatabaseTest { public static void main(String[] args) { // Database connection details String url = "jdbc:mysql://localhost:3306/yourDatabase"; String user = "yourUsername"; String password = "yourPassword"; // Connect to the database try (Connection conn = DriverManager.getConnection(url, user, password)) { // Create a statement Statement stmt = conn.createStatement(); // Execute a query ResultSet rs = stmt.executeQuery("SELECT * FROM yourTable"); // Process the results while (rs.next()) { System.out.println(rs.getString("column1") + " " + rs.getString("column2")); } } catch (Exception e) { e.printStackTrace(); } } } Using Assertj DB public class DatabaseTest { public static void main(String[] args) { // Database connection details String url = "jdbc:mysql://localhost:3306/yourDatabase"; String user = "yourUsername"; String password = "yourPassword"; // Try-with-resources ensures automatic resource cleanup try (Connection conn = DriverManager.getConnection(url, user, password)) { // Create an AssertJ-DB Table object Table table = new Table(conn, "yourTable"); // Assert that the table contains data and verify column values Assertions.assertThat(table) .row(0) // Check first row .value("column1").isNotNull() // Ensure column1 is not null .value("column2").isEqualTo("ExpectedValue"); // Check expected value</code>

	<pre> System.out.println("Database assertion passed successfully!"); } catch (Exception e) { e.printStackTrace(); } } } </pre>
Shadow Dom(JS Executor)	Shadow DOM is a web platform feature that encapsulates a part of the DOM and hides its internal structure from the main document. This can make it challenging to locate elements using traditional Selenium locators like XPath or CSS selectors. <pre> // Access the shadow root JavascriptExecutor jsExecutor = (JavascriptExecutor) driver; WebElement shadowHost = driver.findElement(By.cssSelector("#shadow-host")); WebElement shadowRootElement = (WebElement) jsExecutor.executeScript("return arguments[0].shadowRoot", shadowHost); // Locate an element within the shadow root WebElement inputElement = (WebElement) jsExecutor.executeScript("return arguments[0].querySelector('input#shadow-input')", shadowRootElement); inputElement.sendKeys("Hello, Shadow DOM!"); </pre>
Shadow Dom(Selenium 4)	Components: - Shadow Host: This is the regular DOM element to which a shadow DOM is attached. - Shadow Tree: The DOM tree inside the shadow DOM. It is the root for the encapsulated DOM and CSS. - Shadow Boundary: The boundary between the shadow DOM and the light DOM (the regular DOM outside the shadow DOM). - Shadow Root: The root node of the shadow tree. It can be thought of as the entry point to the shadow DOM. <pre> WebElement shadowHost = driver.findElement(By.id("userName")); SearchContext shadowRoot = shadowHost.getShadowRoot(); WebElement usernameField = shadowRoot.findElement(By.cssSelector("#kils")); usernameField.sendKeys("your_username"); //document.querySelector("#userName").shadowRoot.querySelector("#app2").shadowRoot.querySelector("#pizza") WebElement shadowHost2 = shadowRoot.findElement(By.cssSelector("#app2")); SearchContext shadowRoot2 = shadowHost2.getShadowRoot(); WebElement pizzaText = shadowRoot2.findElement(By.cssSelector("#pizza")); pizzaText.sendKeys("Dominos"); </pre> Shadow-root open can only be identified by selenium  Closed Shadow Dom  <pre> js.executeScript("document.querySelector('my-web-component').myRoot.querySelector('#lname').value='saurav'"); </pre>
Quit Vs Close	Close: Closes Active Window, session id will be maintained Quit: Closes all window, session id=null
Methods in WebDriver interface	Some common methods include get, quit, close, getWindowHandle, getTitle, etc.
Scroll down to a specific element	<pre> JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("arguments[0].scrollIntoView(true);", element); Actions actions = new Actions(driver); actions.moveToElement(element).build().perform(); </pre>
How can you input text into a	<pre> JavascriptExecutor js = (JavascriptExecutor) driver; </pre>

field without using sendKeys()	<code>js.executeScript("document.getElementById('elementId').value='Your Text';");</code>
Get vs navigate	<code>get()</code>: Directly loads a new URL into the browser. <code>navigate()</code>: Provides more control over navigation, including: <code>forward()</code>: Navigates forward in the browser history. <code>back()</code>: Navigates backward in the browser history. <code>refresh()</code>: Reloads the current page. <code>driver.get()</code> : It's used to go to the particular website , But it doesn't maintain the browser History and cookies so , we can't use forward and backward button , if we click on that , page will not get schedule <code>driver.navigate()</code> : it's used to go to the particular website , but it maintains the browser history and cookies, so we can use forward and backward button to navigate between the pages during the coding of Testcase. The key difference is that the navigate API retains cookies with each use. Unlike the <code>get()</code> method, which clears the session state every time it is called, the <code>navigate()</code> method preserves the session state. <code>To()</code> calls <code>get()</code> internally.
Tooltips in Selenium	<code>String tooltipText = element.getAttribute("title");</code>
Default Timeout	Implicit Wait: 0 seconds (fails immediately if element not found) Page Load Timeout: 300 seconds (5 minutes) Async Script Timeout: 30 seconds
SHIFT+ENTER	<code>String press = Keys.chord(Keys.SHIFT,Keys.ENTER);</code> <code>webElement.sendKeys(press);</code>
getCssValue	<code>String color = element.getCssValue("color");</code> <code>Assert.assertEquals(color, "rgba(0, 0, 0, 1)");</code>
Streams	<pre>List<String> originalList = elementsList.stream().map(s-> s.getText()).collect(Collectors.toList()); List<String> sortedList = originalList.stream().sorted().collect(Collectors.toList()); Assert.assertTrue(originalList.equals(sortedList));</pre> <pre>do { List<WebElement> rows = driver.findElements(By.xpath("//tr/td[1]")); price = rows.stream().filter(s -> s.getText().contains("Rice")).map(s-> getPriceVeggie(s)) .collect(Collectors.toList()); if (price.size() < 1) { driver.findElement(By.cssSelector("[aria-label='Next']")).click(); } } while (price.size() < 1);</pre>
Browser Instantiation	We cannot instantiate the WebDriver interface directly as it is not a class. Browser-Specific Driver Instances: Example: <code>FirefoxDriver driver = new FirefoxDriver();</code> This creates an object specific to the Firefox browser and allows invocation of only methods supported by <code>FirefoxDriver</code>. Scripts using this setup will run only on Firefox. For other browsers, individual driver objects must be created, e.g., <code>ChromeDriver driver = new ChromeDriver();</code> or <code>InternetExplorerDriver driver = new InternetExplorerDriver();</code>. Need for Cross-Browser Testing: Using browser-specific objects limits flexibility. Cross-browser compatibility requires dynamic object referencing. Dynamic Polymorphism: Using <code>WebDriver driver = new FirefoxDriver();</code> allows flexibility. Here, <code>driver</code> is a reference of <code>WebDriver</code>, and the object is <code>FirefoxDriver</code>. This setup enables calling methods defined in the <code>WebDriver</code> interface, regardless of the specific browser driver being used. Other browser drivers like <code>ChromeDriver</code>, <code>InternetExplorerDriver</code>, etc., also implement the <code>WebDriver</code> interface via <code>RemoteWebDriver</code>. Important Note: Browser-specific methods cannot be invoked across different drivers. For example, a method specific to <code>ChromeDriver</code> cannot be called on a <code>FirefoxDriver</code> instance. Doing so will lead to script failure.
Do	• Selenium should not be used to prepare a test case. All repetitive actions and preparations for a test case, should be done through other methods. For example, most web UIs have authentication (e.g. a login form). Eliminating logging in via web browser before every test will improve both the speed and stability of the test. • Mock external services • Avoid sharing state: Do not share test data and Create a new <code>WebDriver</code> instance per test
Don't	• Captchas • File downloads • HTTP response codes • Gmail, email and Facebook logins • Test dependency • Performance testing • Link spidering • Two Factor Authentication
Test independency	Write each test as its own unit. Write the tests in a way that will not be reliant on other tests to complete
using a fluent API	<pre>public class GoogleSearchPage extends BasePage { public GoogleSearchPage(WebDriver driver) { super(driver); // Generally do not assert within pages or components. // Effectively throws an exception if the lambda condition is not met. new WebDriverWait(driver, Duration.ofSeconds(3)).until(d -> d.findElement(By.id("logo"))); } public GoogleSearchPage setSearchString(String sstr) { driver.findElement(By.id("gbqfq")).sendKeys(sstr); return this; } }</pre>

	<pre> public void clickSearchButton() { driver.findElement(By.id("gbqfb")).click(); } } </pre>
POM	The public methods represent the services that the page offers Try not to expose the internals of the page Generally don't make assertions Methods return other PageObjects Need not represent an entire page Different results for the same action are modelled as different methods <pre> public class LoginPage { private final WebDriver driver; public LoginPage(WebDriver driver) { this.driver = driver; // Check that we're on the right page. if (!"Login".equals(driver.getTitle())) { // Alternatively, we could navigate to the login page, perhaps logging out first throw new IllegalStateException("This is not the login page"); } } // The login page contains several HTML elements that will be represented as WebElements. // The locators for these elements should only be defined once. By usernameLocator = By.id("username"); By passwordLocator = By.id("passwd"); By loginButtonLocator = By.id("login"); // The login page allows the user to type their username into the username field public LoginPage typeUsername(String username) { // This is the only place that "knows" how to enter a username driver.findElement(usernameLocator).sendKeys(username); // Return the current page object as this action doesn't navigate to a page represented by another PageObject return this; } // The login page allows the user to type their password into the password field public LoginPage typePassword(String password) { // This is the only place that "knows" how to enter a password driver.findElement(passwordLocator).sendKeys(password); // Return the current page object as this action doesn't navigate to a page represented by another PageObject return this; } // The login page allows the user to submit the login form public HomePage submitLogin() { // This is the only place that submits the login form and expects the destination to be the home page. // A separate method should be created for the instance of clicking login whilst expecting a login failure. driver.findElement(loginButtonLocator).submit(); // Return a new page object representing the destination. Should the login page ever // go somewhere else (for example, a legal disclaimer) then changing the method signature // for this method will mean that all tests that rely on this behaviour won't compile. return new HomePage(driver); } // The login page allows the user to submit the login form knowing that an invalid username and / or password were entered public LoginPage submitLoginExpectingFailure() { // This is the only place that submits the login form and expects the destination to be the login page due to login failure. driver.findElement(loginButtonLocator).submit(); // Return a new page object representing the destination. Should the user ever be navigated to the home page after submitting a login with credentials // expected to fail login, the script will fail when it attempts to instantiate the LoginPage PageObject. return new LoginPage(driver); } // Conceptually, the login page offers the user the service of being able to "log into" // the application using a user name and password. public HomePage loginAs(String username, String password) { // The PageObject methods that enter username, password & submit login have already defined and should not be repeated here. typeUsername(username); typePassword(password); return submitLogin(); } } </pre>
Read PDF	<pre> import org.apache.pdfbox.pdmodel.PDDocument; import org.apache.pdfbox.text.PDFTextStripper; import org.testng.annotations.Test; import java.io.File; import java.io.IOException; import java.util.List; public class PDFTest { </pre>


```

private static List<File> getListOfFiles(String directoryName) {
    File directory = new File(directoryName);
    File[] fList = directory.listFiles();
    assert fList != null;
    return List.of(fList);
}

@Test
public void pdfTest() {
    String downloadPath = System.getProperty("user.home") + "/Downloads/";
    List<File> files = getListOfFiles(downloadPath);
    File downloadedFile = null;
    for (File file : files) {
        if (file.getName().endsWith(".pdf")) {
            downloadedFile = file;
            break;
        }
    }
    if (downloadedFile != null) {
        try {
            PDDocument document = PDDocument.load(downloadedFile);
            PDFTextStripper stripper = new PDFTextStripper();
            String pdfText = stripper.getText(document);
            if (pdfText.contains("EPAM")) {
                System.out.println("PDF downloaded and validation passed.");
            } else {
                System.out.println("PDF downloaded but validation failed. Expected text not found.");
            }
            document.close();
        } catch (IOException e) {
            System.err.println("Error reading PDF: " + e.getMessage());
        }
        if (downloadedFile.delete()) {
            System.out.println("Downloaded file deleted.");
        } else {
            System.err.println("Failed to delete downloaded file.");
        }
    } else {
        System.out.println("Downloaded PDF not found.");
    }
}
}

```

Autowiring PageObjectManager

You can automate the creation of boilerplate code for your Page Object Manager using reflection and annotations, similar to how Spring Boot handles dependency injection. Here's how you can do this:

Steps:

1. ****Define a Marker Annotation for Pages****: Create a custom annotation to mark your page classes.
2. ****Scan for Annotated Classes****: Use reflection to find all classes annotated with this marker.
3. ****Instantiate Pages Dynamically****: Dynamically instantiate and cache these classes as needed.

Here's an updated approach to your `PageObjectManager`:

1. Create a Marker Annotation

```

'''java
package annotations;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

```

```

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface Page {
    ...
}

```

2. Annotate Your Page Classes

```

'''java
package pages;

import annotations.Page;
import org.openqa.selenium.WebDriver;

@Page
public class HomePage {
    private WebDriver driver;

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }
}
'''

```

Repeat this for other page classes, e.g., `LoginSignupPage`, `SignUpPage`, etc.

3. Update `PageObjectManager`

```
```java
package pages;

import annotations.Page;
import org.openqa.selenium.WebDriver;

import java.util.HashMap;
import java.util.Map;

public class PageObjectManager {
 private final WebDriver driver;
 private final Map<Class<?>, Object> pageCache = new HashMap<>();

 public PageObjectManager(WebDriver driver) {
 this.driver = driver;
 }

 @SuppressWarnings("unchecked")
 public <T> T getPage(Class<T> pageClass) {
 return (T) pageCache.computeIfAbsent(pageClass, clazz -> {
 if (!clazz.isAnnotationPresent(Page.class)) {
 throw new IllegalArgumentException("Class " + clazz.getName() + " is not annotated with @Page");
 }
 try {
 return clazz.getDeclaredConstructor(WebDriver.class).newInstance(driver);
 } catch (Exception e) {
 throw new RuntimeException("Failed to create instance of " + clazz.getName(), e);
 }
 });
 }
}
```
```

4. Use the `PageObjectManager`

```
```java
PageObjectManager manager = new PageObjectManager(driver);

// Access pages dynamically
HomePage homePage = manager.getPage(HomePage.class);
LoginSignupPage loginSignupPage = manager.getPage(LoginSignupPage.class);
```
```

Benefits:

1. **Dynamic Instantiation**: No need to manually define getter methods for each page.
2. **Scalable**: Adding a new page only requires annotating it with `@Page`.
3. **Cleaner Code**: Reduces boilerplate and centralizes instantiation logic.
4. **Easy to Extend**: You can add additional features like lazy initialization or logging.

This design mimics Spring Boot's autowiring feature, providing a clean, automated approach to manage your page objects.

disable images in selenium

Options.addArguments("profile.managed_default_content_settings.images", 2); // 2 disables images

Environment Variables via Properties File

| | |
|---|--|
| Create global.properties | <pre># Active environment env=qa # QA environment qa.url=https://rahulshettyacademy.com/seleniumPractise qa.browser=edge # UAT environment uat.url=https://rahulshettyacademy.com/seleniumPractise uat.browser=chrome</pre> |
| In Test Base call variables per environment | <pre>String env = System.getProperty("env", "qa"); // Default to 'qa' if not provided Properties prop = new Properties(); String propertiesFile = "src/test/resources/global.properties"; FileInputStream fis = new FileInputStream(propertiesFile); prop.load(fis); String url = prop.getProperty(env+".url"); String browser = prop.getProperty(env+".browser");</pre> |

Good Practice

In page class assume we create 3 method to enter username,password and click submit.in this class itself we can create a method login having all 3 function called.this way our test class will be clean

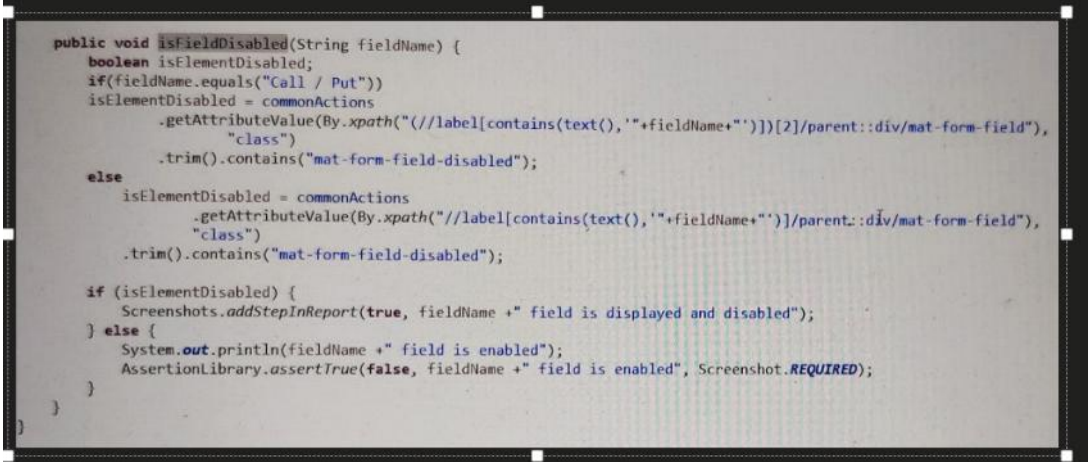
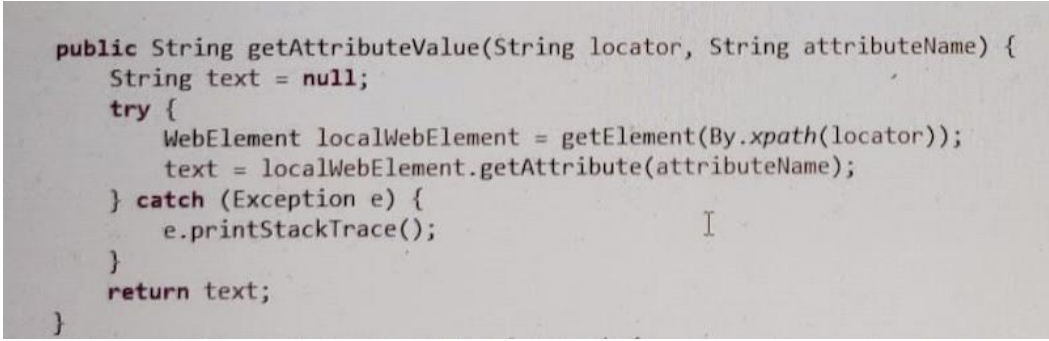
Don't Use PageFactory

At the 2017 SeleniumConf in Austin, **Simon Stewart** (the creator of WebDriver and the lead of the Selenium project) explicitly told the audience **not to use PageFactory**, calling its inclusion in the Selenium library a *"terrible mistake."* While PageFactory was originally intended to be a simple, illustrative example of how a Page Object Model (POM) could be built, the automation community mistook it for the official, gold -standard implementation.

- **Inability to Handle Dynamic Locators**: The @FindBy annotations used in PageFactory require **hardcoded, static locator strings**.
 - **The Problem**: In modern, dynamic web applications, web elements frequently rely on dynamic attributes (e.g., matching a specific row in a dynamic table using an index, or locating an element containing dynamic user text).
 - **The Fallout**: Because annotations cannot be updated at runtime, you cannot pass variables into a @FindBy locator. This forces engineers to abandon

| | <p>PageFactory for dynamic components, leading to a confusing, hybrid framework where some elements use PageFactory and others use standard driver.findElement(By...).</p> <ul style="list-style-type: none">• Extreme Inefficiency (Performance Tax): By default, PageFactory uses Proxies instead of storing actual elements. Every single time you interact with a PageFactory element or check its status, it triggers a completely fresh findElement() command over the network to the browser driver.<ul style="list-style-type: none">• For example, checking if an element is clickable and then clicking it under a standard ExpectedConditions workflow can trigger up to 3 to 6 separate findElement network requests for a single action.• This drastically slows down test execution, especially when running tests at scale on cloud platforms like Sauce Labs or BrowserStack.• The Paradox of @CacheLookup: To fix the inefficiency mentioned above, PageFactory provides a @CacheLookup annotation. It tells Selenium to find the element once and store it in memory. However, this introduces an even worse problem: StaleElementReferenceException.<ul style="list-style-type: none">• Modern web apps heavily rely on frameworks like React, Angular, or Vue, which constantly redraw and refresh the Document Object Model (DOM).• Once an element is cached, if the page updates or rerenders slightly, the cached reference breaks. Because it is cached, Page Factory cannot recover or automatically re-find it, leading to highly brittle, flaky tests.• Poor Handling of Implicit vs. Explicit Waits: PageFactory relies heavily on an internal mechanism for element loading, which often clashes with modern automation best practices. Mixing implicit waits (built into the PageFactory's AjaxElementLocatorFactory) with explicit waits (WebDriverWait) causes unpredictable test execution times and bizarre timeout behaviors that are incredibly hard to debug. | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|--------------------------------------|---|-----------|----------------|----------|---------|------------------|-----|-----|-----|------------------------|-----|-----|-----|----------------------|-----|-----|-----|-------------------------------------|----|-----|-----|--------------------------------------|----|-----|-----|-----------|-----|----|----|------------|-----|----|----|--------------|-----|-----|----|-----------|---------|---------|------------|------------------------|-----|-----|---------------|----------------------|-----|-----|----------------|------------------------|---|-----------------------|--|
| Static Variables | <p>Reading operation like prop file can be static as they don't modify record and also are memory efficient in terms of creating new obj per thread.</p> <p>Static variable are shared across all threads</p> | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Static and Non Static Components | <table><tr><th>Component</th><th>Sequential</th><th>Parallel</th><th>Static?</th></tr><tr><td>Constants</td><td>Yes</td><td>Yes</td><td>YES</td></tr><tr><td>Utility methods</td><td>Yes</td><td>Yes</td><td>YES</td></tr><tr><td>Config values</td><td>Yes</td><td>Yes</td><td>YES</td></tr><tr><td>ThreadLocal<WebDriver></td><td>No</td><td>Yes</td><td>YES</td></tr><tr><td>ThreadLocal<ExtentTest></td><td>No</td><td>Yes</td><td>YES</td></tr><tr><td>WebDriver</td><td>Yes</td><td>No</td><td>NO</td></tr><tr><td>ExtentTest</td><td>Yes</td><td>No</td><td>NO</td></tr><tr><td>Page Objects</td><td>Yes</td><td>Yes</td><td>NO</td></tr><tr><td>Test Data</td><td>Depends</td><td>Depends</td><td>Usually NO</td></tr><tr><td><u>Counters</u></td><td>Yes</td><td>Yes</td><td>Atomic static</td></tr><tr><td><u>Logger</u></td><td>Yes</td><td>Yes</td><td>Usually static</td></tr></table>
<table><tr><td>Good Static Candidates</td><td><pre>private static final String URL; private static Properties prop; private static ThreadLocal<WebDriver> driver; private static ExtentReports extent; private static AtomicInteger counter; configuration constants utility methods thread local containers extent report object counters Use static when the data global for the application and non static when it's unique for each test execution?"</pre></td></tr><tr><td>Bad Static Candidates</td><td><pre>private static WebDriver driver; private static WebElement username; private static LoginPage loginPage; private static TestData data; webdriver webelement page objects test data api response request payload These objects represent execution state. Execution state should almost never be static.</pre></td></tr></table> | Component | Sequential | Parallel | Static? | Constants | Yes | Yes | YES | Utility methods | Yes | Yes | YES | Config values | Yes | Yes | YES | ThreadLocal<WebDriver> | No | Yes | YES | ThreadLocal<ExtentTest> | No | Yes | YES | WebDriver | Yes | No | NO | ExtentTest | Yes | No | NO | Page Objects | Yes | Yes | NO | Test Data | Depends | Depends | Usually NO | <u>Counters</u> | Yes | Yes | Atomic static | <u>Logger</u> | Yes | Yes | Usually static | Good Static Candidates | <pre>private static final String URL; private static Properties prop; private static ThreadLocal<WebDriver> driver; private static ExtentReports extent; private static AtomicInteger counter; configuration constants utility methods thread local containers extent report object counters Use static when the data global for the application and non static when it's unique for each test execution?"</pre> | Bad Static Candidates | <pre>private static WebDriver driver; private static WebElement username; private static LoginPage loginPage; private static TestData data; webdriver webelement page objects test data api response request payload These objects represent execution state. Execution state should almost never be static.</pre> |
| Component | Sequential | Parallel | Static? | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Constants | Yes | Yes | YES | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Utility methods | Yes | Yes | YES | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Config values | Yes | Yes | YES | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ThreadLocal<WebDriver> | No | Yes | YES | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ThreadLocal<ExtentTest> | No | Yes | YES | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| WebDriver | Yes | No | NO | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ExtentTest | Yes | No | NO | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Page Objects | Yes | Yes | NO | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Test Data | Depends | Depends | Usually NO | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| <u>Counters</u> | Yes | Yes | Atomic static | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| <u>Logger</u> | Yes | Yes | Usually static | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Good Static Candidates | <pre>private static final String URL; private static Properties prop; private static ThreadLocal<WebDriver> driver; private static ExtentReports extent; private static AtomicInteger counter; configuration constants utility methods thread local containers extent report object counters Use static when the data global for the application and non static when it's unique for each test execution?"</pre> | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Bad Static Candidates | <pre>private static WebDriver driver; private static WebElement username; private static LoginPage loginPage; private static TestData data; webdriver webelement page objects test data api response request payload These objects represent execution state. Execution state should almost never be static.</pre> | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

| | |
|--|--|
| NETX360 | <pre> public boolean waitForTheElementToAppear(String locator, int time, WebDriver driver) throws InterruptedException { boolean isElementFound = false; for (int i = 0; i < time; i++) { if (isElementPresent(locator, driver)) { isElementFound = true; break; } Thread.sleep(1000); } return isElementFound; } public boolean isElementPresent(String locator, WebDriver driver) { boolean isElementFound = false; try { waitForTheElementToAppear(locator, 10, driver); if (driver.findElements(By.xpath(locator)).size() > 0) { isElementFound = true; } } catch (Exception e) { e.printStackTrace(); } return isElementFound; } public boolean isElementDisplayed(String locator, WebDriver driver) { boolean isElementFound = false; try { waitForTheElementToAppear(locator, 10, driver); if (driver.findElements(By.xpath(locator)).size() > 0) { isElementFound = driver.findElement(By.xpath(locator)).isDisplayed(); } } catch (Exception e) { e.printStackTrace(); } return isElementFound; } </pre> |
| isElementPresent | <pre> public boolean isElementPresent(String locator, WebDriver driver) { return !driver.findElements(By.xpath(locator)).isEmpty(); } ----- public boolean isElementPresent(By locator) { try { WebDriverWait wait = new WebDriverWait(driver, Duration.ofMillis(1500)); // Wait for 1.5s wait.until(ExpectedConditions.presenceOfElementLocated(locator)); return true; } catch (TimeoutException e) { return false; // Element not found within 1.5s } } </pre> |
| Identifying Xpath of disappearing elements | <ul style="list-style-type: none"> • Inspect the element which will display the disappearing elements • Right click on the element tag and select 'Break on' and select 'attribute modifications'. Once you selected them, you will be able to see it under DOM Breakpoints section. • Now click on the element which will display the disappearing elements. Debugger mode will open. • Now press F8, until you are able to inspect the disappearing element. <p>-----</p> <ul style="list-style-type: none"> • Open Dev tools, click anywhere inside elements tab in dev tools • Ctrl+Shift+P • In run command type <u>emulate a focussed page</u> • Now elements wont disappear |

| | |
|--|---|
| Xpath when we want to iterate by updating numbers in xpath | <pre>String ele = "/div[@row-index='%s']"; public String getElementText(String ele, int num) { WebElement element = driver.findElement(By.xpath(String.format(ele, String.valueOf(num)))); String actualtext = element.getText(); return actualtext; }</pre> |
| Validate is field disabled |  <pre>public void isFieldDisabled(String fieldName) { boolean isElementDisabled; if(fieldName.equals("Call / Put")) isElementDisabled = commonActions .getAttributeValue(By.xpath("//label[contains(text(),'"+fieldName+"')][2]/parent::div/mat-form-field", "class") .trim().contains("mat-form-field-disabled"); else isElementDisabled = commonActions .getAttributeValue(By.xpath("//label[contains(text(),'"+fieldName+"')]/parent::div/mat-form-field", "class") .trim().contains("mat-form-field-disabled"); if (isElementDisabled) { Screenshots.addStepInReport(true, fieldName + " field is displayed and disabled"); } else { System.out.println(fieldName + " field is enabled"); AssertionLibrary.assertTrue(false, fieldName + " field is enabled", Screenshot.REQUIRED); } }</pre> |
| Get Attribute Value |  <pre>public String getAttributeValue(String locator, String attributeName) { String text = null; try { WebElement localWebElement = getElement(By.xpath(locator)); text = localWebElement.getAttribute(attributeName); } catch (Exception e) { e.printStackTrace(); } return text; }</pre> |

Design Patterns for High-Quality Automated Tests

26 March 2026 11:05

| | | | | | | |
|-----------|--|--|---------|--|-----|--|
| Chapter 1 | Automated tests are software too and should be treated with the same "Clean Code" standards as production code. | | | | | |
| | Key Principles for Maintenance | <ul style="list-style-type: none">• Don't Repeat Yourself (DRY): Abstract repetitive actions (like logging in or filling forms) into reusable methods.• Readability over Complexity: A test should read like a story, clearly stating the Arrange, Act, and Assert phases.• Separation of Concerns: Keep the "What" (the test logic) separate from the "How" (the technical implementation of clicking buttons or finding elements). | | | | |
| | SRP | <p>"Every software module should have only one reason to change" which means that every class, method, etc. can do many things, but they should serve a single purpose.</p> <table><tr><td>Non SRP</td><td><p>LoginPage class is doing too much: managing UI elements, performing the login logic, and handling the reporting/logging. If the logging library changes, you have to modify your Page Object. If the UI changes, you modify the same class.</p><pre>public class LoginPage { private WebDriver driver; public LoginPage(WebDriver driver) { this.driver = driver; } // Responsibility 1: UI Interaction public void enterCredentials(String user, String pass) { driver.findElement(By.id("user")).sendKeys(user); driver.findElement(By.id("pass")).sendKeys(pass); } // Responsibility 2: Logging/Reporting (Wrong!) public void logToExtentReport(String message) { System.out.println("ExtentReport: " + message); // Logic for ExtentReports library goes here... } // Responsibility 3: Database Verification (Wrong!) public boolean checkUserInDb(String user) { // JDBC code to check if user exists... return true; } }</pre></td></tr><tr><td>SRP</td><td><p>1. The Page Object (UI Only): Its only job is to know about the elements on the screen.</p><pre>public class LoginPage { private WebDriver driver; private By userField = By.id("user"); public void login(String user) { driver.findElement(userField).sendKeys(user); } }</pre><p>2. The Logger (Reporting Only): Its only job is to format and send logs to your reporting tool (Allure, Extent, etc.).</p><pre>public class TestLogger { public static void logStep(String message) { // High-level logging logic } }</pre><p>3. The DB Utility (Data Only): Its only job is to talk to the database.</p><pre>public class UserDatabase { public boolean isUserActive(String username) { // Database query logic return true; } }</pre></td></tr></table> | Non SRP | <p>LoginPage class is doing too much: managing UI elements, performing the login logic, and handling the reporting/logging. If the logging library changes, you have to modify your Page Object. If the UI changes, you modify the same class.</p> <pre>public class LoginPage { private WebDriver driver; public LoginPage(WebDriver driver) { this.driver = driver; } // Responsibility 1: UI Interaction public void enterCredentials(String user, String pass) { driver.findElement(By.id("user")).sendKeys(user); driver.findElement(By.id("pass")).sendKeys(pass); } // Responsibility 2: Logging/Reporting (Wrong!) public void logToExtentReport(String message) { System.out.println("ExtentReport: " + message); // Logic for ExtentReports library goes here... } // Responsibility 3: Database Verification (Wrong!) public boolean checkUserInDb(String user) { // JDBC code to check if user exists... return true; } }</pre> | SRP | <p>1. The Page Object (UI Only): Its only job is to know about the elements on the screen.</p> <pre>public class LoginPage { private WebDriver driver; private By userField = By.id("user"); public void login(String user) { driver.findElement(userField).sendKeys(user); } }</pre> <p>2. The Logger (Reporting Only): Its only job is to format and send logs to your reporting tool (Allure, Extent, etc.).</p> <pre>public class TestLogger { public static void logStep(String message) { // High-level logging logic } }</pre> <p>3. The DB Utility (Data Only): Its only job is to talk to the database.</p> <pre>public class UserDatabase { public boolean isUserActive(String username) { // Database query logic return true; } }</pre> |
| Non SRP | <p>LoginPage class is doing too much: managing UI elements, performing the login logic, and handling the reporting/logging. If the logging library changes, you have to modify your Page Object. If the UI changes, you modify the same class.</p> <pre>public class LoginPage { private WebDriver driver; public LoginPage(WebDriver driver) { this.driver = driver; } // Responsibility 1: UI Interaction public void enterCredentials(String user, String pass) { driver.findElement(By.id("user")).sendKeys(user); driver.findElement(By.id("pass")).sendKeys(pass); } // Responsibility 2: Logging/Reporting (Wrong!) public void logToExtentReport(String message) { System.out.println("ExtentReport: " + message); // Logic for ExtentReports library goes here... } // Responsibility 3: Database Verification (Wrong!) public boolean checkUserInDb(String user) { // JDBC code to check if user exists... return true; } }</pre> | | | | | |
| SRP | <p>1. The Page Object (UI Only): Its only job is to know about the elements on the screen.</p> <pre>public class LoginPage { private WebDriver driver; private By userField = By.id("user"); public void login(String user) { driver.findElement(userField).sendKeys(user); } }</pre> <p>2. The Logger (Reporting Only): Its only job is to format and send logs to your reporting tool (Allure, Extent, etc.).</p> <pre>public class TestLogger { public static void logStep(String message) { // High-level logging logic } }</pre> <p>3. The DB Utility (Data Only): Its only job is to talk to the database.</p> <pre>public class UserDatabase { public boolean isUserActive(String username) { // Database query logic return true; } }</pre> | | | | | |
| | OCP | <ul style="list-style-type: none">• The Principle: "Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification."• The Meaning: If new requirements are written for the already implemented functionality, these can be added in a way so that we don't need to change the whole structure of the existing code that has been already unit tested.• The Risk: If we change it and add it directly, we can trigger new regression problems. <table><tr><td>Non OCP</td><td><p>Every time you want to support a new browser (like Edge or Safari), you have to manually edit the DriverFactory class and add another else-if block. This risks breaking the existing Chrome/Firefox setup.</p><pre>public class DriverFactory {</pre></td></tr></table> | Non OCP | <p>Every time you want to support a new browser (like Edge or Safari), you have to manually edit the DriverFactory class and add another else-if block. This risks breaking the existing Chrome/Firefox setup.</p> <pre>public class DriverFactory {</pre> | | |
| Non OCP | <p>Every time you want to support a new browser (like Edge or Safari), you have to manually edit the DriverFactory class and add another else-if block. This risks breaking the existing Chrome/Firefox setup.</p> <pre>public class DriverFactory {</pre> | | | | | |

| | | | |
|---------|--|---------|---|
| | <pre> public WebDriver getDriver(String browser) { if (browser.equalsIgnoreCase("chrome")) { return new ChromeDriver(); } else if (browser.equalsIgnoreCase("firefox")) { return new FirefoxDriver(); } // To add Edge, I MUST modify this existing, tested class. return new ChromeDriver(); } </pre> | | |
| OCP | <p>we hide the implementation behind an interface. The main engine stays "Closed," while the ability to add browsers remains "Open."</p> <p>1. The Abstraction (The "Contract"): We create an interface so the factory doesn't care <i>how</i> a browser is started.</p> <pre> public interface BrowserConfiguration { WebDriver createDriver(); } </pre> <p>2. The Implementation (Extension): When a new browser is needed, you just create a new class. You don't touch the others.</p> <pre> public class ChromeConfig implements BrowserConfiguration { public WebDriver createDriver() { return new ChromeDriver(); } } public class FirefoxConfig implements BrowserConfiguration { public WebDriver createDriver() { return new FirefoxDriver(); } } </pre> <p>3. The "Closed" Manager: Now, you can use a Map to store these. When you need to add "Edge" next week, you add a new class and register it. You never change the logic of the getDriver method itself.</p> <pre> public class DriverManager { private static final Map<String, BrowserConfiguration> BROWSER_MAP = new HashMap<>(); static { BROWSER_MAP.put("chrome", new ChromeConfig()); BROWSER_MAP.put("firefox", new FirefoxConfig()); } public static WebDriver getDriver(String type) { return BROWSER_MAP.get(type.toLowerCase()).createDriver(); } } </pre> | | |
| LSP | <p>Objects of a superclass should be replaceable with objects of its subclasses without breaking the application.</p> <p>In simpler terms: A child class should be able to do everything the parent class can do, without changing the expected behavior.</p> <table border="1"> <tr> <td>Non-LSP</td><td> <p>Imagine we have a base class for all test browsers.</p> <pre> public abstract class WebDriverWrapper { public abstract void navigateTo(String url); public abstract void manageCookies(); } public class ChromeBrowser extends WebDriverWrapper { @Override public void navigateTo(String url) { /* Open URL */ } @Override public void manageCookies() { /* Logic to delete/add cookies */ } } public class HtmlUnitBrowser extends WebDriverWrapper { @Override public void navigateTo(String url) { /* Open URL */ } @Override public void manageCookies() { // ERROR: This "headless" browser doesn't support cookies! throw new UnsupportedOperationException("Cookies not supported in Headless"); } } </pre> </td></tr> </table> | Non-LSP | <p>Imagine we have a base class for all test browsers.</p> <pre> public abstract class WebDriverWrapper { public abstract void navigateTo(String url); public abstract void manageCookies(); } public class ChromeBrowser extends WebDriverWrapper { @Override public void navigateTo(String url) { /* Open URL */ } @Override public void manageCookies() { /* Logic to delete/add cookies */ } } public class HtmlUnitBrowser extends WebDriverWrapper { @Override public void navigateTo(String url) { /* Open URL */ } @Override public void manageCookies() { // ERROR: This "headless" browser doesn't support cookies! throw new UnsupportedOperationException("Cookies not supported in Headless"); } } </pre> |
| Non-LSP | <p>Imagine we have a base class for all test browsers.</p> <pre> public abstract class WebDriverWrapper { public abstract void navigateTo(String url); public abstract void manageCookies(); } public class ChromeBrowser extends WebDriverWrapper { @Override public void navigateTo(String url) { /* Open URL */ } @Override public void manageCookies() { /* Logic to delete/add cookies */ } } public class HtmlUnitBrowser extends WebDriverWrapper { @Override public void navigateTo(String url) { /* Open URL */ } @Override public void manageCookies() { // ERROR: This "headless" browser doesn't support cookies! throw new UnsupportedOperationException("Cookies not supported in Headless"); } } </pre> | | |

| | |
|-----|--|
| | <div>LSP</div> <p>We should only put behaviors in the parent class that all children can actually perform. If not, we move that behavior to a more specific sub-group or interface.</p> <pre> public interface Navigatable { void navigateTo(String url); } public interface CookieManageable { void manageCookies(); } // Chrome supports both public class ChromeBrowser implements Navigatable, CookieManageable { public void navigateTo(String url) { ... } public void manageCookies() { ... } } // HtmlUnit only implements what it can actually do public class HtmlUnitBrowser implements Navigatable { public void navigateTo(String url) { ... } } </pre> |
| ISP | <p>The Interface Segregation Principle (ISP) states that a client should never be forced to depend on methods it does not use. In test automation, we often create "Fat Interfaces" (interfaces with too many methods), which forces our Page Objects or Utility classes to implement logic they don't actually need.</p> <div> <div>Non-ISP</div> <p>Imagine a single interface for all element interactions. If you have a ReadOnlyLabel, it is forced to implement type() and click(), even though a label can't do those things.</p> <pre> public interface WebElementActions { void click(); void type(String text); void clear(); String getText(); } public class ReadOnlyLabel implements WebElementActions { public String getText() { return "Order Confirmed"; } // Forced to implement these! public void click() { /* Do nothing */ } public void type(String text) { throw new UnsupportedOperationException(); } public void clear() { throw new UnsupportedOperationException(); } } </pre> </div> <div> <div>ISP</div> <p>We break the "Fat Interface" into smaller, specific ones. This allows classes to "pick and choose" only the behaviors they actually support.</p> <pre> public interface Clickable { void click(); } public interface Typeable { void type(String text); } public interface Readable { String getText(); } </pre> <p>Now, a Button implements what it needs, and a Label stays clean.</p> <pre> public class CustomButton implements Clickable, Readable { public void click() { System.out.println("Button clicked"); } public String getText() { return "Submit"; } } public class ReadOnlyLabel implements Readable { public String getText() { return "Order Confirmed"; } } </pre> </div> |
| DIP | <p>It states that high-level modules should not depend on low-level modules; both should depend on abstractions.</p> <p>In test automation, this is the secret to "Environment Agnostic" code. Your test scripts shouldn't care if they are running on your local Chrome, a Docker container, or a cloud provider like BrowserStack.</p> <div> <div>Non-DIP</div> <p>the BaseTest is "tightly coupled" to a specific implementation (LocalChromeDriver). If you want to run your tests on a Remote Selenium Grid, you have to modify the BaseTest class.</p> <pre> public class BaseTest { // High-level module depends directly on a low-level implementation private LocalChromeDriver driver = new LocalChromeDriver(); public void setup() { driver.start(); } } </pre> </div> |

| | | |
|--|----------------------------|--|
| | | <pre>} }</pre> <p>The Problem: The high-level BaseTest is a slave to the LocalChromeDriver. You cannot switch to RemoteDriver without changing the source code of BaseTest.</p> |
| | DIP | <p>We introduce an interface (WebDriverManager). Now, BaseTest depends on the Interface, and the specific drivers also depend on that same Interface.</p> <p>1. The Abstraction (The Bridge)</p> <pre>public interface WebDriverManager { WebDriver getDriver(); }</pre> <p>2. Low-Level Implementations</p> <pre>public class LocalManager implements WebDriverManager { public WebDriver getDriver() { return new ChromeDriver(); } } public class CloudManager implements WebDriverManager { public WebDriver getDriver() { // Logic for SauceLabs/BrowserStack RemoteWebDriver return new RemoteWebDriver(url, capabilities); } }</pre> <p>3. High-Level Module (Decoupled)</p> <p>BaseTest now accepts <i>any</i> manager. You "inject" the one you want at runtime (via a config file or command line).</p> <pre>public class BaseTest { private WebDriverManager driverManager; // The test doesn't know (or care) which manager it gets public BaseTest(WebDriverManager manager) { this.driverManager = manager; } public void startTest() { WebDriver driver = driverManager.getDriver(); } }</pre> |
| Modular Approach(Scripting to Framework) | Low Quality | <pre>@Test public void loginTest() { WebDriver driver = new ChromeDriver(); driver.get("https://example.com"); // Hardcoded selectors are hard to maintain driver.findElement(By.id("user_123")).sendKeys("admin"); driver.findElement(By.id("pass_456")).sendKeys("password"); driver.findElement(By.xpath("//button[@type='submit']")).click(); String message = driver.findElement(By.id("welcome")).getText(); Assert.assertEquals(message, "Welcome!"); }</pre> |
| | Clean Code | <pre>// Logic moved to a reusable method or Page Object public void login(String username, String password) { type(userNameField, username); type(passwordField, password); click(submitButton); } @Test public void cleanLoginTest() { // The test focuses on the business flow, not the technical details loginPage.navigate(); loginPage.login("admin", "password"); Assert.assertTrue(loginPage.isUserLoggedIn()); }</pre> |
| Chapter 2 | IMPLICIT VS EXPLICIT WAITS | <p>Implicit Wait: A global timeout applied to all findElement calls.</p> <pre>driver.manage().timeouts().implicitlyWait(30, TimeUnit.SECONDS);</pre> <p>Problem: If some elements need longer waits, you have to increase the global timeout which affects all tests. Mixing implicit and explicit waits causes undefined, unpredictable behavior.</p> <p>When we mix implicit and explicit waits, we've strayed into 'undefined behavior'... So don't do it. Don't mix implicit and explicit waits.</p> |

| | |
|--------------------------|---|
| | <p>Reason: Implicit waits are implemented on the remote/driver side (chromedriver.exe, etc.). Explicit waits are implemented in the local language bindings. When combined, both sides trigger waits simultaneously leading to inconsistent timing behavior.</p> <p>Explicit Wait: Waits for a specific condition before proceeding. Implemented via <code>WebDriverWait</code> + <code>ExpectedConditions</code>.</p> <p>WHEN TO USE WHICH:</p> <ul style="list-style-type: none"> - Use explicit waits for asynchronous operations, dynamic content, AJAX - Never mix implicit and explicit waits in the same test suite - Prefer explicit waits for fine-grained, reliable control <p>Implementing Explicit Wait</p> <pre>// Wait for element to be present in DOM private WebElement waitAndFindElement(By by) { var webDriverWait = new WebDriverWait(driver, 30); return webDriverWait.until(ExpectedConditions.presenceOfElementLocated(by)); } // Wait for list of elements to be present in DOM private List<WebElement> waitAndFindElements(By by) { var webDriverWait = new WebDriverWait(driver, 30); return webDriverWait.until(ExpectedConditions.presenceOfAllElementsLocatedBy(by)); } // Wait for element to be clickable before interacting private void waitToBeClickable(By by) { var webDriverWait = new WebDriverWait(driver, 30); webDriverWait.until(ExpectedConditions.elementToBeClickable(by)); }</pre> <p>NOTE: The method name <code>waitAndFindElement</code> suggests two operations, which breaks Single Responsibility Principle. It gets renamed and relocated in later chapters.</p> <p>Usage after applying explicit waits:</p> <pre>var billingFirstName = waitAndFindElement(By.id("billing_first_name")); billingFirstName.sendKeys("Anton"); // Wait before clicking an element that may not be ready after AJAX waitToBeClickable(By.cssSelector("[value*='Update cart']")); var updateCart = waitAndFindElement(By.cssSelector("[value*='Update cart']")); updateCart.click();</pre> |
| Dry Principle | <p>Reduce long-term maintenance cost by eliminating unnecessary code duplication. Treat test code as production code - the same software engineering principles apply.</p> <p>Refactored reusable private methods:</p> <pre>private void login(String userName) { var userNameTextField = waitAndFindElement(By.id("username")); userNameTextField.sendKeys(userName); var passwordField = waitAndFindElement(By.id("password")); passwordField.sendKeys(getUserPasswordFromDb(userName)); var loginButton = waitAndFindElement(By.xpath("//button[@name='login']")); loginButton.click(); }</pre> |
| Decorator Design Pattern | <p>Attaches additional responsibilities to an object dynamically. Provides a flexible alternative to subclassing for extending functionality.</p> <p>KEY CHARACTERISTICS:</p> <ul style="list-style-type: none"> - You can wrap a component with any number of decorators (unlimited layering) - You can change behavior before and/or after the component's method is called - The decorator mirrors the type of the component it decorates (same interface) - Alternative to subclassing - uses composition instead of inheritance <p>WHEN TO USE:</p> <ul style="list-style-type: none"> - When you want to add behavior to individual objects without affecting others - When subclassing leads to an explosion of subclasses for every combination - When you need to add responsibilities at runtime - When adhering to Open/Closed Principle (open for extension, closed for modification) |

UML PARTICIPANTS:

- **Component:** Defines the interface for objects that can have responsibilities added
- **ConcreteComponent:** The object that is going to be enhanced dynamically. It inherits the Component.
- **Decorator:** Implements the same interface (abstract class) as the component they are going to decorate. It has a HAS-A relationship with the object that it is extending, which means that the Component has an instance variable that holds a reference to the latter.
- **ConcreteDecorator:** Extends Decorator. Adds new behavior before/after existing methods.

NOTE ON COMPOSITION OVER INHERITANCE: In Java, multiple class inheritance is not allowed. Composition (holding instances of other classes) is preferred over inheritance for achieving polymorphic behavior and code reuse.

Decorator Implementation

Component

```
public interface Actions {
    void click(By locator);
    void sendText(By locator, String text);
    String getText(By locator);
    void selectByVisibleText(By locator, String text);
    void selectByIndex(By locator, int index);
    void selectByValue(By locator, String value);
    String getFirstSelectedOption(By locator);
    void assertEquals(Object actual, Object expected, String
message);
}
```

Concrete Component

```
public class BaseActions implements Actions {
    protected WebDriver driver;
    protected WebDriverWait wait;
    public BaseActions(WebDriver driver, WebDriverWait wait) {
        this.driver = driver;
        this.wait = wait;
    }
    @Override
    public void click(By locator) {
        wait.until(ExpectedConditions.elementToBeClickable(locator)).
click();
    }
    @Override
    public void sendText(By locator, String text) {
        WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        element.clear();
        element.sendKeys(text);
    }
    @Override
    public String getText(By locator) {
        return
wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).ge
tText();
    }
    @Override
    public void selectByVisibleText(By locator, String text) {
        WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        Select select = new Select(element);
        select.selectByVisibleText(text);
    }
    @Override
    public void selectByIndex(By locator, int index) {
        WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        Select select = new Select(element);
        select.selectByIndex(index);
    }
    @Override
    public void selectByValue(By locator, String value) {
        WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        Select select = new Select(element);
        select.selectByValue(value);
    }
    @Override
    public String getFirstSelectedOption(By locator) {
        WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        Select select = new Select(element);
        return select.getFirstSelectedOption().getText();
    }
    @Override
    public void assertEquals(Object actual, Object expected, String
message) {
```

| | |
|--------------------|--|
| | <pre> Assert.assertEquals(actual, expected, message); } } </pre> |
| Decorator | <p>While UML often suggests an abstract class here, in modern Java, we often use the HAS-A relationship directly in the concrete decorators.</p> <pre> private final Actions decorated; // This is the HAS-A relationship </pre> |
| Concrete Decorator | <pre> public class HighlightDecorator implements Actions { private final Actions decorated; private final WebDriver driver; public HighlightDecorator(Actions decorated, WebDriver driver) { this.decorated = decorated; this.driver = driver; } private void highlight(By locator) { try { WebElement element = driver.findElement(locator); JavascriptExecutor js = (JavascriptExecutor) driver; // Apply a yellow background and red border js.executeScript("arguments[0].setAttribute('style', 'background: yellow; border: 2px solid red;');", element); Thread.sleep(200); // Short pause to make the highlight visible // Reset the style (optional, or leave it to see what was touched) js.executeScript("arguments[0].setAttribute('style', 'border: 2px solid white;');", element); } catch (Exception e) { // Ignore highlighting errors to not break the actual test } } @Override public void click(By locator) { highlight(locator); decorated.click(locator); } @Override public void sendText(By locator, String text) { highlight(locator); decorated.sendText(locator, text); } @Override public String getText(By locator) { highlight(locator); return decorated.getText(locator); } @Override public void selectByVisibleText(By locator, String text) { highlight(locator); decorated.selectByVisibleText(locator, text); } @Override public void selectByIndex(By locator, int index) { highlight(locator); decorated.selectByIndex(locator, index); } @Override public void selectByValue(By locator, String value) { highlight(locator); decorated.selectByValue(locator, value); } @Override public String getFirstSelectedOption(By locator) { highlight(locator); return decorated.getFirstSelectedOption(locator); } @Override public void assertEquals(Object actual, Object expected, String message) { // Assertions don't usually interact with elements directly via By locator here decorated.assertEquals(actual, expected, message); } } public class ReportingDecorator implements Actions { private final Actions decorated; private final ExtentTest test; </pre> |

```

        private final WebDriver driver;
        public ReportingDecorator(Actions decorated, ExtentTest test,
        WebDriver driver) {
            this.decorated = decorated;
            this.test = test;
            this.driver = driver;
        }
        private String captureScreenshot() {
            return ((TakesScreenshot)
driver).getScreenshotAs(OutputType.BASE64);
        }
        @Override
        public void click(By locator) {
            test.log(Status.INFO, "Attempting to click on element: " +
locator.toString());
            try {
                decorated.click(locator);
                test.log(Status.INFO, "Clicked on element: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
            } catch (Exception e) {
                test.log(Status.FAIL, "Failed to click on element: " +
locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
                throw e;
            }
        }
        @Override
        public void sendText(By locator, String text) {
            test.log(Status.INFO, "Attempting to send text '" + text + "'
to element: " + locator.toString());
            try {
                decorated.sendText(locator, text);
                test.log(Status.INFO, "Sent text '" + text + "' to
element: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
            } catch (Exception e) {
                test.log(Status.FAIL, "Failed to send text to element: "
+ locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
                throw e;
            }
        }
        @Override
        public String getText(By locator) {
            test.log(Status.INFO, "Attempting to get text from element: "
+ locator.toString());
            try {
                String text = decorated.getText(locator);
                test.log(Status.INFO, "Retrieved text '" + text + "' from
element: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
                return text;
            } catch (Exception e) {
                test.log(Status.FAIL, "Failed to get text from element: "
+ locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
                throw e;
            }
        }
        @Override
        public void selectByVisibleText(By locator, String text) {
            test.log(Status.INFO, "Attempting to select by visible text
'" + text + "' from dropdown: " + locator.toString());
            try {
                decorated.selectByVisibleText(locator, text);
                test.log(Status.INFO, "Selected by visible text '" + text
+ "' from dropdown: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
            } catch (Exception e) {
                test.log(Status.FAIL, "Failed to select by visible text
from dropdown: " + locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
            }
        }
    }

```

```

        throw e;
    }
}
@Override
public void selectByIndex(By locator, int index) {
    test.log(Status.INFO, "Attempting to select by index '" +
index + "' from dropdown: " + locator.toString());
    try {
        decorated.selectByIndex(locator, index);
        test.log(Status.INFO, "Selected by index '" + index + "'
from dropdown: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
    } catch (Exception e) {
        test.log(Status.FAIL, "Failed to select by index from
dropdown: " + locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
        throw e;
    }
}
@Override
public void selectByValue(By locator, String value) {
    test.log(Status.INFO, "Attempting to select by value '" +
value + "' from dropdown: " + locator.toString());
    try {
        decorated.selectByValue(locator, value);
        test.log(Status.INFO, "Selected by value '" + value + "'
from dropdown: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
    } catch (Exception e) {
        test.log(Status.FAIL, "Failed to select by value from
dropdown: " + locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
        throw e;
    }
}
@Override
public String getFirstSelectedOption(By locator) {
    test.log(Status.INFO, "Attempting to get first selected
option from dropdown: " + locator.toString());
    try {
        String selectedOption =
decorated.getFirstSelectedOption(locator);
        test.log(Status.INFO, "Retrieved first selected option '"
+ selectedOption + "' from dropdown: " + locator,
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
        return selectedOption;
    } catch (Exception e) {
        test.log(Status.FAIL, "Failed to get first selected
option from dropdown: " + locator + ". Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
        throw e;
    }
}
@Override
public void assertEquals(Object actual, Object expected, String
message) {
    try {
        decorated.assertEquals(actual, expected, message);
        test.log(Status.PASS, message + " - Expected: [" +
expected + "]" Actual: [" + actual + "]");
    } catch (AssertionError e) {
        test.log(Status.FAIL, message + " - Expected: [" +
expected + "]" Actual: [" + actual + "] - Error: " + e.getMessage(),
MediaEntityBuilder.createScreenCaptureFromBase64S
tring(captureScreenshot()).build());
        throw e;
    }
}
}
}

```

**CommonAct
ions**

```

public class CommonActions implements Actions {
    private final Actions actions;
    public CommonActions(WebDriver driver, WebDriverWait wait,
ExtentTest test) {
        // Base Selenium logic
    }
}

```

| | | | |
|-----------|-------------------------|---|---|
| | | | <pre> Actions baseActions = new BaseActions(driver, wait); // Add highlighting layer on top of base Actions highlightingActions = new HighlightDecorator(baseActions, driver); // Add reporting layer on top of highlighting (or base) this.actions = new ReportingDecorator(highlightingActions, test, driver); } @Override public void click(By locator) { actions.click(locator); } @Override public void sendText(By locator, String text) { actions.sendText(locator, text); } @Override public String getText(By locator) { return actions.getText(locator); } @Override public void selectByVisibleText(By locator, String text) { actions.selectByVisibleText(locator, text); } @Override public void selectByIndex(By locator, int index) { actions.selectByIndex(locator, index); } @Override public void selectByValue(By locator, String value) { actions.selectByValue(locator, value); } @Override public String getFirstSelectedOption(By locator) { return actions.getFirstSelectedOption(locator); } @Override public void assertEquals(Object actual, Object expected, String message) { actions.assertEquals(actual, expected, message); } } </pre> |
| | | Summary | <p>In the Decorator pattern, every decorator delegates to the next object in the chain.</p> <p>Eventually, that delegation must stop and an actual operation must happen. BaseActions is where that delegation stops:</p> <ul style="list-style-type: none"> • ReportingDecorator says: "I'll log this, then someone else click." • HighlightDecorator says: "I'll highlight this, then someone else click." • BaseActions says: "I will actually click." <p>The primary utility is to be the ConcreteComponent that provides the base implementation. It allows the decorators to focus purely on adding extra value (logging, highlighting, retrying, etc.) without worrying about the underlying Selenium mechanics.</p> <p>The Chain Flow: Test → ReportingDecorator (Log) → HighlightDecorator (Color) → BaseActions (Selenium Click) → Browser.</p> <p>When you call actions().click(locator) in your LocatorTest.java:</p> <ol style="list-style-type: none"> 1. ReportingDecorator (Is-A Actions) receives the call. It logs the start. 2. It calls its internal decorated.click() (the Has-A link). 3. That "decorated" object is actually the HighlightDecorator. It highlights the element. 4. The HighlightDecorator then calls its internal decorated.click(). 5. That "decorated" object is finally BaseActions, which performs the actual Selenium driver.findElement().click(). <p>By using this approach, you've avoided creating a giant, messy BaseActions class that tries to do everything (clicking, logging, highlighting, and screenshotting) in one place. Instead, you have small, clean, "world-class" classes that each do exactly one thing.</p> |
| Chapter 3 | Optimise Authentication | 99% of tests need to be logged in but are NOT testing the login itself. Authenticate by generating the authentication cookie via an internal web API call, bypassing the login UI entirely. | |
| | Wait for page to load | readyState | <pre> @Override public void waitUntilPageIsReady() { </pre> |

| | |
|--|---|
| | <pre> var js = (JavascriptExecutor) webDriver; // 1. Wait for static assets (Your current code) webDriverWait.until(d -> js.executeScript("return document.readyState").equals("complete")); // 2. Wait for the Framework to mount (React/Angular usually have a root ID) webDriverWait.until(ExpectedConditions.presenceOfElementLocated (By.id("app-root"))); } </pre> |
| Spinner Disappear | <pre> /** * Waits for the loading overlay to disappear. This is the most * reliable way * to handle React/Angular transitions. */ public void waitForLoadingToFinish() { By loaderLocator = By.cssSelector(".spinner, .loading-overlay, #app-loader"); // 1. Brief wait to see if the spinner even appears (sometimes it's too fast) try { new WebDriverWait(webDriver, Duration.ofMillis(500)) .until(ExpectedConditions.visibilityOfElementLocate d(loaderLocator)); } catch (TimeoutException e) { // Spinner never showed up, which is fine! } // 2. Wait for the spinner to be GONE from the DOM or hidden webDriverWait.until(ExpectedConditions.invisibilityOfElementLoc ated(loaderLocator)); } </pre> |
| | <p>If you want to move away from "flaky" tests entirely, follow this hierarchy of waiting:</p> <ol style="list-style-type: none"> 1. Primary: Wait for a specific element to be clickable or visible (e.g., ExpectedConditions.elementToBeClickable). 2. Secondary: Wait for the Loading Spinner to disappear. 3. Tertiary (Last Resort): document.readyState (Only use this for the very first driver.get() call). 4. Never: Thread.sleep(). <p>Stale-Proof" Fluent Wait Configuration</p> <pre> public WebElement waitForElementFluent(By locator) { FluentWait<WebDriver> wait = new FluentWait<>(webDriver) .withTimeout(Duration.ofSeconds(30)) // Max time to wait .pollingEvery(Duration.ofMillis(500)) // How often to check .ignoring(NoSuchElementException.class) // Ignore if not there yet .ignoring(StaleElementReferenceException.class); // Ignore if React re-renders it return wait.until(driver -> driver.findElement(locator)); } </pre> |
| Observer Design Pattern for Browser Initialization | <p>Defines a one-to-many relationship between objects so that when one object changes state, all dependents are notified and updated automatically.</p> <p>Currently the browser restarts before every test and closes after every test. Browser initialization costs ~2 seconds per test. We need to make this configurable: reuse the browser when safe, restart when needed, per test class or per test method.</p> <p>KEY CHARACTERISTICS:</p> <ul style="list-style-type: none"> Loosely coupled designs between interacting objects Data is sent to many observers efficiently from a single subject New observers can be added without modifying the subject Observers can be added/removed at runtime Order of observer notification is not guaranteed <p>UML PARTICIPANTS:</p> <ul style="list-style-type: none"> TestExecutionSubject (interface): Defines attach/detach for observers and notification methods for each test lifecycle event. ExecutionSubject (concrete subject): Implements TestExecutionSubject. Maintains a list of observers. Notifies all of them at each lifecycle point. TestBehaviorObserver (interface): All observers implement this. Defines the notification method signatures. BaseTestBehaviorObserver: Default empty implementation of TestBehaviorObserver. Observers extend this and only override the lifecycle hooks they care about. BrowserLaunchTestBehaviorObserver (concrete observer): Reads @ExecutionBrowser annotations and decides whether to restart or reuse the browser. BaseTest: Parent class for all test classes. Wires subject and observers together. Fires lifecycle notifications in @BeforeMethod and @AfterMethod. |

| | |
|--------------------------|--|
| TestExecutionSubject | <pre> public interface TestExecutionSubject { void attach(TestBehaviorObserver observer); void detach(TestBehaviorObserver observer); void preTestInit(ITestResult result, Method memberInfo); void postTestInit(ITestResult result, Method memberInfo); void preTestCleanup(ITestResult result, Method memberInfo); void postTestCleanup(ITestResult result, Method memberInfo); } </pre> <p>The Subject is the object that contains the state and controls it. So, there is only one subject with a state. The observers, on the other hand, use the state, even though they don't own it. There are many observers, and they rely on the Subject to tell them when its state changes. So, there is a relationship between the one subject and the many observers.</p> <p>The methods attach and detach are used by the observer classes to associate themselves with the subject. The rest of the methods are invoked in the various steps of the test execution workflow to notify the observers about changes in the state of the subject. In order for the subject to be able to notify the observers, they need to implement the TestBehaviorObserver interface where all notification methods are defined.</p> |
| TestBehaviorObserver | <pre> public interface TestBehaviorObserver { void preTestInit(ITestResult testResult, Method memberInfo); void postTestInit(ITestResult testResult, Method memberInfo); void preTestCleanup(ITestResult testResult, Method memberInfo); void postTestCleanup(ITestResult testResult, Method memberInfo); } </pre> |
| ExecutionSubject | <p>The specific subject knows nothing about the implementations of the observers. It is working with a list of TestBehaviorObserver objects. The attach and detach methods add and remove observers to/from the collection. In this classic implementation of the observer design pattern, the observers are responsible to associate themselves with the subject class.</p> <pre> public class ExecutionSubject implements TestExecutionSubject { private final List<TestBehaviorObserver> observers = new ArrayList<>(); @Override public void attach(TestBehaviorObserver observer) { observers.add(observer); } @Override public void detach(TestBehaviorObserver observer) { observers.remove(observer); } @Override public void preTestInit(ITestResult result, Method memberInfo) { observers.forEach(o -> o.preTestInit(result, memberInfo)); } @Override public void postTestInit(ITestResult result, Method memberInfo) { observers.forEach(o -> o.postTestInit(result, memberInfo)); } @Override public void preTestCleanup(ITestResult result, Method memberInfo) { observers.forEach(o -> o.preTestCleanup(result, memberInfo)); } @Override public void postTestCleanup(ITestResult result, Method memberInfo) { observers.forEach(o -> o.postTestCleanup(result, memberInfo)); } } </pre> |
| BaseTestBehaviorObserver | <pre> public abstract class BaseTestBehaviorObserver implements TestBehaviorObserver { @Override public void preTestInit(ITestResult testResult, Method memberInfo) { // Default empty implementation } @Override public void postTestInit(ITestResult testResult, Method memberInfo) { // Default empty implementation } @Override public void preTestCleanup(ITestResult testResult, Method memberInfo) { // Default empty implementation } } </pre> |

| | | |
|-----------|---|---|
| | | <pre> @Override public void postTestCleanup(ITestResult testResult, Method memberInfo) { // Default empty implementation } </pre> |
| | BrowserLaunchTestBehaviorObserver | <pre> @Override public void preTestInit(ITestResult testResult, Method memberInfo) { BrowserConfiguration currentConfig = getBrowserConfiguration(memberInfo); currentConfigThreadLocal.set(currentConfig); BrowserConfiguration previousConfig = previousConfigThreadLocal.get(); WebDriver driver = driverSupplier.get(); boolean needsRestart = false; if (driver == null) { needsRestart = true; } else if (currentConfig.getBrowserBehavior() == BrowserBehavior.RESTART_EVERY_TIME) { needsRestart = true; } else if (previousConfig != null && currentConfig.getBrowser() != previousConfig.getBrowser()) { needsRestart = true; } if (needsRestart) { // Quit old and start new ONLY when we are about to run a test restartBrowser(currentConfig); } previousConfigThreadLocal.set(currentConfig); } </pre> |
| | ISOLATED BROWSER
INITIALIZATION FOR EACH
TEST | <p>When Test Passes: Delete all cookies after each test when the browser is being reused.
When Test Fails: Re Create the browser instance</p> <pre> public class ExecutionListener implements ITestListener { @Override public void onTestSuccess(ITestResult result) { WebDriver driver = DriverFactory.getDriver(); if (driver != null) { driver.manage().deleteAllCookies(); // The driver remains open for the next @Test } } @Override public void onTestFailure(ITestResult result) { WebDriver driver = DriverFactory.getDriver(); if (driver != null) { driver.quit(); // Force the factory to create a new instance for the next test DriverFactory.setDriver(null); } } } </pre> |
| Chapter 4 | POM Design Pattern | <p>The page object wraps all elements, actions, and validations happening on a page into a single object. Tests talk to page objects, not raw WebDriver.</p> <p>PRIMARY GOAL: Hide low-level WebDriver details from tests. Tests read as business actions. Element locators live in one place - the page object.</p> <p>Simon Stewart (Selenium project lead and PageFactory creator) himself advises against using PageFactory.
Reason: Java annotations lack extensibility, and PageFactory cannot be integrated with WebDriverWait for handling dynamic/async scenarios. The author prefers explicit findElement calls through the decorated Driver.</p> <p>POM WITHOUT PageFactory (RECOMMENDED APPROACH)</p> <p>After POM - test reads as business intent:</p> |

```

@Test(priority=1)
public void completePurchaseSuccessfully_whenNewClient() {
    mainPage.addRocketToShoppingCart();
    cartPage.applyCoupon("happybirthday");
    Assert.assertEquals(cartPage.getMessageNotification(), "Coupon code applied successfully.");
    cartPage.increaseProductQuantity(2);
    Assert.assertEquals("114.00EUR", cartPage.getTotal());
    cartPage.clickProceedToCheckout();
}

```

MainPage implementation:

```

public class MainPage {
    private final Driver driver;
    private final String url = "http://demos.bellatrix.solutions/";

    public MainPage(Driver driver) {
        this.driver = driver;
    }

    // Elements are PRIVATE - only accessed through public action methods
    private Element addToCartFalcon9() {
        return driver.findElement(By.cssSelector("[data-product_id='28']"));
    }

    private Element viewCartButton() {
        return driver.findElement(By.cssSelector("[class*='added_to_cart wc-forward']"));
    }

    public void addRocketToShoppingCart() {
        driver.goToUrl(url);
        addToCartFalcon9().click();
        viewCartButton().click();
    }
}

```

Usage in test class:

```

@BeforeMethod
public void testInit() {
    driver = new LoggingDriver(new WebCoreDriver());
    driver.start(Browser.CHROME);
    mainPage = new MainPage(driver);
    cartPage = new CartPage(driver);
}

```

Common Page
Actions

AUT has four distinct sections on the pages:

- Search (search input + search action)
- Main Menu (navigation links)
- Cart Info (cart icon + current amount)
- Breadcrumb (breadcrumb navigation - only on cart-type pages)

| | |
|------------------|--|
| SearchSection | <pre> public class SearchSection { private final Driver driver; public SearchSection(Driver driver) { this.driver = driver; } private Element searchField() { return driver.findElement(By.id("woocommerce-product-search-field-0")); } public void searchForItem(String searchText) { searchField().typeText(searchText); } } </pre> |
| MainMenu Section | <pre> public class MainMenuSection { private final Driver driver; public MainMenuSection(Driver driver) { this.driver = driver; } private Element homeLink() { return driver.findElement(By.linkText("Home")); } private Element blogLink() { return driver.findElement(By.linkText("Blog")); } private Element myAccountLink() { return driver.findElement(By.linkText("My Account")); } private Element promotionsLink() { return driver.findElement(By.linkText("Promotions")); } public void openHomePage() { homeLink().click(); } public void openBlogPage() { blogLink().click(); } public void openMyAccountPage() { myAccountLink().click(); } } </pre> |

| | |
|--------------------|--|
| | <pre> public void openPromotionsPage() { promotionsLink().click(); } } </pre> |
| CartInfo Section | <pre> public class CartInfoSection { private final Driver driver; public CartInfoSection(Driver driver) { this.driver = driver; } private Element cartIcon() { return driver.findElement(By.className("cart-contents")); } private Element cartAmount() { return driver.findElement(By.className("amount")); } public String getCurrentAmount() { return cartAmount().getText(); } public void openCart() { cartIcon().click(); } } </pre> |
| Breadcrumb Section | <pre> public class BreadcrumbSection { private final Driver driver; public BreadcrumbSection(Driver driver) { this.driver = driver; } private Element breadcrumb() { return driver.findElement(By.className("woocommerce- breadcrumb")); } public void openBreadcrumbItem(String itemToOpen) { breadcrumb().findElement(By.linkText(itemToOpen)).click(); } } </pre> |
| BaseEShopPage | <p>BaseEShopPage holds only the three sections present on ALL pages. Breadcrumb is NOT here. Only CartPage exposes it.</p> <pre> public abstract class BaseEShopPage { protected final Driver driver; // protected so child pages can use it public BaseEShopPage(Driver driver) { this.driver = driver; } // Sections present on all pages - exposed via public getters public MainMenuSection mainMenuSection() { return new MainMenuSection(driver); } public CartInfoSection cartInfoSection() { return new CartInfoSection(driver); } public SearchSection searchSection() { return new SearchSection(driver); } protected abstract String getUrl(); public void open() { driver.goToUrl(getUrl()); } } MainPage - only the three shared sections via base class: public class MainPage extends BaseEShopPage { public MainPage(Driver driver) { super(driver); } @Override protected String getUrl() { return "http://demos.bellatrix.solutions/"; } private Element addToCartFalcon9() { return driver.findElement(By.cssSelector("[data-product_id='28']")); } private Element viewCartButton() { return driver.findElement(By.cssSelector("[class*='added_to_cart wc- forward']")); } public void addRocketToShoppingCart() { open(); addToCartFalcon9().click(); viewCartButton().click(); } } CartPage - inherits shared sections, adds breadcrumb only here: public class CartPage extends BaseEShopPage { public CartPage(Driver driver) { super(driver); } @Override protected String getUrl() { return "http://demos.bellatrix.solutions/cart/"; } // Breadcrumb only accessible from CartPage - not in base class public BreadcrumbSection breadcrumbSection() { return new BreadcrumbSection(driver); } // Cart-specific private elements private Element couponCodeTextField() { return driver.findElement(By.id("coupon_code")); } private Element applyCouponButton() { return driver.findElement(By.cssSelector("[value*='Apply coupon']")); } private Element quantityBox() { return driver.findElement(By.cssSelector("[class*='input-text qty text']")); } private Element updateCart() { return driver.findElement(By.cssSelector("[value*='Update cart']")); } } </pre> |

| | |
|----------------------------|---|
| | <pre> } private Element messageAlert() { return driver.findElement(By.cssSelector("[class*='woocommerce-message']")); } private Element totalSpan() { return driver.findElement(By.xpath("//*[@class='order-total']//span")); } private Element proceedToCheckout() { return driver.findElement(By.cssSelector("[class*='checkout-button button alt wc-forward']")); } public void applyCoupon(String coupon) { couponCodeTextField().typeText(coupon); applyCouponButton().click(); driver.waitForAjax(); } public void increaseProductQuantity(int newQuantity) { quantityBox().typeText(String.valueOf(newQuantity)); updateCart().click(); driver.waitForAjax(); } public void clickProceedToCheckout() { proceedToCheckout().click(); driver.waitUntilPageLoadsCompletely(); } public String getTotal() { return totalSpan().getText(); } public String getMessageNotification() { return messageAlert().getText(); } } --- USING SECTIONS IN TESTS --- @BeforeMethod public void testInit() { driver = new LoggingDriver(new WebCoreDriver()); driver.start(Browser.CHROME); mainPage = new MainPage(driver); cartPage = new CartPage(driver); mainPage.open(); } @Test public void openBlogPage() { mainPage.mainMenuSection().openBlogPage(); } @Test public void searchForItem() { mainPage.searchSection().searchForItem("Falcon 9"); } @Test public void openCart() { mainPage.cartInfoSection().openCart(); } </pre> |
| COMPOSITION VS INHERITANCE | <p>READABILITY WIN: <code>mainPage.mainMenuSection().openBlogPage()</code> tells you exactly where on the page the action happens. Tests read like user intent.</p> <p>COMPOSITION VS INHERITANCE FOR SECTIONS:
 Using section getters (composition) is better than putting all section elements in a base class (inheritance) because:</p> <ul style="list-style-type: none"> - Composition enforces access control: breadcrumb is only on CartPage, not all pages - Inheritance pollutes all child classes with methods that do not apply to them - Composition is more flexible - sections can be reused across unrelated pages - Follows ISP: pages only expose the sections they actually have <p>INTERVIEW POINT: "Why sections through composition instead of inheritance?"
 Inheritance for sharing UI sections allows calling methods for elements that do not exist on a given page. Composition scopes access correctly.</p> |
| MEANINGFUL NAMES | <p>A big part of readable code is communicating intent without relying on comments.
 Names should answer: What does this class do? What is this variable for?
 What does this method return? Comments go stale; readable names do not.</p> <p>--- GENERAL NAMING GUIDELINES ---</p> <p>Always use English. Avoid abbreviations.</p> <p>Incorrect: <code>buttonsCnt</code>, <code>usrLt</code>, <code>cdCrdsNums</code>, <code>dtenRegExPtrn</code>
 Correct: <code>buttonsCount</code>, <code>usersList</code>, <code>creditCardNumbers</code>, <code>dateTimeEnglishRegExPattern</code></p> <p>Prefer meaningful over short. Names that require looking up or asking colleagues are bad names. The meaning should be immediately obvious.</p> <p>Correct: <code>DiscountsCalculator</code>, <code>buttonsCount</code>, <code>Math.PI</code>, <code>fileNames</code>, <code>generateInvoice</code>
 Incorrect: <code>d</code>, <code>d8</code>, <code>d6</code>, <code>u33</code>, <code>HCI</code>, <code>anchor3</code>, <code>variable</code>, <code>temp</code>, <code>tmp</code>, <code>something</code></p> <p>Context matters. A method named <code>generate()</code> in class <code>DiscountsGenerator</code> is fine.</p> |

| | |
|---|--|
| | <p>The same generate() in class Program tells you nothing.</p> <p>--- NAMING CLASSES ---</p> <p>Use UpperCamelCase. Classes are nouns or noun phrases.
 Do NOT use Hungarian notation (bBusy for Boolean, iSize for Integer).
 Do NOT use underscores, hyphens, or non-alphanumeric characters.
 Do NOT use abbreviations or acronyms unless universally accepted.
 Use getWindow not getWin.</p> <p>Correct: SalesOrdersInvoiceCreator, SuggestedOrdersInvoiceCreator
 Incorrect: InvoiceCreator1, InvoiceCreator2 (meaningless numbering)</p> <p>Unit test classes: name starts with the class being tested, ends with Test(s).
 Correct: OrderCalculatorTests, InvoiceCreatorTests
 Incorrect: Orders, ScreenshotCapture</p> <p>UI/system test classes: start with a short description of the context covered.
 Correct: ProductPurchaseWithoutPageObjectsTests, SearchEngineTests
 Incorrect: PerformSearches, MakeOrders</p> <p>Exception classes: always end with suffix Exception.
 Correct: NumberFormatException, StaleElementException
 Incorrect: ElementNotFound, StaleElement</p> <p>Name length: as long as required. Prefer readability over brevity. IDEs handle long names.
 Correct: createOrderWithActiveCustomerViaShoppingCart, ClientSupportInvoiceService
 Incorrect: createOrdActiveCustViaSC, CustSprlnvSrvc</p> <p>--- NAMING NEW VERSIONS OF EXISTING API ---</p> <p>When a new version of a class must coexist with the old one:</p> <ul style="list-style-type: none"> - Prefer suffix over prefix to differentiate (NewClass vs ClassNew). - Numbers as suffix are acceptable when no meaningful alternative exists (e.g., X509Certificate and X509Certificate2). - Do NOT delete old API immediately. Mark it @Deprecated with migration guidance. <p>Example from official WebDriver Java bindings:</p> <pre>/** * @deprecated Use {@link #implicitlyWait(Duration)} * Specifies the amount of time the driver should wait when searching for an element. * @param time The amount of time to wait. * @param unit The unit of measure for {@code time}. */ @Deprecated Timeouts.implicitlyWait(Long time, TimeUnit unit);</pre> <p>INTERVIEW POINT: Deprecation gives users time to migrate. Deleting API immediately breaks downstream consumers. Always use @Deprecated + Javadoc explanation first.</p> |
| <p>NAMING THE METHODS THE RIGHT WAY</p> | <p>Methods perform actions. Their names must include verbs or verb phrases.
 If you cannot answer "What does this method do?" the method likely does more than one thing and violates SRP. Split it.</p> <p>Correct: findItem, loadPage, placePurchase, createInvoice
 Incorrect: method1, doSomething, handleStuff, sampleMethod, somewhere</p> <p>Preferred format: [Verb], [Verb]+[Noun], [Verb]+[Adjective]+[Noun]
 Correct: generate, loadConfigurationFile, findItemByPattern, toString, printItemsList
 Incorrect: student, creator, counter, black, mathUtilities</p> <p>Do NOT use underscores in production method names.
 Exception: test method names (underscores are acceptable there for readability).</p> <p>Correct (production):
 public void performActionKeys(Keyboard keyboard, String keysToSend) { }</p> <p>Incorrect (production):
 public void perform_Action_Keys(Keyboard keyboard, String keysToSend) { }</p> <p>--- MEANINGFUL METHOD NAMES ---</p> <p>Correct: int count = getCountById(client, itemId);</p> |

Incorrect: `getAllByItemCount(client, itemCount); // ambiguous intent`

Length: as long as needed. No abbreviations.

Correct: `loadClientSupportInvoiceService, createAnnualDiscountsReport`

Incorrect: `loadClnSupplnSvc, createMonthDiscReport`

--- BEST PRACTICES FOR METHOD PARAMETERS ---

Keep parameter lists short. More than 4 parameters - consider grouping them into a dedicated class and passing that as a single parameter.

Correct (acceptable length):

```
public void createInvoiceReport(  
    LocalDate startDate,  
    LocalDate endDate,  
    Integer limit,  
    Boolean isForExistingClients) { }
```

Incorrect (too many parameters):

```
public void createInvoiceReport(  
    LocalDate startDate,  
    LocalDateTime endDate,  
    Integer limit,  
    Boolean isForExistingClients,  
    Boolean isForMonthlyBucket,  
    Double requestedAmount,  
    Double dependentClientAmount,  
    Double futureCustomerDemand, ...) { }
```

Parameter naming format: [Noun] or [Adjective]+[Noun]. Use camelCase.

Include unit in the name when it adds clarity.

Correct: `lastName, reportName, supportTicket, recurringBillingOrder, timeInSeconds`

Incorrect: `d, d1, d2, generate, FirstName, first_name, convertFormat`

| | | |
|-----------|---------------------------|---|
| Chapter 1 | Unit Tests | <ul style="list-style-type: none"> - Smallest test units. Test individual objects and methods. - Rarely use real data. Rely on generated/mock data. |
| | Integration Tests | <ul style="list-style-type: none"> - Multiple modules working together passing data between each other. - Third-party services (network, filesystem, OS) are still isolated. |
| | System Tests | <ul style="list-style-type: none"> - Testing a complete, fully integrated software product. Highest level. - Executed against PROD or PROD-like environments (e.g., staging). |
| | UI Tests | All actions and assertions are performed against the user interface. |
| | End-to-end Tests | Verify a real user scenario from start to finish. |
| | System Tests | <p>Uses internal APIs or DB directly for setup (login, user creation) instead of going through UI every time.</p> <p>Calling a test that uses API shortcuts a "UI test" is incorrect. A UI test must perform ALL actions through the UI.</p> |
| | Test Automation Framework | <ul style="list-style-type: none"> • Application: Software created to execute specific functions for users. Automated tests are also applications. Test engineers should treat test code as production code, following the same quality standards. • Software Library: Pre-written code (classes, methods, interfaces, constants) that other programs can use without redefining. • API: The group of methods giving access to features of an application or service. The API is the specification (what you can do). The library is the implementation. Multiple implementations can satisfy the same API (e.g., Java and Scala both implement the Java API and compile to bytecode). • Library: Used to build your app. You call it. Cannot be modified. • Framework: Generic/abstract. Controls the flow (Inversion of Control). Provides default behavior. You customize parts of it. You cannot change its core code. Examples: ASP.NET, Android Application Framework, Cocoa. • Test Automation Framework: Consists of guidelines, coding standards, concepts, processes, practices, project hierarchies, modularity, etc. to support automated testing. It is NOT just a collection of helper methods. |
| | SOLID | |
| | SRP | <p>Every module should have only one reason to change. A class can do many things, but they must all serve a single purpose.</p> <p>Violation example - CustomerOrder class doing logging in its catch block:</p> <pre> public void create() { try { // DB logic } catch (Exception ex) { Path path = Paths.get("C:\\exception.txt"); BufferedWriter writer = Files.newBufferedWriter(path); writer.write(ex.getLocalizedMessage()); } } </pre> <p>Fix - extract logging into a dedicated class:</p> <pre> public class FileLogger { public void createLogEntry(String error) { Path path = Paths.get("C:\\errors.txt"); BufferedWriter writer = Files.newBufferedWriter(path); writer.write(error); } } public class CustomerOrder { private final FileLogger fileLogger = new FileLogger(); public void create() { try { // DB logic } catch (Exception ex) { fileLogger.createLogEntry(ex.getLocalizedMessage()); } } } </pre> |
| | OCP | <p>Software entities should be open for extension but closed for modification. New requirements should be added via new code, not by modifying tested code.</p> <p>Violation - adding new order types requires modifying DiscountCalculator:</p> <pre> public double calculateDiscount(OrderType orderType, double totalPrice) { if (orderType == OrderType.Silver) { return totalPrice - 20; } else { return totalPrice; } } </pre> <p>Fix - each order type gets its own class extending the base:</p> <pre> public abstract class DiscountCalculator { public double calculateDiscount(double totalPrice) { return totalPrice; } } public class SilverDiscountCalculator extends DiscountCalculator { @Override public double calculateDiscount(double totalPrice) { return super.calculateDiscount(totalPrice) - 20; } } public class GoldDiscountCalculator extends DiscountCalculator { </pre> |


```

@Override
public double calculateDiscount(double totalPrice) {
    return super.calculateDiscount(totalPrice) - 50;
}
}

```

Adding a new order type = new class only. No existing code touched.

LSP You should be able to use any derived class in place of a parent class and have it behave in the same manner without modification. Child classes must not change the expected behavior of parent methods.

Violation - PlatinumDiscountCalculator throws an exception for a method it inherits but cannot support:

```

public class PlatinumDiscountCalculator extends DiscountCalculator {
    @Override
    public double calculateBonusPointsDiscount(double totalPrice, int points) {
        throw new NoSuchMethodException("Not applicable for Platinum orders.");
    }
}

```

```

// Fails at runtime when iterated polymorphically:
List<DiscountCalculator> calculators = new ArrayList<>();
calculators.add(new PlatinumDiscountCalculator());
for (var calc : calculators) {
    calc.calculateBonusPointsDiscount(1250, 10); // throws exception
}

```

Fix - split into separate interfaces so Platinum never implements what it cannot support:

```

public interface RegularDiscountCalculator {
    double calculateRegularDiscount(double totalPrice);
}

```

```

public interface BonusPointsDiscountCalculator {
    double calculateBonusPointsDiscount(double totalPrice, int points);
}

```

```

// Only Silver and Gold implement BonusPointsDiscountCalculator.
// PlatinumDiscountCalculator only implements RegularDiscountCalculator.
List<BonusPointsDiscountCalculator> calculators = new ArrayList<>();
calculators.add(new SilverDiscountCalculator());
calculators.add(new GoldDiscountCalculator());
// PlatinumDiscountCalculator cannot be added here - compile-time safety.

```

ISP Clients should not be forced to implement interfaces they don't use. Prefer many small, focused interfaces over one fat interface. Applies SRP to interfaces: each interface serves one submodule.

Violation - one fat interface forces Platinum to implement bonus points even though it doesn't support it:

```

public interface DiscountCalculator {
    double calculateRegularDiscount(double totalPrice);
    double calculateBonusPointsDiscount(double totalPrice, int points);
}

```

```

// PlatinumDiscountCalculator is forced to implement calculateBonusPointsDiscount
// and has to throw an exception - bad design.

```

Fix - split into focused interfaces (same as shown in LSP fix above).
Each class implements only the interfaces that match its capabilities.

INTERVIEW POINT: ISP and LSP are closely related. A fat interface often forces LSP violations. Splitting interfaces fixes both.

DIP High-level classes should not depend on low-level classes. Both should depend on abstractions. Details should depend on abstractions.

High-level = business logic. Low-level = DB operations, file I/O, API calls.
Tightly coupled = a class directly instantiates its dependencies (using "new").

Violation - CustomerOrder decides which logger to create inside its method:

```

public void create(OrderType orderType) {
    try {
        // DB logic
    } catch (Exception ex) {
        switch (orderType) {
            case GOLD: new EmailLogger().createLogEntry(ex.getMessage()); break;
            case PLATINUM: new FileLogger().createLogEntry(ex.getMessage()); break;
        }
    }
}

```

Fix - define a Logger abstraction, inject it via constructor:

```

public interface Logger {
    void createLogEntry(String errorMessage);
}

```

```

public class CustomerOrder {
    private final Logger logger;
    public CustomerOrder(Logger logger) {
        this.logger = logger;
    }
    public void create() {
        try {
            // DB logic
        } catch (Exception ex) {

```

| | | |
|--------------------|---|---|
| | | <pre> logger.createLogEntry(ex.getLocalizedMessage()); } } public class GoldCustomerOrder extends CustomerOrder { public GoldCustomerOrder() { super(new EmailLogger()); } } public class PlatinumCustomerOrder extends CustomerOrder { public PlatinumCustomerOrder() { super(new SmsLogger()); } } </pre> <p>CustomerOrder now depends on the Logger abstraction, not any concrete implementation.
In production, IoC containers handle the injection automatically.</p> |
| High Quality Tests | <p>Design Pattern: Prescribed solutions to everyday software challenges. Not code or algorithms, they describe how to structure, group and reuse logic to make it maintainable. It's a template for solving design problems.</p> <p>The Five High-Quality Test Attributes:</p> <ul style="list-style-type: none"> • Maintainability: Ease with which the solution can be changed to accommodate new requirements or fix bugs. Poor maintainability means changes must be made in multiple places, increasing the risk of missing some and introducing more bugs. Connected to: DRY principle (Don't Repeat Yourself). DRY is also known as Single Source of Truth (SSOT) - every unique piece of information exists in exactly one place. • Readability: By reading the code you should immediately understand what it does. Unreadable code takes longer to maintain and is more prone to bugs. Good naming of variables, methods, and classes eliminates the need for excessive comments. Comments go stale; readable code does not. • API Usability: How easy is it for a user to find out what the methods do and how to use them? <ul style="list-style-type: none"> ◦ For a test library: "How much time does a new user need to write a new test?" ◦ Related term: syntactic sugar - expressions that make code more concise and human-readable. Good API usability means the framework reads like plain language. • Extensibility: The ability to customize parts of the framework for different teams and contexts without modifying core code. A shared library must work across multiple teams with different needs. Engineers should be able to extend behavior without breaking existing tests. Closely connected to: OCP (Open/Closed Principle). • Learning Curve: "How easy is it for a new team member to learn to write or maintain tests independently?" Tightly coupled to API usability but focused on onboarding. If new members need constant mentoring to use the framework, the learning curve is too steep. The goal: self-service learning through code clarity. | |
| Chapter 2 | Core Components of Framework | <ul style="list-style-type: none"> • TEST SCRIPTS: Core executable instructions that verify features work as expected. <ul style="list-style-type: none"> • Key attributes <ul style="list-style-type: none"> ◦ Modularisation: break into smaller reusable modules to reduce duplication ◦ Parametrization: same script runs multiple times with different input datasets, critical for covering various scenarios • LIBRARIES: Collections of prebuilt functions and methods that support test script creation. Abstracts complex operations into reusable components. <ul style="list-style-type: none"> • Three categories <ul style="list-style-type: none"> ◦ Reusable methods: common actions like logging in, navigating, stored centrally ◦ Custom libraries: projectspecific actions shared across test cases ◦ Thirdparty libraries: HTTP requests, data handling, advanced reporting • TEST DATA: Inputs, values, and parameters fed into test scripts during execution. In a robust framework, test data is externalised from test scripts. <ul style="list-style-type: none"> • Key practices <ul style="list-style-type: none"> ◦ Datadriven testing: run scripts with multiple data sets (CSV, JSON, XML, DB) ◦ Separation of data from test logic: changing test data does not touch test scripts • TEST RESULTS: Detailed feedback on the outcome of automated tests. A well structured framework generates reports tracking passed, failed, skipped. <ul style="list-style-type: none"> • Key features <ul style="list-style-type: none"> ◦ Comprehensive reporting: builtin tools or external integrations for visual reports (pie charts, failure logs, traceability matrices) ◦ Traceability: detailed logging to trace root causes of failures • TEST RUNNERS: Tools responsible for executing test scripts. Manage setup, execution, and teardown processes. <ul style="list-style-type: none"> • Capabilities <ul style="list-style-type: none"> ◦ Control execution order, filter tests by tags ◦ Configure parallel or sequential execution ◦ Integrate with CI systems to automate test execution as part of the build • REPORTING TOOLS: Gather and present automated test results. Provide visualisations, statistics, and detailed logs. <ul style="list-style-type: none"> • Capabilities <ul style="list-style-type: none"> ◦ Custom reports tailored to team needs (pass/fail ratios, execution time, errors) ◦ Dashboard integration: Jenkins or custom monitoring systems for continuous feedback • CI SYSTEMS: Enable automated tests to run continuously as part of the software delivery pipeline. Tools: Jenkins, GitLab, Azure DevOps, CircleCI, Travis CI. <ul style="list-style-type: none"> • Two key functions <ul style="list-style-type: none"> ◦ Automation of build and test: every code change triggers test execution automatically ◦ Regression testing: ensures new changes do not break existing functionality |
| | THE TESTING PYRAMID | <p>The testing pyramid defines how automation effort should be distributed. Majority of code coverage at unit level. E2E and UI tests at the top (fewer, slower).</p> <ul style="list-style-type: none"> • UNIT TESTS (base of pyramid): Fast, lowlevel, validate individual functions in isolation. Highly automated, immediate feedback to developers. • INTEGRATION TESTS (middle): Verify multiple components, services, or modules work together. Focus on API interactions, databases, thirdparty services. Slower than unit tests but essential for system coherence. • ENDTOEND TESTS (top): Simulate real user workflows. Validate the entire application front to back. High confidence but slow and resourceintensive. Use sparingly alongside unit and integration tests. |
| | BALANCING COVERAGE AND EXECUTION TIME | <ul style="list-style-type: none"> • Prioritise critical user journeys: Not every feature needs E2E coverage. Focus on workflows where failure has the greatest impact (payment processing, account registration, orderfulfillment). • Layered testing approach: unit and integration tests cover most edge cases and lowlevel functionality. E2E tests focus only on what cannot be validated in isolation. • Selective E2E test suites <ul style="list-style-type: none"> • Smoke tests: subset of E2E covering most critical workflows. Fast. Run on every deployment. Example: can users log in, can transactions complete. |

| | | |
|-----------|------------------------------------|--|
| | | <ul style="list-style-type: none"> • Full regression tests: broader E2E suite. Run less frequently nightly or before major releases. • Do NOT aim for 100% E2E coverage: impractical. Focus on sufficient coverage for critical workflows while balancing time and reliability constraints. • Parallelisation: run E2E tests in parallel across multiple VMs, containers, or browser instances. • FLAKINESS: Flaky tests = tests that fail intermittently due to timing, environment, or network instability rather than actual application bugs. Flaky tests undermine E2E value. <ul style="list-style-type: none"> • Fixes: Use proper wait mechanisms for asynchronous events (page loads, API responses). Avoid hardcoded wait times. Use explicit waits (wait until element visible). • Ensure test environment stability closely mimic production conditions • Maintenance costs: <ul style="list-style-type: none"> • Use POM to decouple test scripts from UI elements • Keep tests focused on user workflows, not internal implementation details • Avoid frequent changes due to minor UI updates • Environment stability: E2E tests depend on a fully functional environment (backend, DB, thirdparty APIs). Invest in reliable, productionlike test environments or use mock service to improve reliability and reduce false negatives |
| | TRAITS OF A WELLDESIGNED FRAMEWORK | <p>REUSABILITY AND MAINTAINABILITY</p> <ul style="list-style-type: none"> • Code reusability across test cases: Common functions (login, setup data, UI interactions) abstracted into reusable modules. Written once, reused everywhere. Reduces duplication. Faster test creation. • Reduction in redundancy: Centralising key actions (API calls, DB queries, form submissions) into reusable libraries cuts redundancy and makes the framework easier to maintain. • Consistency in test implementation: Standardised structure, format, and naming conventions. Easier to understand, update, and manage. Simplifies onboarding. • Scalability in test creation: Modular and reusable framework allows rapid development of new test cases. Add or modify by reusing existing components no need to start from scratch. <p>IMPORTANCE OF MODULAR AND DECOUPLED TESTS</p> <ul style="list-style-type: none"> • Decoupling of test logic from data and configuration: Externalise test data to JSON/XML/CSV files. Environment specific settings in config files. Same tests run across DEV, QA, staging without modifying test logic. • Modularity for ease of updates: Break test cases into smaller, single responsibility components. When a feature changes, only the corresponding module needs updating not all tests using it. • POM as modularity in practice: Encapsulates UI element interactions in page classes. When UI component changes, only that page object needs updating. All tests using it are automatically fixed. • Separation of concerns: Different layers of test architecture execution, data, application logic are separated. Changes in one layer have minimal impact on others. <p>BEST PRACTICES FOR REUSABILITY AND MAINTAINABILITY:</p> <ul style="list-style-type: none"> • Follow SOLID principles: Single Responsibility ensures each test class/method does one thing. Open/Closed ensures tests can be extended without modification. • Implement design patterns: POM for UI testing, factory pattern for object creation. Reduces direct dependencies. Allows test logic to evolve independently. • Automate test data management: externalise data. Use data driven testing (DDT) and keyword driven testing. Vary datasets without modifying test logic. • Regularly refactor test scripts: like production code, test scripts accumulate technical debt. Ongoing refactoring removes redundancy and improves readability. • Centralise test configuration: environment specific settings in one place. Reduces maintenance when updating environments. Scales easily. • Use version control and CI/CD: test automation scripts version controlled alongside application code. Track changes, enable collaboration, integrate into CI/CD pipelines. |
| Chapter 3 | Core Thesis | Test automation cannot be separated from the architecture of the systems it tests. Whether you are designing a framework to capitalise on an application's design, or advocating for testable application architecture, the two work hand in hand. You cannot build a successful automation framework without a strong understanding of software architecture principles. |
| | Architecture Principles | <ul style="list-style-type: none"> • SEPARATION OF CONCERNS: This principle promotes the idea that a system should be divided into distinct modules, each addressing a specific feature or functionality. This separation allows for easier maintenance, scalability, and understanding of the system. • MODULARITY: Software should be composed of modular components that can be developed, tested, and maintained independently. This principle facilitates easier updates and enhancements while minimizing the impact on the overall system. • REUSABILITY: Encourage reuse of components across different areas. Reduces redundancy, increases efficiency, and accelerates development cycles. • SCALABILITY: Architecture must accommodate growth. A well-architected system handles increased loads without significant modifications. • FLEXIBILITY AND EXTENSIBILITY: Allow changes and enhancements without extensive rework. Critical in a rapidly changing technological landscape. |
| | Factory Pattern | <p>A creational design pattern providing an interface for creating objects without specifying the exact class to be made. Variations: Simple Factory, Factory Method, Abstract Factory.</p> <p>Automation Use Cases</p> <ul style="list-style-type: none"> • Test case creation: Factory class responsible for instantiating different types of test cases (unit, integration, UI). Decouples test case creation from test execution. Cleaner code. • Dynamic test data generation: Data factory instantiates data objects based on specific requirements (data types, structures, input conditions). Consistent, relevant data without hard-coded values. • Parameterised tests: factory generates different test instances with specific parameters. Comprehensive testing of functions with multiple input scenarios. <p>Benefits:</p> <ul style="list-style-type: none"> • Consistency: Test instances created uniformly. Centralised instantiation enforces standard approach across test suites. • Enhanced maintainability: Changes to instantiation logic made in one location. Reduces risk of errors, simplifies maintenance. • Decoupled design: Test creation separated from test execution. Modify either without affecting the other. • Increased flexibility: Create different types of tests or data on demand. New testing requirements accommodated by updating the factory. • Improved readability: encapsulated object creation makes intent clearer. • Simplified expansion: add new test types by extending the factory. No disruption to existing framework. |
| | Observer Pattern | <p>A behavioural pattern where a subject broadcasts state changes to multiple observers. In test automation, it decouples test lifecycle events from side effects like logging, reporting, and notifications.</p> <ul style="list-style-type: none"> • Subject → TestManager • Observer Interface → TestObserver • Event Type → TestEvent (enum) • Observers → Logging, Dashboard, Notification <pre>import java.util.*; // Strongly typed events enum TestEvent { TEST_STARTED, TEST_COMPLETED, TEST_FAILED } // Observer contract interface TestObserver {</pre> |

```

        void onEvent(TestEvent event, String data);
    }
    // Subject
    class TestManager {
        private final List<TestObserver> observers = new ArrayList<>();
        public void registerObserver(TestObserver observer) {
            observers.add(observer);
        }
        public void notifyObservers(TestEvent event, String data) {
            for (TestObserver observer : observers) {
                observer.onEvent(event, data);
            }
        }
        public void executeTest(String testName) {
            notifyObservers(TestEvent.TEST_STARTED, testName);
            // simulate failure
            notifyObservers(TestEvent.TEST_FAILED, testName);
            notifyObservers(TestEvent.TEST_COMPLETED, testName);
        }
    }
    // Concrete Observers
    class LoggingObserver implements TestObserver {
        public void onEvent(TestEvent event, String data) {
            System.out.println("[LOG] " + event + " -> " + data);
        }
    }
    class DashboardObserver implements TestObserver {
        public void onEvent(TestEvent event, String data) {
            if (event == TestEvent.TEST_COMPLETED) {
                System.out.println("[DASHBOARD] Updated with " + data);
            }
        }
    }
    class SlackObserver implements TestObserver {
        public void onEvent(TestEvent event, String data) {
            if (event == TestEvent.TEST_FAILED) {
                System.out.println("[SLACK] Alert: Test failed -> " + data);
            }
        }
    }
    // Wiring (important improvement)
    class Main {
        public static void main(String[] args) {
            TestManager manager = new TestManager();
            manager.registerObserver(new LoggingObserver());
            manager.registerObserver(new DashboardObserver());
            manager.registerObserver(new SlackObserver());
            manager.executeTest("LoginTest");
        }
    }

```

| Event | Observer | Action |
|----------------|-------------|---------------|
| TEST_STARTED | Logger | Start time |
| TEST_FAILED | Slack/Email | Alert |
| TEST_COMPLETED | Dashboard | Update report |

Strategy Pattern

Encapsulates interchangeable **execution algorithms** and allows switching **at runtime without modifying client code**.

```

// Strategy
interface ExecutionStrategy {
    void execute(String suite);
}
// Implementations
class LocalStrategy implements ExecutionStrategy {
    public void execute(String suite) {
        System.out.println("Running locally: " + suite);
    }
}
class RemoteStrategy implements ExecutionStrategy {
    public void execute(String suite) {
        System.out.println("Running on remote grid: " + suite);
    }
}
class CloudStrategy implements ExecutionStrategy {
    public void execute(String suite) {
        System.out.println("Running on cloud: " + suite);
    }
}
// Factory for strategy selection
class StrategyFactory {
    public static ExecutionStrategy getStrategy() {
        String env = System.getenv("EXECUTION_ENV");
        if ("REMOTE".equalsIgnoreCase(env)) return new RemoteStrategy();
        if ("CLOUD".equalsIgnoreCase(env)) return new CloudStrategy();
        return new LocalStrategy();
    }
}
// Context
class TestExecutor {
    private final ExecutionStrategy strategy;
    public TestExecutor(ExecutionStrategy strategy) {
        this.strategy = strategy;
    }
    public void run(String suite) {
        strategy.execute(suite);
    }
}
// Usage
class Main {

```

```

    public static void main(String[] args) {
        ExecutionStrategy strategy = StrategyFactory.getStrategy();
        TestExecutor executor = new TestExecutor(strategy);
        executor.run("RegressionSuite");
    }
}

```

| Environment | Strategy |
|------------------|-------------|
| DEV | Local |
| CI | Remote Grid |
| Release Pipeline | Cloud |

Decorator

Dynamically adds responsibilities to objects **without modifying their class**.

```

// Component
interface TestCase {
    void execute();
}

// Core logic
class BaseTestCase implements TestCase {
    public void execute() {
        System.out.println("Executing core test");
    }
}

// Decorator base
abstract class TestDecorator implements TestCase {
    protected TestCase wrapped;
    public TestDecorator(TestCase wrapped) {
        this.wrapped = wrapped;
    }
}

// Logging
class LoggingDecorator extends TestDecorator {
    public LoggingDecorator(TestCase wrapped) {
        super(wrapped);
    }
    public void execute() {
        System.out.println("Start log");
        wrapped.execute();
        System.out.println("End log");
    }
}

// Retry (real-world useful)
class RetryDecorator extends TestDecorator {
    private final int maxAttempts;
    public RetryDecorator(TestCase wrapped, int maxAttempts) {
        super(wrapped);
        this.maxAttempts = maxAttempts;
    }
    public void execute() {
        int attempts = 0;
        while (attempts < maxAttempts) {
            try {
                wrapped.execute();
                return;
            } catch (Exception e) {
                attempts++;
                System.out.println("Retry attempt: " + attempts);
            }
        }
        throw new RuntimeException("Test failed after retries");
    }
}

// Usage
class Main {
    public static void main(String[] args) {
        TestCase test =
            new RetryDecorator(
                new LoggingDecorator(
                    new BaseTestCase()
                ),
                3
            );
        test.execute();
    }
}

```

| Concern | Decorator |
|---------|------------------------|
| Logging | LoggingDecorator |
| Retry | RetryDecorator |
| Setup | PreconditionDecorator |
| Cleanup | PostconditionDecorator |

Chapter 4

- MODULARITY IS THE FOUNDATION:
 - Divides testing into smaller, independent, reusable modules. Each module
 - has one responsibility. Changes in one module do not affect others.
 - Enables flexibility, adaptability, and ease of updating.
- THREE-WAY SEPARATION IS NON-NEGOTIABLE:
 - Test logic (what to test), test data (inputs), configuration (environment
 - settings) must be separate. This decision alone determines long-term
 - maintainability. Never hardcode any of the three into test scripts.
- FIVE TEST DATA FORMATS WITH CLEAR USE CASES:
 - CSV for simple flat data. JSON for hierarchical/API data. Database for
 - large relational data. Excel for business-managed data. Parameterised
 - for small static datasets in code. Knowing when to use each is critical.
- CONFIGURATION MANAGEMENT PROTECTS AGAINST ENVIRONMENT FRAGILITY:

- YAML, JSON, Properties, INI for config. Environment variables for secrets.
- Secret management tools (Vault, AWS Secrets Manager) for production.
- Never commit credentials to source control. Runtime environment selection
- via command-line parameter, no code changes needed.
- REUSABLE COMPONENTS ELIMINATE DUPLICATION:
 - Parameterised utility functions for login, API calls, DB queries, UI actions.
 - Utility library structure (/utils with api.py, db.py, ui_actions.py).
 - Generic functions with error handling and retry logic.
- POM IS THE PRIMARY UI TEST PATTERN:
 - Each page = one class encapsulating all elements and actions.
 - Tests call page object methods, never reference locators directly.
 - Navigation between pages returns next page object.
 - One locator change = one file update.
- HOOKS AND SETUP/TEARDOWN ENSURE TEST ISOLATION:
 - Setup creates required state. Teardown restores clean state.
 - try/finally guarantees teardown runs even when tests fail.
 - pytest fixtures are the standard mechanism in Python.
 - Prevents data corruption and cascading test failures.
- CUSTOM ASSERTION LIBRARIES IMPROVE READABILITY AND CONSISTENCY:
 - Centralise validation logic. Encapsulate complex multi-step checks.
 - Descriptive failure messages. Test the assertion functions themselves.
 - CustomAssertions class or AssertionLibrary class as central access points.
- VERSION CONTROL + CI/CD COMPLETES THE LOOP:
 - Every reusable component tracked in Git. PR reviews for standards.
 - CI validates all components on every commit. CD deploys immediately.
 - Monitoring reports on test results, coverage, and build status.
- DOCUMENTATION AND STANDARDS ARE MAINTENANCE INFRASTRUCTURE:
 - Docstrings for every reusable function. Explain the WHY in comments.
 - Automated formatters and linters enforced in CI. Directory structure
 - documented and consistently applied. New team members self-serve.